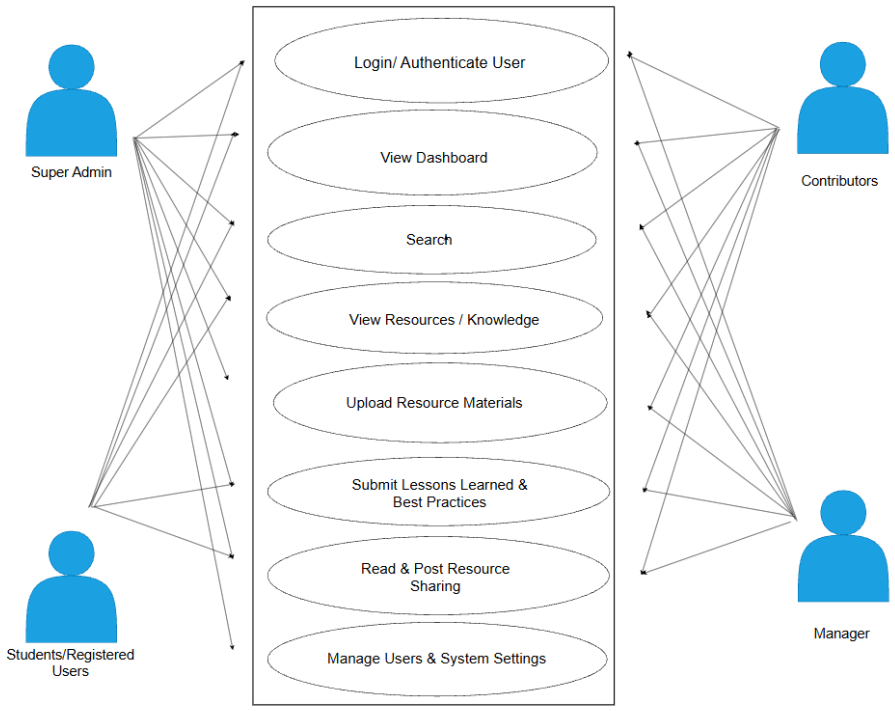
FINAL PROJECT DOCUMENTATION	
NAME	**Jester A. Patriarca**
PROJECT TITLE	**LearnLink: Student Resource Management System**
SUBJECT: **CODE:** **TIME:**	**IT15/L Integrative Programming and Technologies** **8441** **10:00AM – 12:00PM**
TOPIC (Type of Business Process)	**#4 Knowledge Management System**
Products/Services	**Educational Institutions/ Junior High School**
Website/Deployed (Link)	**https://learnlink.runasp.net/**
API/ALGO/MODEL:	**Google OAuth 2.0, Google Docs/Drive API (For Resource Viewing) , Google Books API, Open Library API, KNN (K-Nearest Neighbors), Bootstrap 5**
Security Features	1. ASP.NET Identity & Password Hashing, 2. Role-Based Authorization (Authorize Resources), 3. Multi-Tenancy Isolation (School_Id Filtering), 4. Resource Access Control 5.Password Reset via Secure Email Token
Target user/s:	Administrators, Students/Authorized Users, Teachers/Staff
Subs System / Management Transaction/Modules	1. Knowledge Repository Module This is where all educational knowledge resources are uploaded, organized, and maintained. It allows contributors to store learning materials, sheets, and reviewers, ensuring that information is easy to access, retrieve, and manage based on user roles. 2. Policy and Procedure Management Module This is where the contributor's set policy and procedure in these resources, like having the users to access to updated versions of each file, limited access for a short period of time, checking the history of the changes of the files. Also, for maintaining and checking the longevity or version control of each educational material (pdf's, reviewers, sheets). 3. Lesson Learned Management Module In this module, this is where the users or registered users can input their ratings, comments, ideas, learning's, and insights regarding on the educational files that they are reading. Also, the system will manage their reading history so that it they can go back to read and continue. 4. Best Practices Module

	This is where the system recommends both users such as contributors and users/registered users. In the contributors the system will suggest what should the learning materials that they upload, and for the registered users is for the learning materials they should read or study next that relates to their history learning's. 5. Knowledge Sharing Portal Module It provides an interactive platform where users can collaborate, exchange ideas, and engage in discussions related to knowledge resources. This module encourages active participation and knowledge exchange, transforming stored information into shared and collaboration sharing within users.
LOGO	
Project Objectives:	1. To develop a centralized knowledge management system that stores and organizes educational resources efficiently. 2. To provide policies and procedures to contributors to take control of their educational resources. 3. To promote knowledge sharing and collaboration among users through an interactive portal. 4. To help readers, students for the system to be understandable to provide learnings to help the contributors improve. 5. To improve learning efficiency and information accessibility within the system.
Project Description:	LearnLink is a web-based Knowledge Management System designed to support educational institutions by providing a centralized platform for storing, managing, and sharing educational resources. They will provide organized and collaborative access to learning materials to enhance their knowledge and learnings. The system enables contributors to upload, organize, and retrieve documents through the Knowledge Repository module, while the Policy and Procedure Management module ensures their control and accessibility of their educational resources. The Lesson Learned and Best Practices modules support the users to understand and gain new learnings in their educational resources that they read. They can provide

	insights, comments, and shared information for the contributors to improve their learning materials even more. In addition, the Knowledge Sharing Portal encourages collaboration, discussion, and interaction among different users to gain knowledge and learnings with each other It will be developed as a web-based application using ASP.NET with C# as the primary programming language. This framework was selected for its strong support for secure authentication, role-based access control, which are essential requirements for an ERP-based knowledge management system.
Type of Users/ Role-Based Access	**Admin Account (SuperAdmin):** **Email: admin@learnlink.edu** **Password: Admin123** **Manager Account:** **Email: manager@learnlink.edu** **Password: Manager123** **Contributor Account:** **Email: contributor@learnlink.edu** **Password: Contributor123** **Student Account:** **Email: student@learnlink.edu** **Password: Student123**

Use Case Diagram (Role-Based Access)



Data Dictionary

LearnLink: Student Resource Management System

Users Table

Field Names	Datatype	Length	Description
UserId-PK	nvarchar	9	Unique user identifier
FirstName	nvarchar	25	User's first name
LastName	nvarchar	25	User's last name
MiddleName	nvarchar	25	User's middle name
UserName	nvarchar	256	User login username
Email	nvarchar	256	User email address
Status	nvarchar	20	User account status
DateCreated	datetime	N/A	Date and time the user account was created
DepartmentId-FK	int	N/A	Linked department ID

SchoolId-FK	int	N/A	Linked school ID
Initials	nvarchar	5	User initials
AvatarColor	nvarchar	200	User avatar color for display
GradeOrPosition	nvarchar	100	User grade level or position
SuspensionReason	nvarchar	200	Reason for suspension
SuspensionDate	datetime2	N/A	Date and time of suspension

Schools Table

Field Names	Datatype	Length	Description
SchoolId-PK	int(AI)	N/A	Unique school identifier
Name	nvarchar	150	School name
Code	nvarchar	20	Unique school code
Description	nvarchar	200	School description
Address	nvarchar	200	School address
ContactEmail	nvarchar	100	School contact email
LogoPath	nvarchar	200	School logo path or URL
IsActive	bit	N/A	Indicates whether the school is active
AllowCrossSchoolSharing	bit	N/A	Indicates whether cross-school sharing is allowed
DateCreated	Datetime	N/A	Date and time the school record was created

SchoolSettings Table

Field Names	Datatype	Length	Description
SchoolSettings-PK	int(AI)	N/A	Unique school settings identifier
SchoolId-FK	int	N/A	Linked school ID
InstitutionName	nvarchar	150	Institution name used in settings
AdminEmail	nvarchar	100	Administrator email address

Subjects	nvarchar	50	List of available subjects in JSON format
GradeLevels	nvarchar	50	List of available grade levels in JSON format
ResourceTypes	nvarchar	50	List of available resource types in JSON format
Quarters	nvarchar	50	List of school quarters in JSON format

Departments Table

Field Names	Datatype	Length	Description
DepartmentId-PK	int(AI)	N/A	Unique department identifier
DepartmentName	nvarchar	50	Department name
Description	nvarchar	100	Department description
SchoolId-FK	int	N/A	Linked school ID

Resources Table

Field Names	Datatype	Length	Description
ResourceId-PK	int(AI)	N/A	Unique resource identifier
Title	nvarchar	100	Resource title
Description	nvarchar	500	Resource description
FilePath	nvarchar	200	Resource file path or URL
ResourceType	nvarchar	30	Type of resource
UserId-FK	nvarchar	450	User who uploaded the resource
SchoolId-FK	int	N/A	Linked school ID
IsSharedCrossSchool	bit	N/A	Indicates whether the resource is shared across schools
DateUploaded	datetime2	N/A	Date and time the resource was uploaded
Status	nvarchar	20	Resource status
Subject	nvarchar	50	Resource subject

GradeLevel	nvarchar	30	Target grade level
FileFormat	nvarchar	20	Resource file format
FileSize	nvarchar	20	Resource file size
Quarter	nvarchar	50	Applicable school quarter
ViewCount	int	N/A	Number of views
DownloadCount	int	N/A	Number of downloads
Rating	float	N/A	Average resource rating
RatingCount	int	N/A	Number of ratings submitted
RejectionReason	nvarchar	500	Reason for rejection if not approved
AccessLevel	nvarchar	20	Resource access level
AccessDuration	nvarchar	30	Resource access duration
AllowDownloads	bit	N/A	Indicates whether downloads are allowed
EnableVersionHistory	bit	N/A	Indicates whether version history is enabled
AllowComments	bit	N/A	Indicates whether comments are allowed
AllowRatings	bit	N/A	Indicates whether ratings are allowed
ModerateComments	bit	N/A	Indicates whether comments require moderation
RequireVersionNotes	bit	N/A	Indicates whether version notes are required
AccessExpiresAt	datetime2	N/A	Resource access expiration date
ThumbnailUrl	nvarchar	500	Resource thumbnail image URL

ResourceCategories Table

Field Names	Datatype	Length	Description
CategoryId-PK	int(AI)	N/A	Unique category identifier
CategoryName	nvarchar	50	Resource category name
Description	nvarchar	100	Category description

ResourceCategoryMaps Table

Field Names	Datatype	Length	Description
MapId-PK	int(AI)	N/A	Unique category mapping identifier
ResourceId-FK	int	N/A	Linked resource ID
CategoryId-FK	int	N/A	Linked category ID

Tags Table

Field Names	Datatype	Length	Description
TagId-PK	int(AI)	N/A	Unique tag identifier
TagName	nvarchar	50	Tag name

ResourceTags Table

Field Names	Datatype	Length	Description
ResourceTagId-PK	int(AI)	N/A	Unique resource tag mapping identifier
ResourceId-FK	int	N/A	Linked resource ID
TagId-FK	int	N/A	Linked tag ID

ResourceVersions Table

Field Names	Datatype	Length	Description
VersionId-PK	int(AI)	N/A	Unique resource version identifier
ResourceId-FK	int	N/A	Linked resource ID
VersionNumber	nvarchar	50	Resource version number
VersionNotes	nvarchar	200	Notes describing the version update
FilePath	nvarchar	500	File path or URL of this version

FileFormat	nvarchar	10	File format of this version
FileSize	nvarchar	20	File size of this version
DateUpdated	datetime2	N/A	Date and time this version was created

ResourceComments Table

Field Names	Datatype	Length	Description
CommentId-PK	int(AI)	N/A	Unique comment identifier
ResourceId-FK	int	N/A	Linked resource ID
UserId-FK	nvarchar	450	User who posted the comment
Content	nvarchar	500	Comment content
DatePosted	datetime2	N/A	Date and time the comment was posted
DateUpdated	datetime2	N/A	Date and time the comment was updated
ParentCommentId-FK	int	N/A	Parent comment ID for reply threads
LikeCount	int	N/A	Number of likes on the comment

ResourceAccessGrants Table

Field Names	Datatype	Length	Description
GrantId-PK	int(AI)	N/A	Unique access grant identifier
ResourceId-FK	int	N/A	Linked resource ID
UserId-FK	nvarchar	450	User granted access to the resource
GrantedAt	datetime2	N/A	Date and time access was granted

LessonsLearned Table

Field Names	Datatype	Length	Description
LessonId-PK	int(AI)	N/A	Unique lesson learned identifier
ResourceId-FK	int	N/A	Linked resource ID

UserId-FK	nvarchar	450	User who submitted the lesson learned
SchoolId-FK	int	N/A	Linked school ID
Rating	real	N/A	Rating given by the user
Comment	nvarchar	500	Short lesson learned comment
DateSubmitted	datetime2	N/A	Date and time submitted
Title	nvarchar	100	Lesson learned title
Content	nvarchar	500	Detailed lesson learned content
Category	nvarchar	50	Lesson learned category
Tags	nvarchar	500	Comma-separated lesson tags
LikeCount	int	N/A	Number of likes
CommentCount	int	N/A	Number of comments

BestPractices Table

Field Names	Datatype	Length	Description
PracticeId-PK	int(AI)	N/A	Unique best practice identifier
ResourceId-FK	int	N/A	Linked resource ID
Description	nvarchar	500	Description of the best practice
UserId-FK	nvarchar	450	User who submitted the best practice

Recommendations Table

Field Names	Datatype	Length	Description
RecommendationId-PK	int(AI)	N/A	Unique recommendation identifier
UserId-FK	nvarchar	450	Linked user ID
ResourceId-FK	int	N/A	Recommended resource ID
RecommendationType	nvarchar	30	Type of recommendation

ReadingHistory Table

Field Names	Datatype	Length	Description
HistoryId-PK	int(AI)	N/A	Unique reading history identifier
UserId-FK	nvarchar	450	Linked user ID
ResourceId-FK	int	N/A	Linked resource ID
LastAccessed	datetime2	N/A	Last date and time accessed
ProgressStatus	nvarchar	20	Reading progress status
ProgressPercent	int	N/A	Reading progress percentage
LastPosition	nvarchar	50	Last saved reading position
CompletedDate	datetime2	N/A	Date reading was completed
IsBookmarked	bit	N/A	Indicates whether the resource is bookmarked

Discussions Table

Field Names	Datatype	Length	Description
DiscussionId-PK	int(AI)	N/A	Unique discussion identifier
Title	nvarchar	100	Discussion title
UserId-FK	nvarchar	450	User who created the discussion
SchoolId-FK	int	N/A	Linked school ID
DateCreated	datetime2	N/A	Date and time the discussion was created
Content	nvarchar	500	Main discussion content
Category	nvarchar	50	Discussion category
Type	nvarchar	30	Discussion type
Tags	nvarchar	500	Comma-separated discussion tags
Status	nvarchar	20	Discussion status
ViewCount	int	N/A	Number of views
LikeCount	int	N/A	Number of likes

DiscussionPosts Table

Field Names	Datatype	Length	Description
PostId-PK	int(AI)	N/A	Unique discussion post identifier
DiscussionId-FK	int	N/A	Linked discussion ID
UserId-FK	nvarchar	450	User who posted the reply
Content	nvarchar	500	Reply content
DatePosted	datetime2	N/A	Date and time the reply was posted
LikeCount	int	N/A	Number of likes on the reply
IsBestAnswer	bit	N/A	Indicates whether the reply is marked as best answer

Like Table

Field Names	Datatype	Length	Description
LikeId-PK	int(AI)	N/A	Unique like identifier
UserId-FK	nvarchar	450	User who liked the target item
TargetType	nvarchar	20	Type of liked target
TargetId	int	N/A	ID of the liked record
CreatedAt	datetime2	N/A	Date and time the like was created

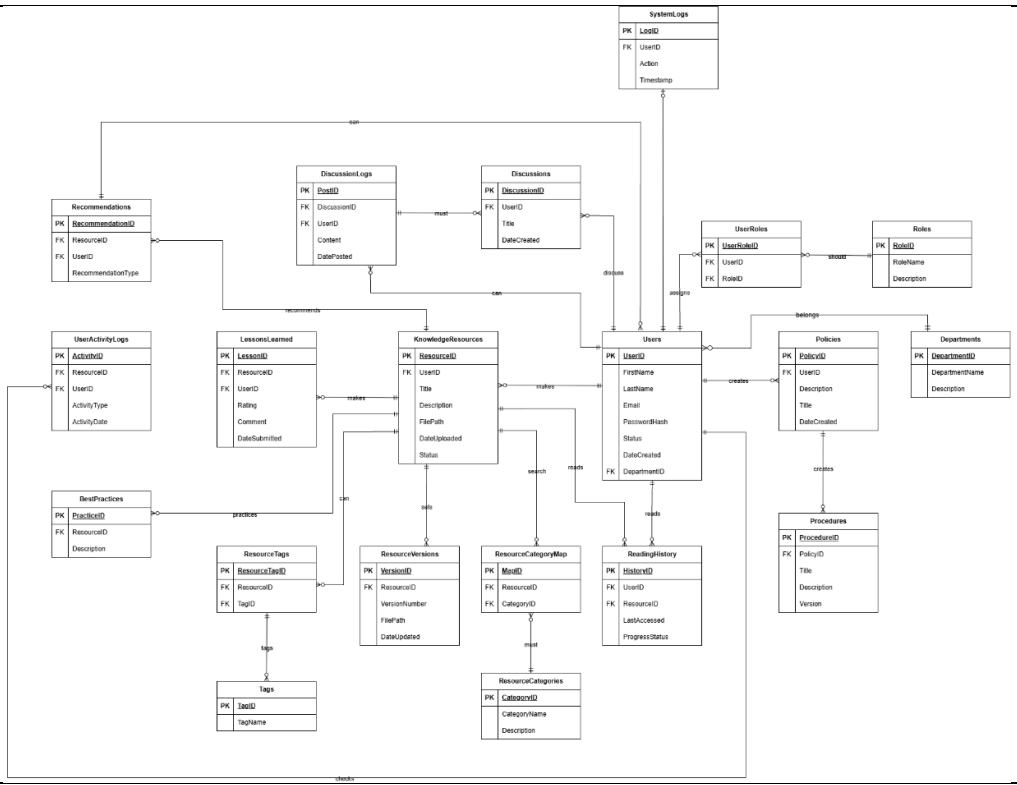
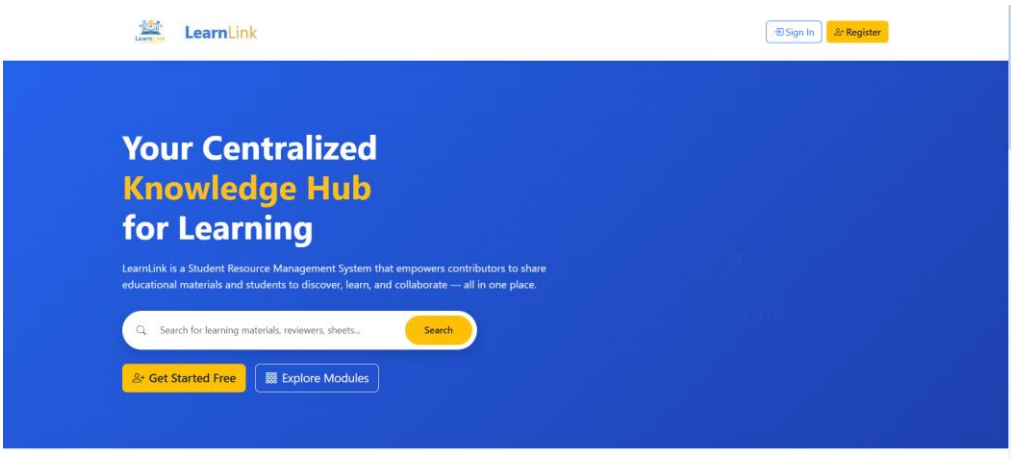
Notifications Table

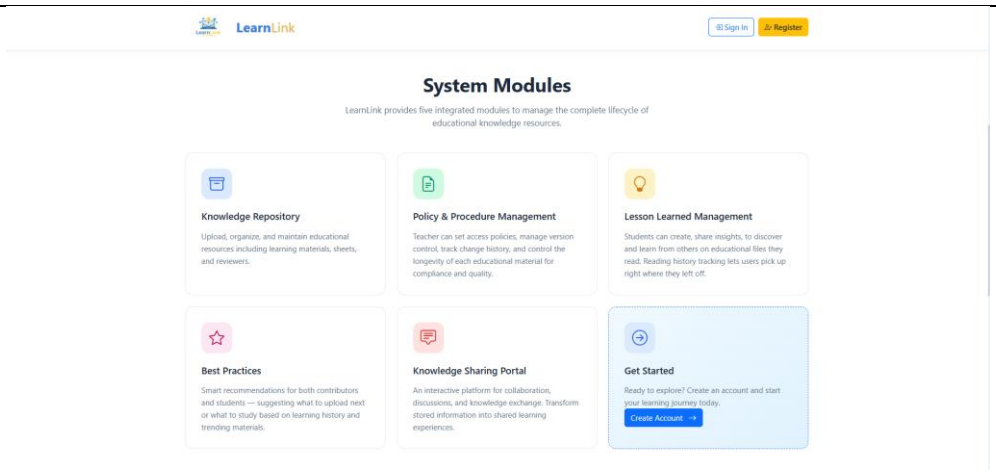
Field Names	Datatype	Length	Description
NotificationId-PK	int(AI)	N/A	Unique notification identifier
UserId-FK	nvarchar	450	User receiving the notification
Title	nvarchar	100	Notification title
Message	nvarchar	500	Notification message body
Type	nvarchar	30	Type of notification
Icon	nvarchar	50	Notification icon class

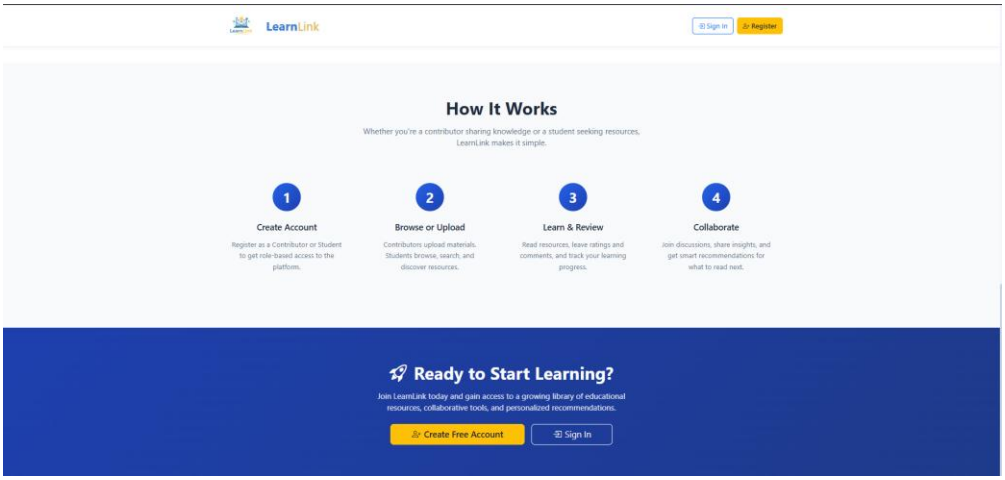
IconBg	nvarchar	20	Notification icon background style or color
Link	nvarchar	500	Related notification link
ResourceId-FK	int	N/A	Related resource ID
IsRead	bit	N/A	Indicates whether the notification has been read
CreatedAt	datetime2	N/A	Date and time the notification was created

UserActivityLogs Table

Field Names	Datatype	Length	Description
ActivityId-PK	int(AI)	N/A	Unique activity log identifier
UserId-FK	nvarchar	450	User who performed the activity
ResourceId-FK	int	N/A	Related resource ID
ActivityType	nvarchar	50	Type of activity
ActivityDate	datetime2	N/A	Date and time of the activity
TargetTitle	nvarchar	200	Title of the affected item

Entity Relational Diagram (ERD)	
Prototype (Frontend)	Landing Page – This page is the where the user will first visit the website, the purpose of this is to welcome them by introducing the modules and their descriptions. If the user will search there it will pop up all the resources that is related to. If the user clicks the “Get Started Free” it will direct to the register page. In the header there’s the website logo, name, sign in and register button for easy access. 



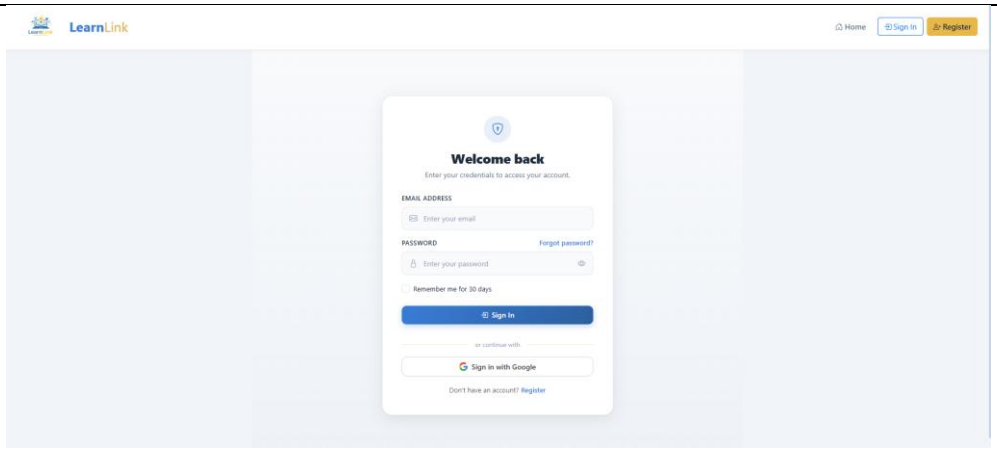


Login Page – This the login page where the user will input their credentials like email address and password.

If the user forget's his password it will direct and notify the user via gmail message and it will send it's new password.

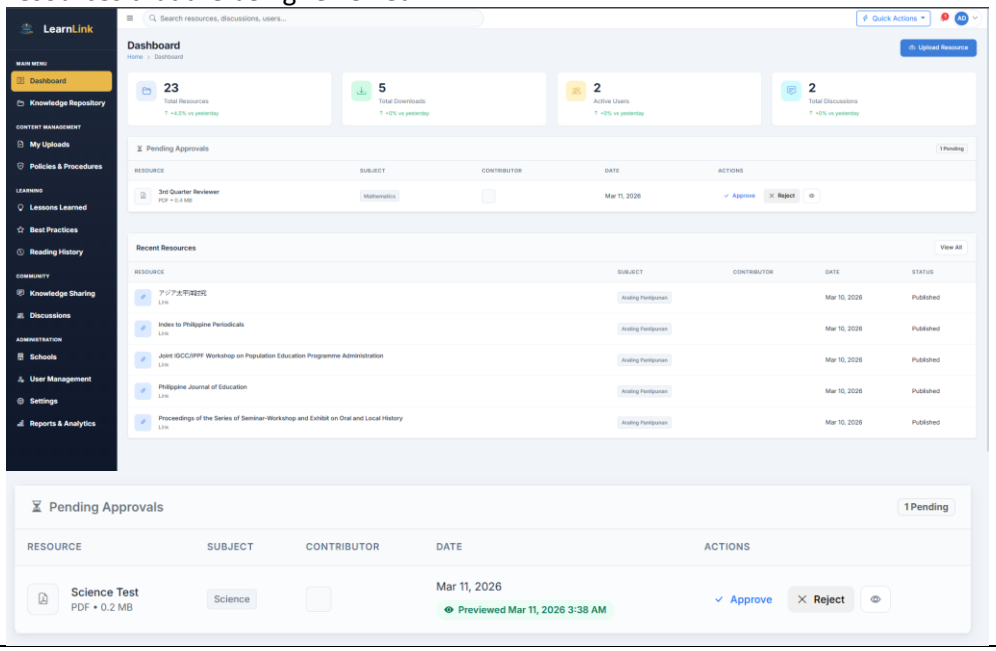
The "Remember for 30 days" for the user's who always use the system so that everytime they will visit the email and password will be automatically added.

The "Sign in with Google" is where the user will belong to the Google Auth for more secure register process with Google.

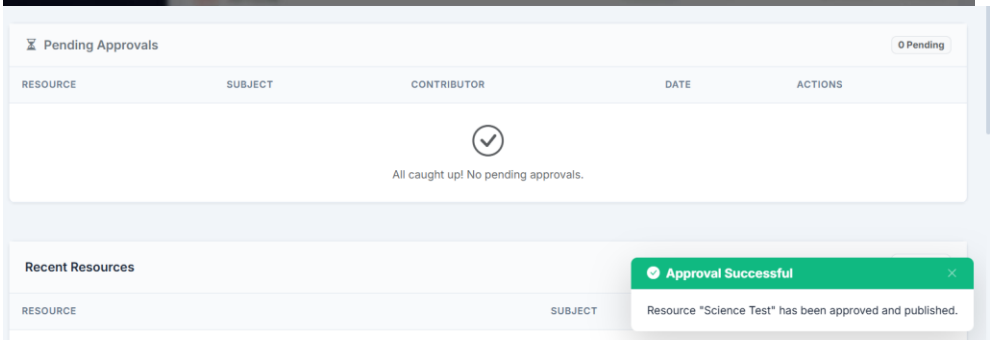


Dashboard (For Admin/Managers): This is the main dashboard for administrators and managers. It gives a broad view of system activity, pending work, and overall platform status.

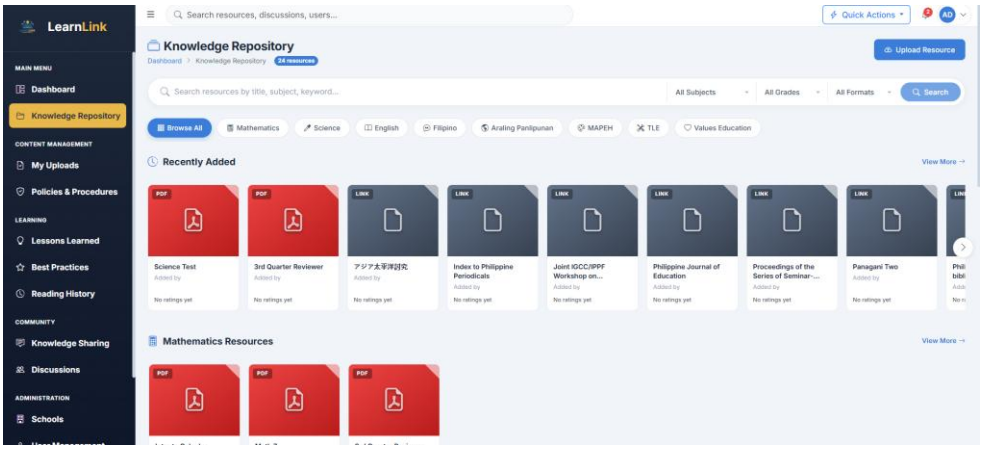
Pending Review is where the managers/admin will verify the resources that the contributor uploads, to ensure the security of the resources. There's actions for approved or reject. Both actions will notify the uploader about the status of their resources that are being reviewed.

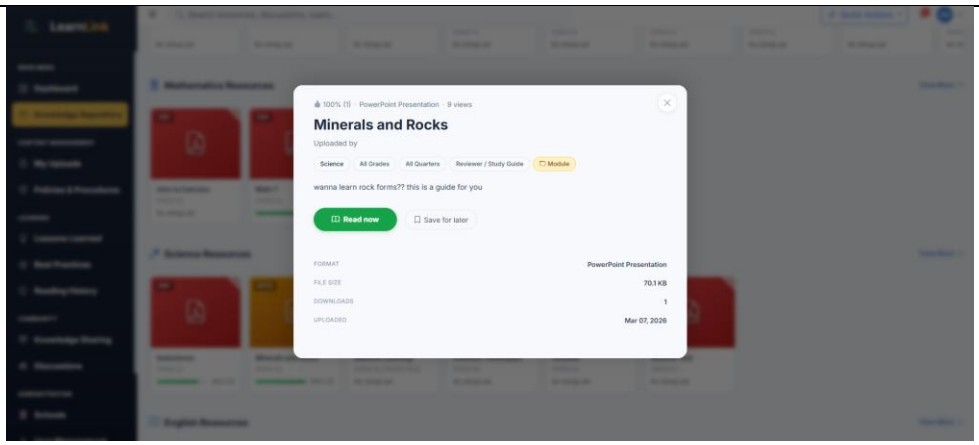




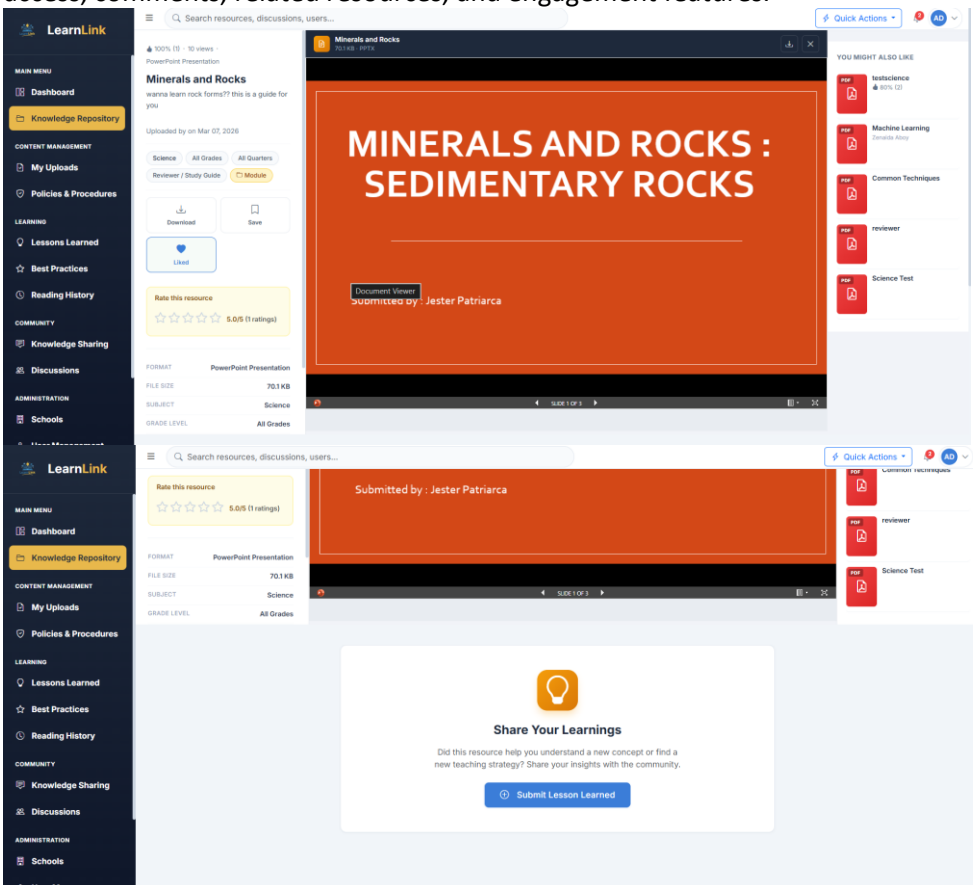


Knowledge Repository : This page is the main library of learning resources. Users can browse, search, and filter educational materials according to their needs.

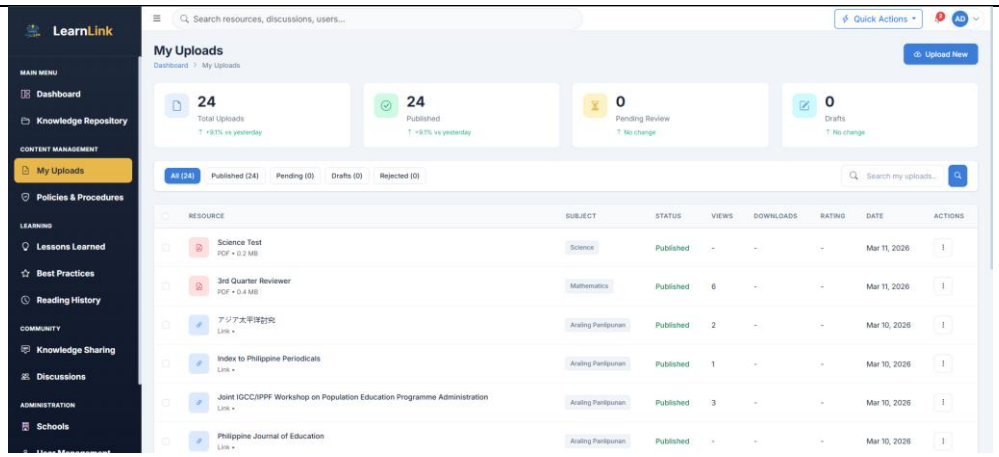




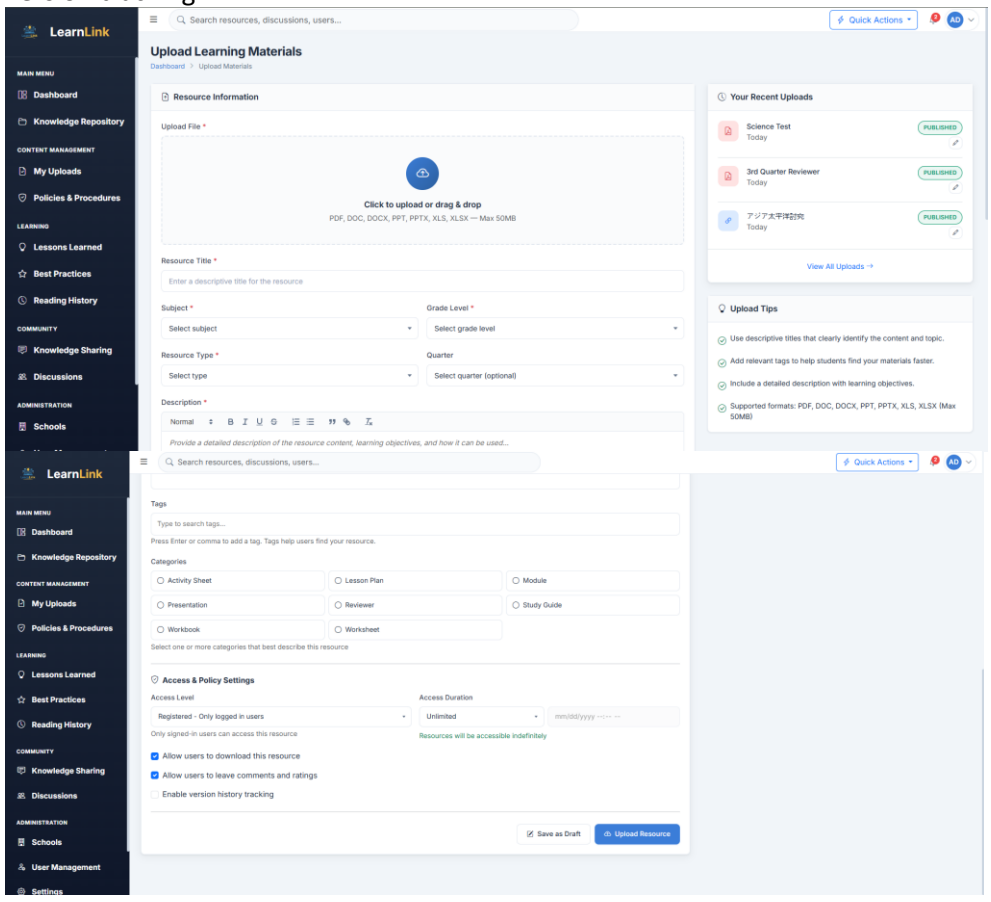
This page presents the full details of a selected resource, including its description, file access, comments, related resources, and engagement features.



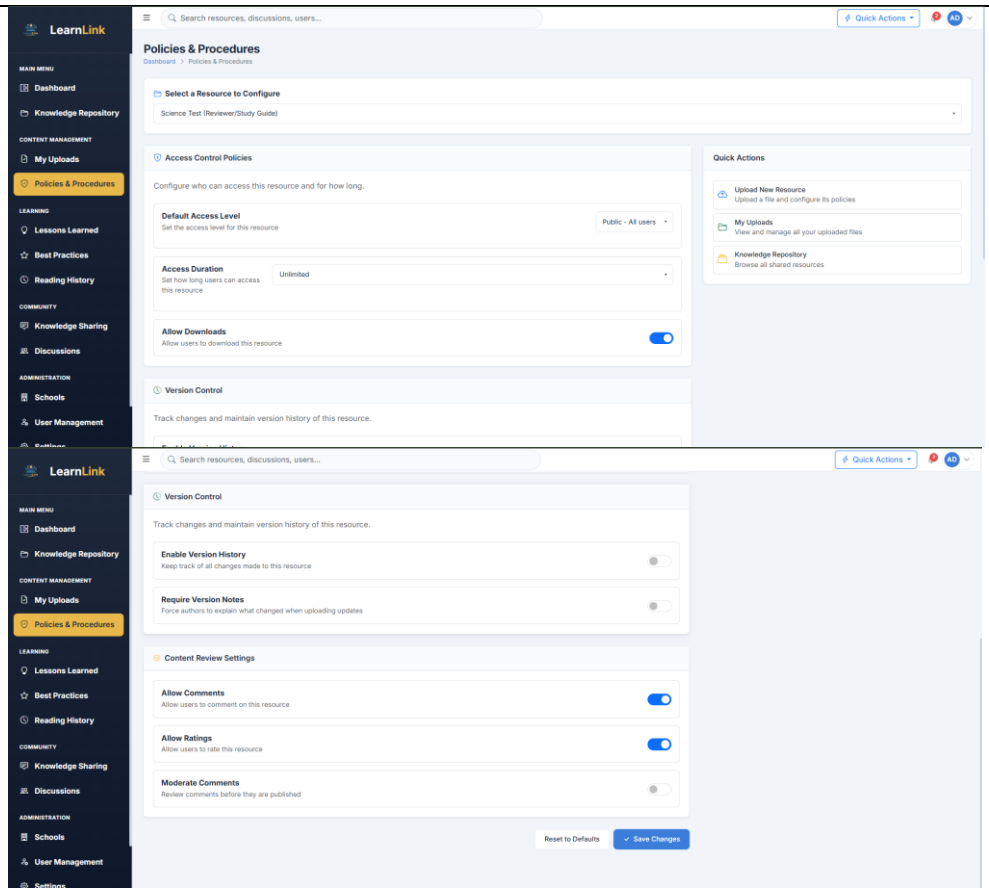
My Uploads: This page gives contributors an organized view of all their uploaded materials and their current status.



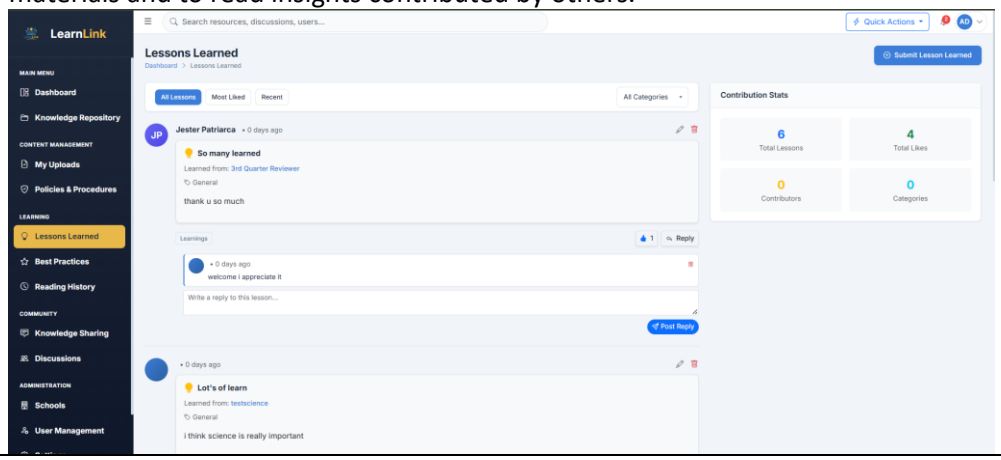
This page allows contributors to submit educational resources. They will input the title, subject, grade level, quarter, description, tags, categories, and lastly the access policy settings such as toggle for download, likes and ratings, or they can see the version tracking.

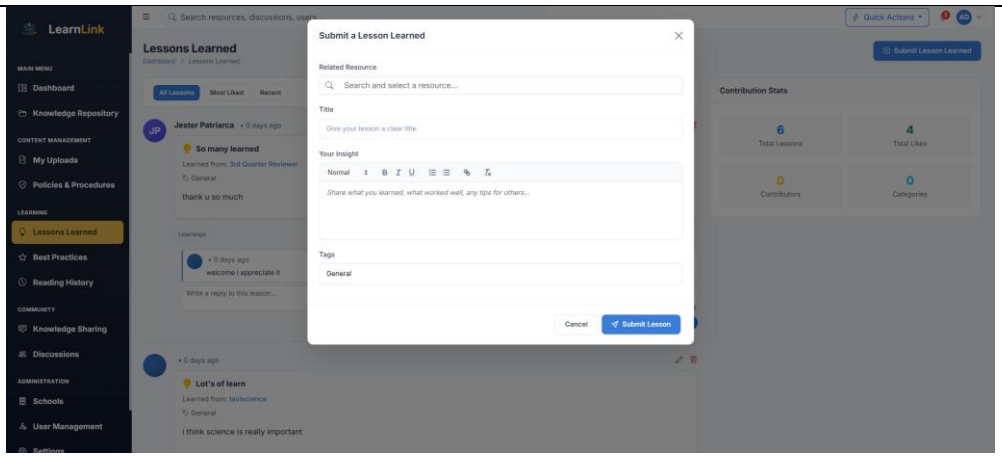


Policies & Procedures – This module is where the uploader can set policies to their resources. They can enable or disable such as downloads, ratings, setting duration to their resources (they can make the resources visible for 3 days, 30 days, or even unlimited.



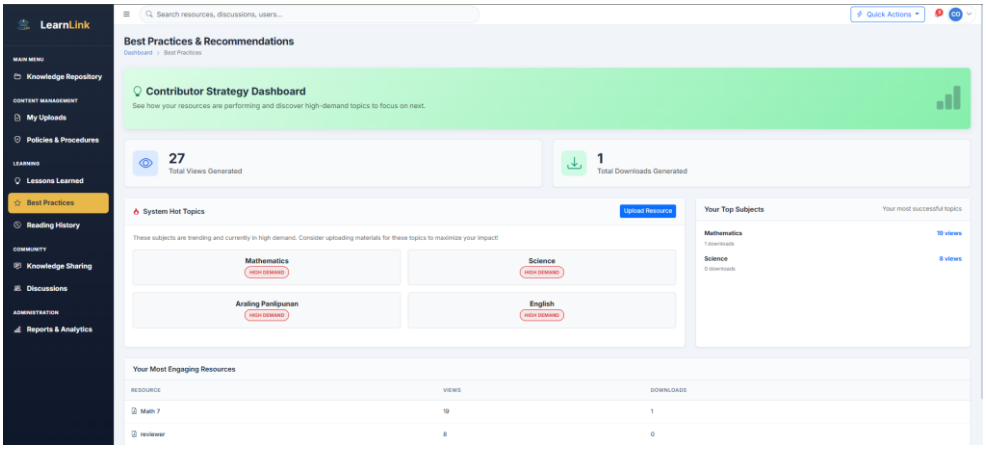
Lessons Learned: This page allows users to share what they learned from educational materials and to read insights contributed by others.



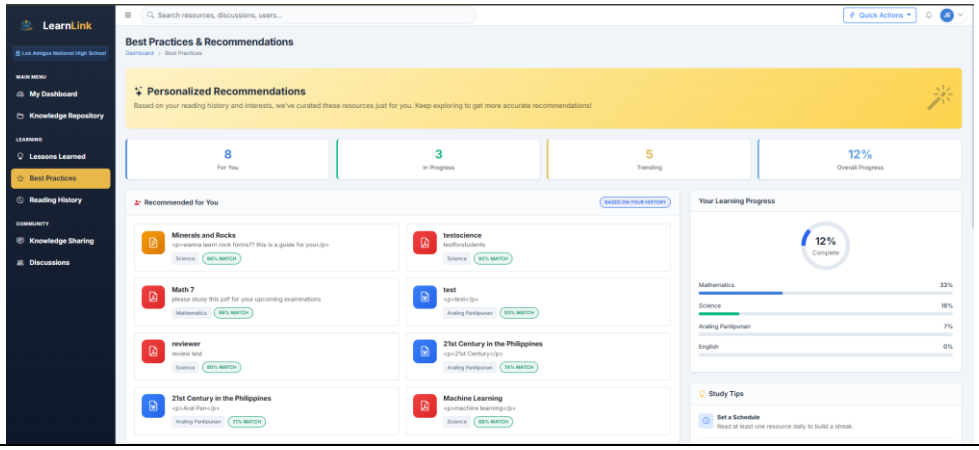


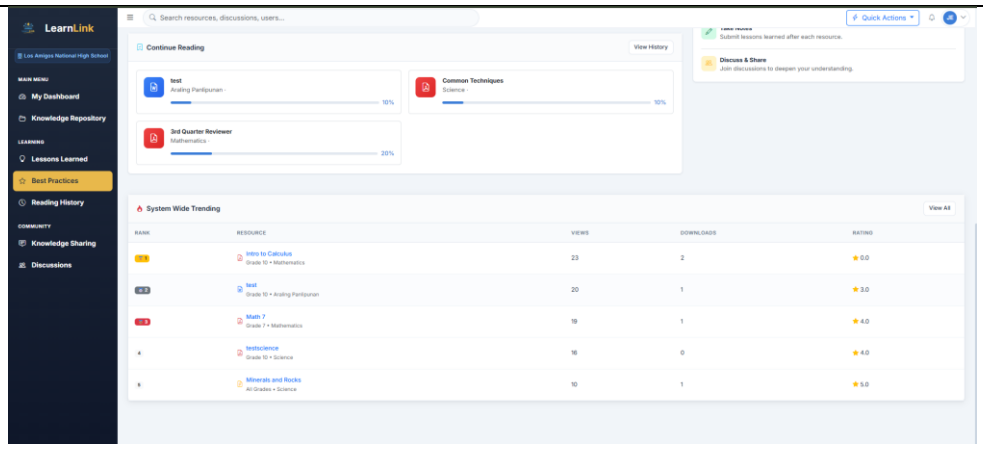
Best Practices & Recommendations : This page highlights useful learning and contribution patterns. It supports reflection, improvement, and guidance for both learners and contributors.

For contributor:

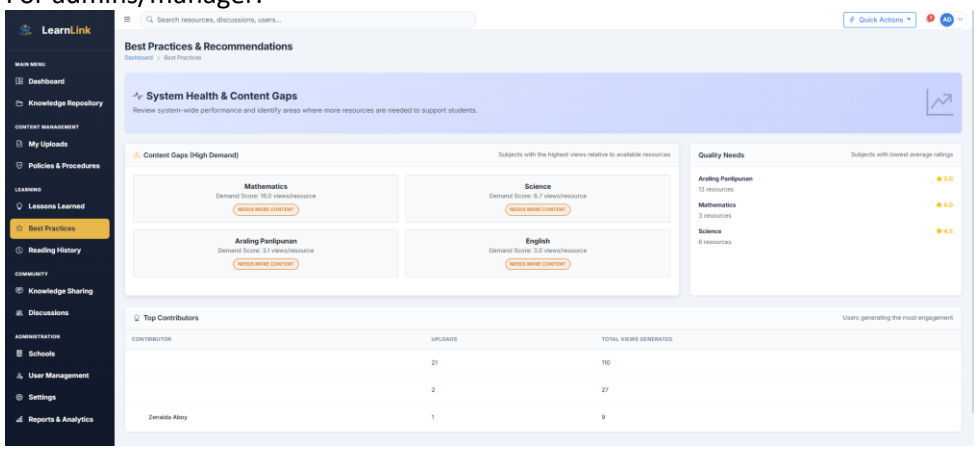


For student:

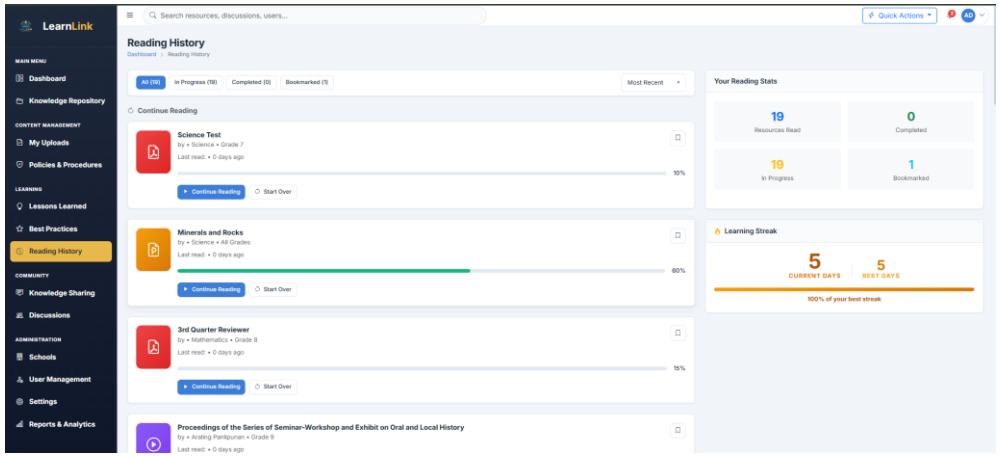




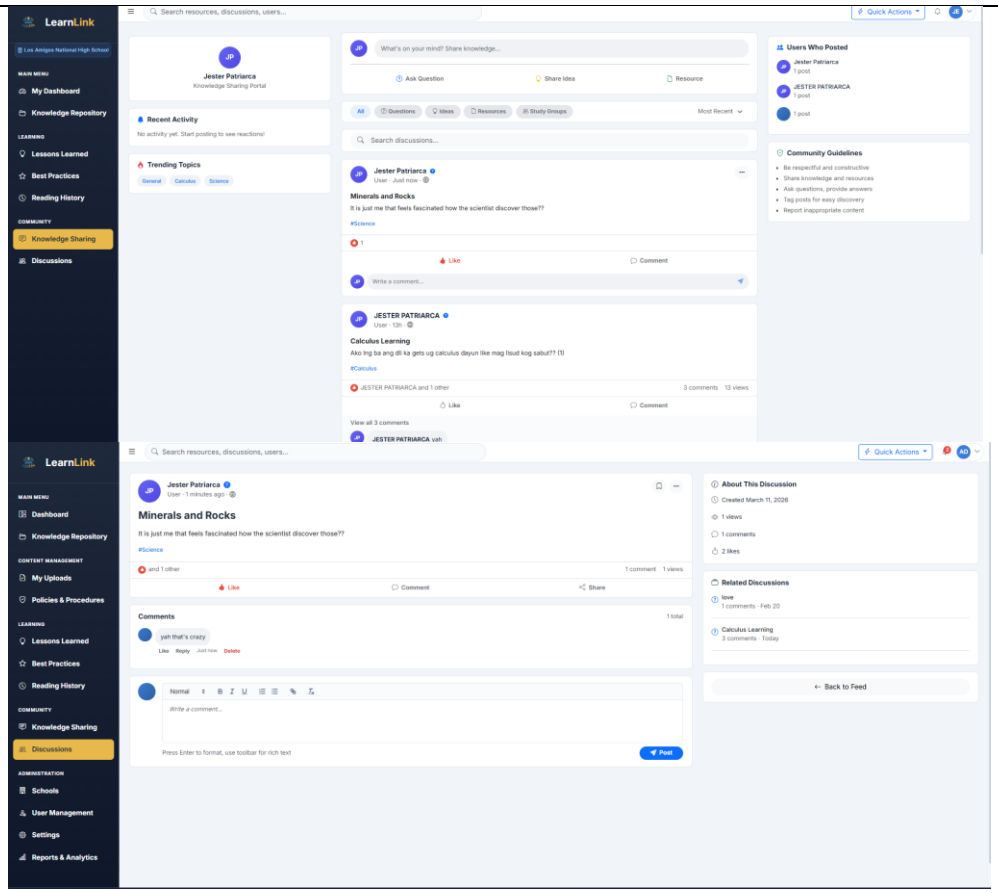
For admins/manager:



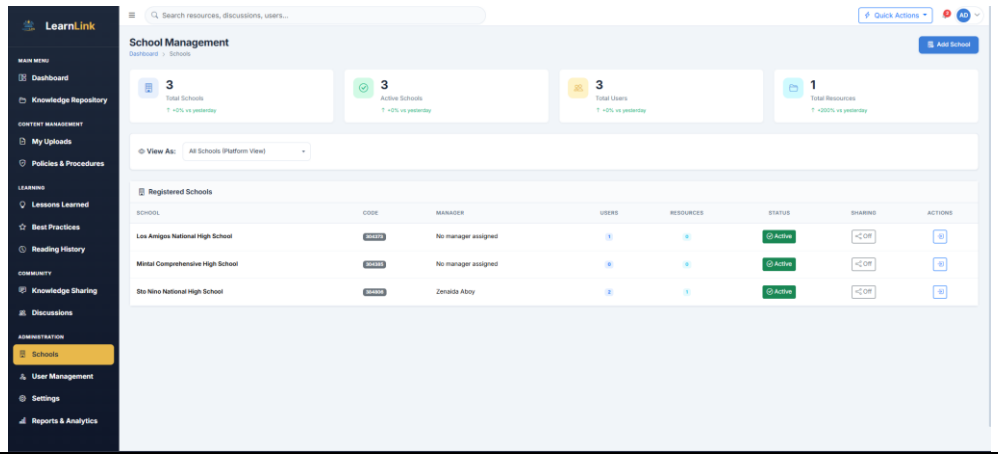
Reading History: This page helps students review what they have already opened, how far they have progressed, and what they have completed.

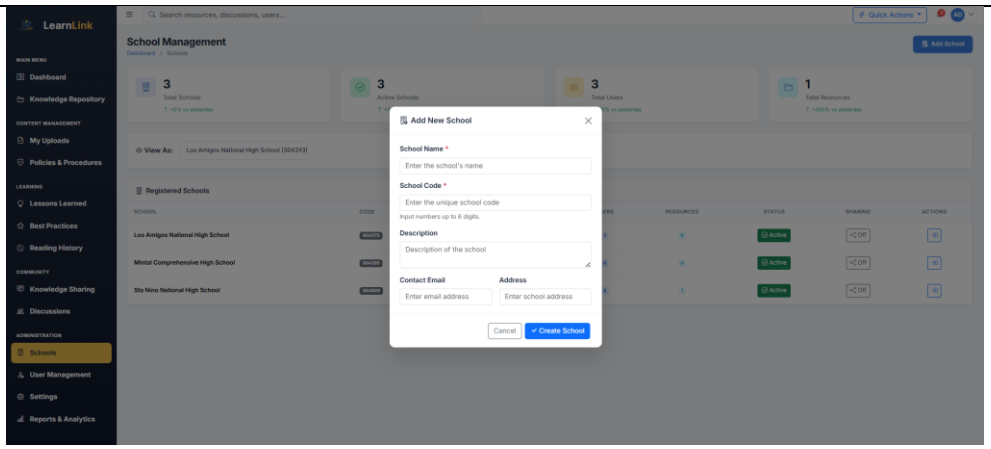


Knowledge Sharing Portal: This page is the main collaboration space, where users can participate in knowledge exchange and community discussion.

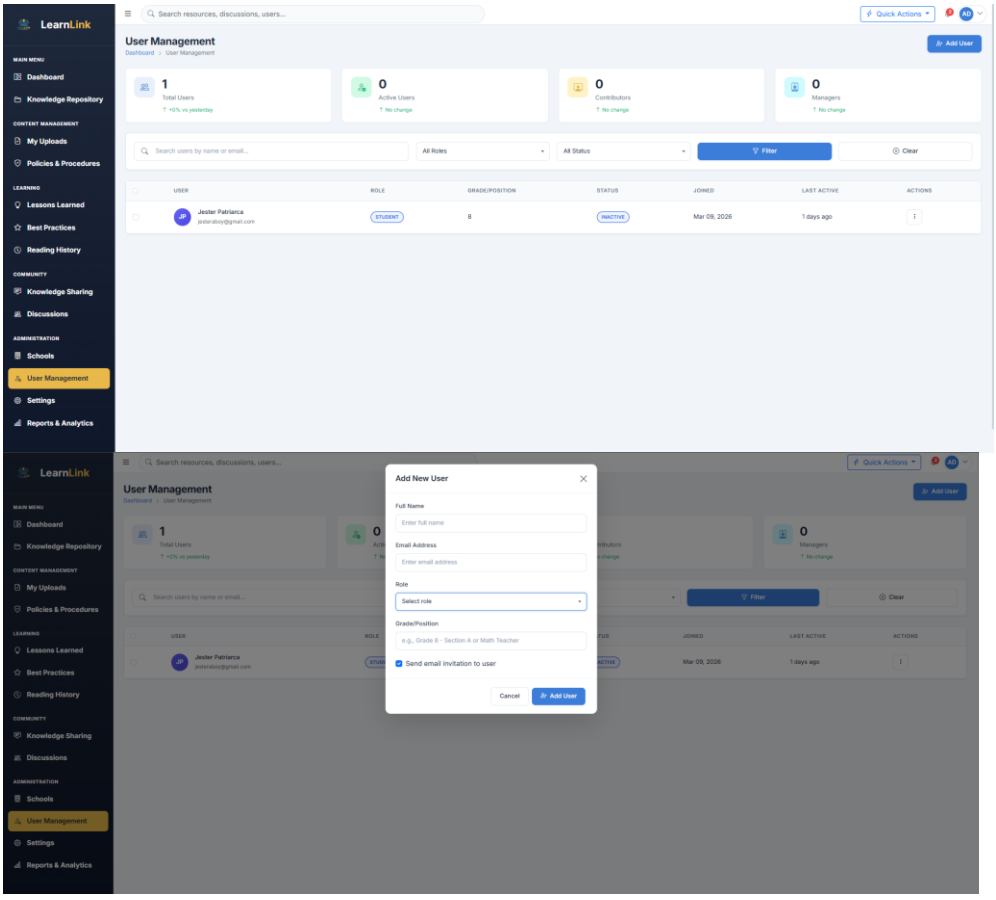


School Management: This page gives SuperAdmin users a centralized place to manage schools registered in the platform. They can add school, description, school code, contact email, and address.

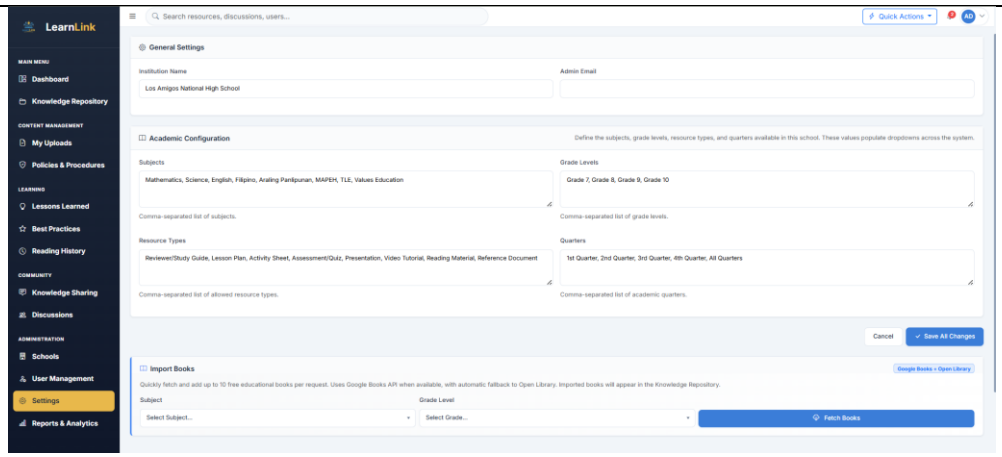




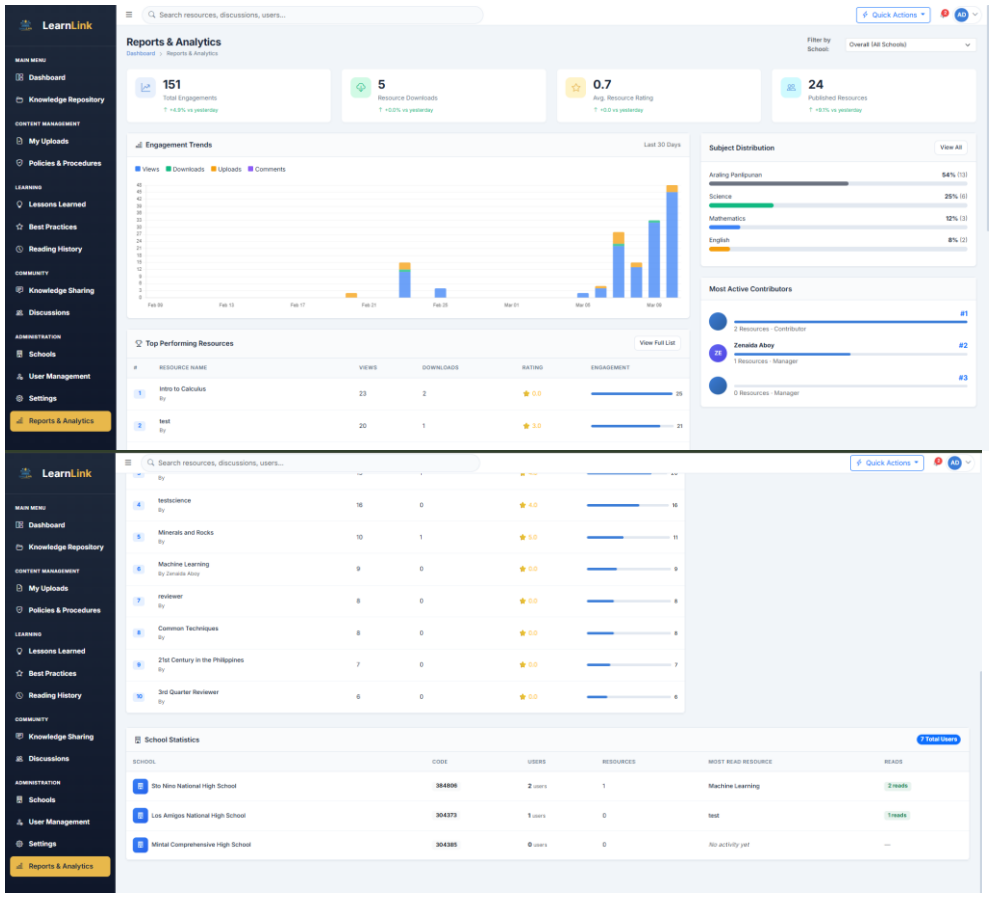
User Management: This page allows administrators and managers to oversee user accounts, roles, and statuses. They can add users and set their roles, then they can notify the users through gmail to complete the registration.



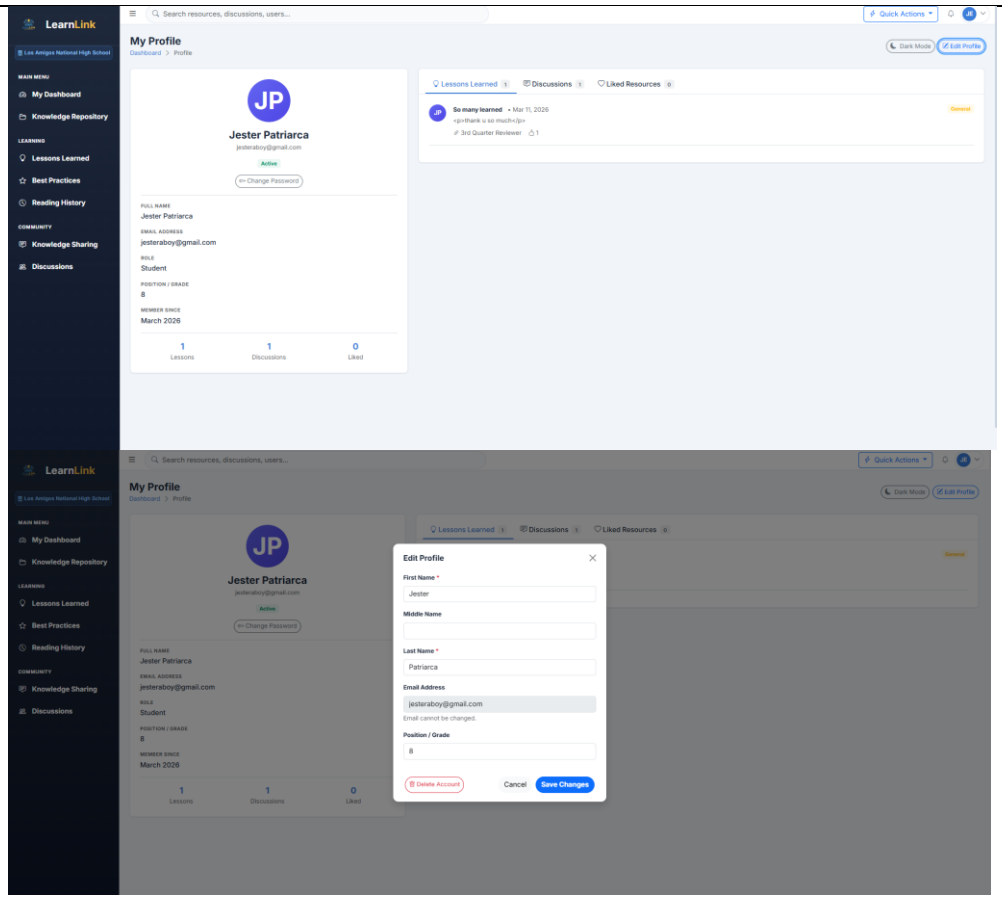
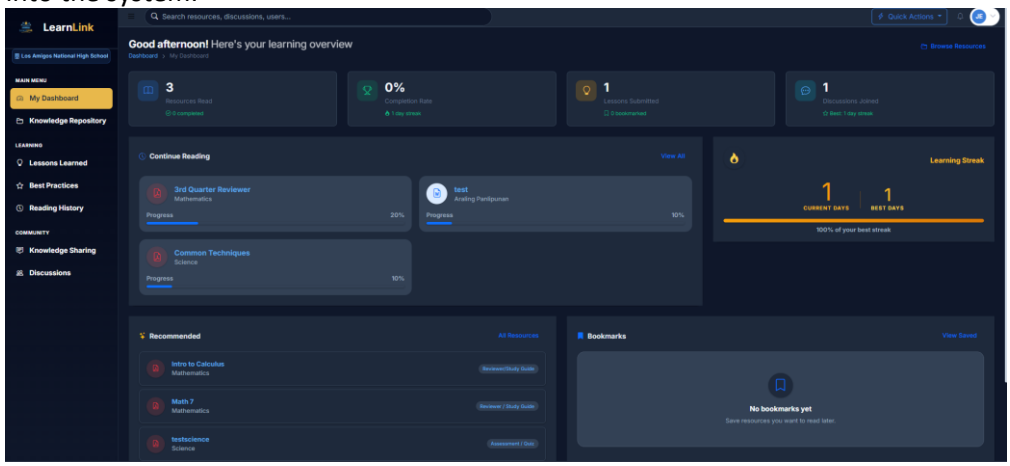
Settings: This page allows school-specific platform settings to be maintained. In the bottom there's a page for Import Books which uses an Google Books API for fetching data online.

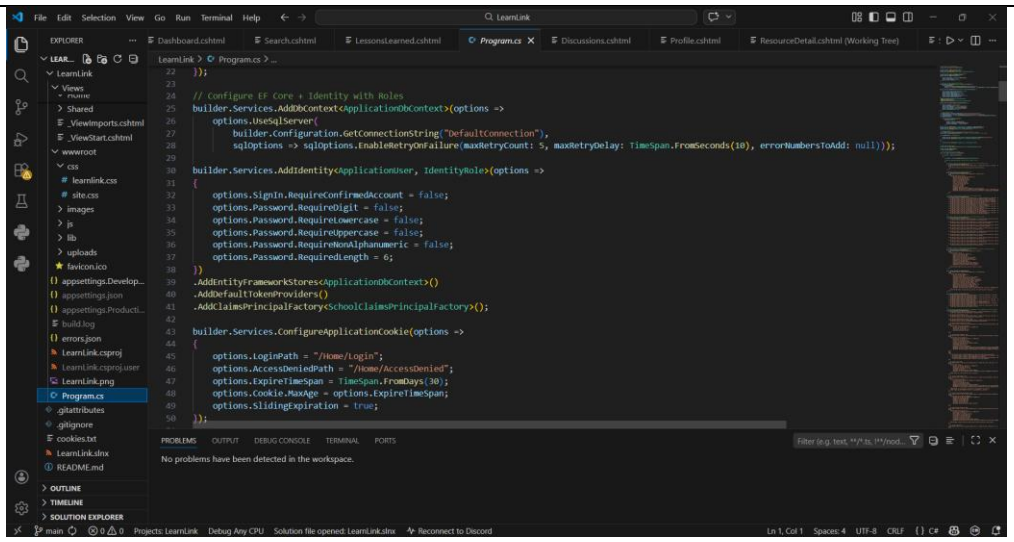


Analytics: This page presents system insights, activity summaries, and analytical information for monitoring purposes.
Engagement trends is where to see the website activities evey day, it's being labeled such as views, downloads, comments, etc.
Subject Distribution is where the subjects are mostly being added, there's a rating also for the top resources that the user reads.



Profile: This page provides a closer view of an individual user's profile, activity, and resources. They can edit their profile, change password, or delete their account.

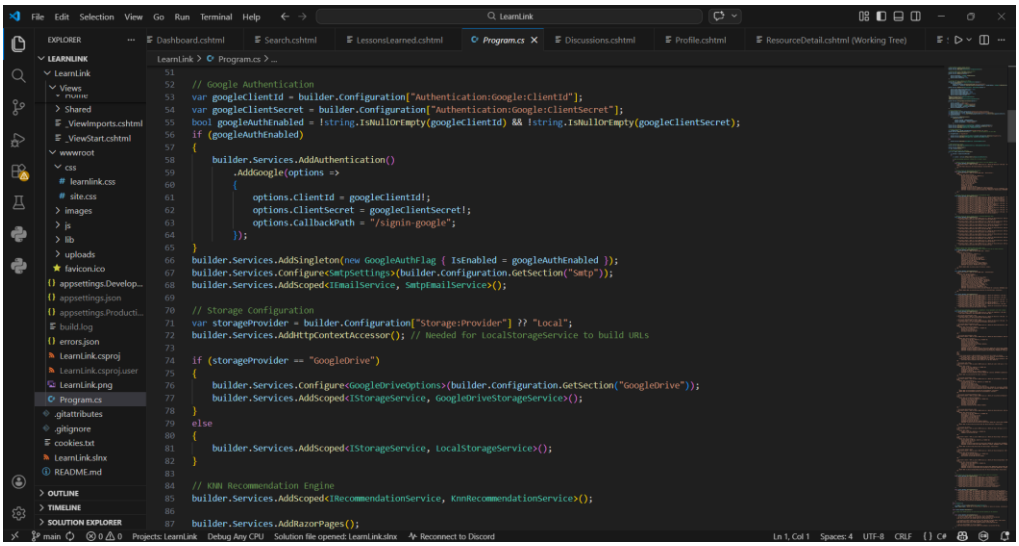
	 Bonus feature: Since this is a knowledge repository system where the user will read, the dark mode feature is implemented for comfortable and ease of users to navigate into the system. 
Prototype (Backend)	Program.cs: Configures MVC, EF Core, ASP.NET Identity, session handling, Google authentication, email delivery, storage service registration, and recommendation services, Also runs schema repair and migration logic on startup to reduce failures caused by database drift.



```
// Configure EF Core + Identity with Roles
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection"),
        sqlOptions => sqlOptions.EnableRetryOnFailure(maxRetryCount: 5, maxRetryDelay: TimeSpan.FromSeconds(10), errorNumbersToAdd: null)));

builder.Services.AddIdentity<ApplicationUser, IdentityRole>(options =>
{
    options.SignIn.RequireConfirmedAccount = false;
    options.Password.RequireDigit = false;
    options.Password.RequireLowercase = false;
    options.Password.RequireUppercase = false;
    options.Password.RequireNonAlphanumeric = false;
    options.Password.RequiredLength = 6;
})
.AddEntityFrameworkStores<ApplicationDbContext>()
.AddDefaultTokenProviders()
.AddClaimsPrincipalFactory<SchoolClaimsPrincipalFactory>();

builder.Services.ConfigureApplicationCookie(options =>
{
    options.LoginPath = "/Home/Login";
    options.AccessDeniedPath = "/Home/AccessDenied";
    options.ExpireTimeSpan = TimeSpan.FromDays(30);
    options.Cookie.MaxAge = options.ExpireTimeSpan;
    options.SlidingExpiration = true;
});
```



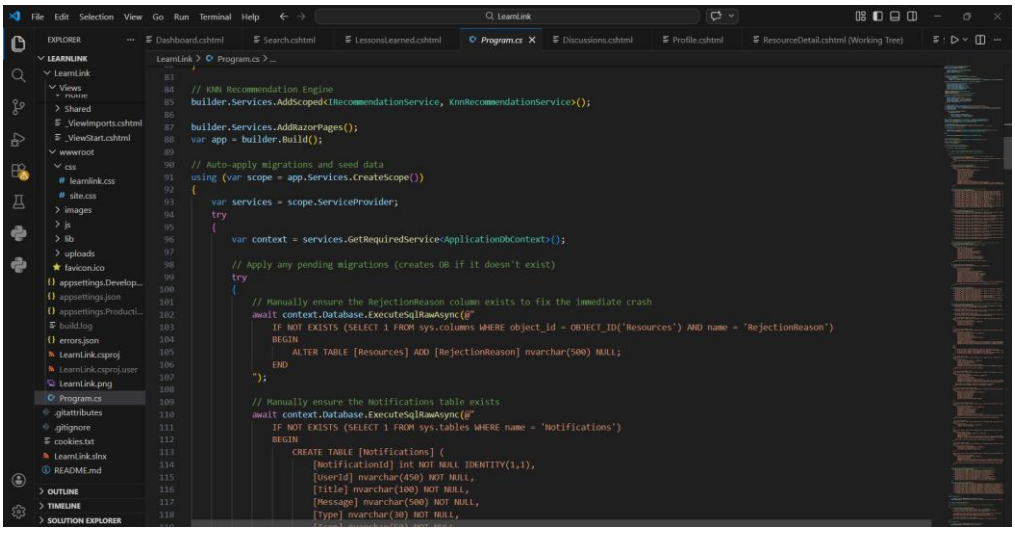
```
// Google Authentication
var googleClientId = builder.Configuration["Authentication:Google:ClientId"];
var googleClientSecret = builder.Configuration["Authentication:Google:ClientSecret"];
bool googleAuthEnabled = !string.IsNullOrEmpty(googleClientId) && !string.IsNullOrEmpty(googleClientSecret);
if (googleAuthEnabled)
{
    builder.Services.AddAuthentication()
        .AddGoogle(options =>
        {
            options.ClientId = googleClientId!;
            options.ClientSecret = googleClientSecret!;
            options.CallbackPath = "/signin-google";
        });
}
builder.Services.AddSingleton(new GoogleAuthFlag { IsEnabled = googleAuthEnabled });
builder.Services.Configure<SmtpSettings>(builder.Configuration.GetSection("Smtp"));
builder.Services.AddScoped<IEmailService, SmtpEmailService>();

// Storage Configuration
var storageProvider = builder.Configuration["Storage:Provider"] ?? "Local";
builder.Services.AddHttpContextAccessor(); // Needed for LocalStorageService to build URLs

if (storageProvider == "GoogleDrive")
{
    builder.Services.Configure<GoogleDriveOptions>(builder.Configuration.GetSection("GoogleDrive"));
    builder.Services.AddScoped<IStorageService, GoogleDriveStorageService>();
}
else
{
    builder.Services.AddScoped<IStorageService, LocalStorageService>();
}

// KNN Recommendation Engine
builder.Services.AddScoped<IRecommendationService, KnnRecommendationService>();

builder.Services.AddRazorPages();
```



```
// KNN Recommendation Engine
builder.Services.AddScoped<IRecommendationService, KnnRecommendationService>();

builder.Services.AddRazorPages();
var app = builder.Build();

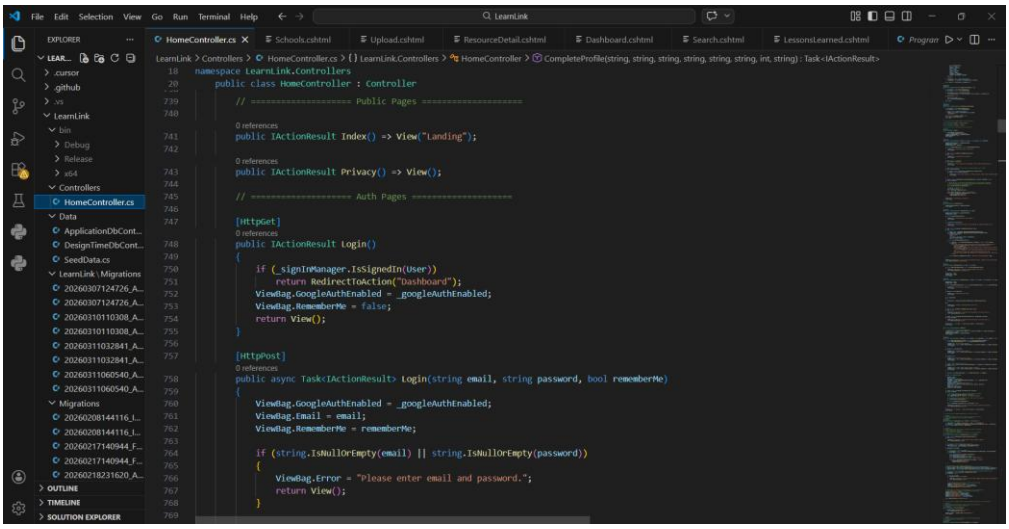
// Auto-apply migrations and seed data
using (var scope = app.Services.CreateScope())
{
    var services = scope.ServiceProvider;
    try
    {
        var context = services.GetRequiredService<ApplicationDbContext>();

        // Apply any pending migrations (creates DB if it doesn't exist)
        try
        {
            // Manually ensure the RejectionReason column exists to fix the immediate crash
            await context.Database.ExecuteSqlRawAsync(@"
                IF NOT EXISTS (SELECT 1 FROM sys.columns WHERE object_id = OBJECT_ID('Resources') AND name = 'RejectionReason')
                BEGIN
                    ALTER TABLE [Resources] ADD [RejectionReason] nvarchar(500) NULL;
                END
            ");

            // Manually ensure the Notifications table exists
            await context.Database.ExecuteSqlRawAsync(@"
                IF NOT EXISTS (SELECT 1 FROM sys.tables WHERE name = 'Notifications')
                BEGIN
                    CREATE TABLE [Notifications] (
                        [NotificationId] int NOT NULL IDENTITY(1,1),
                        [UserId] nvarchar(450) NOT NULL,
                        [Title] nvarchar(100) NOT NULL,
                        [Message] nvarchar(500) NOT NULL,
                        [Type] nvarchar(30) NOT NULL,
```

Authentications and Google Sign In Implementation

HomeController.cs:748: Login() GET renders the login page.



```
namespace LearnLink.Controllers
    public class HomeController : Controller
        // ================== Public Pages ==================

        0 references
        public IActionResult Index() => View("Landing");

        0 references
        public IActionResult Privacy() => View();

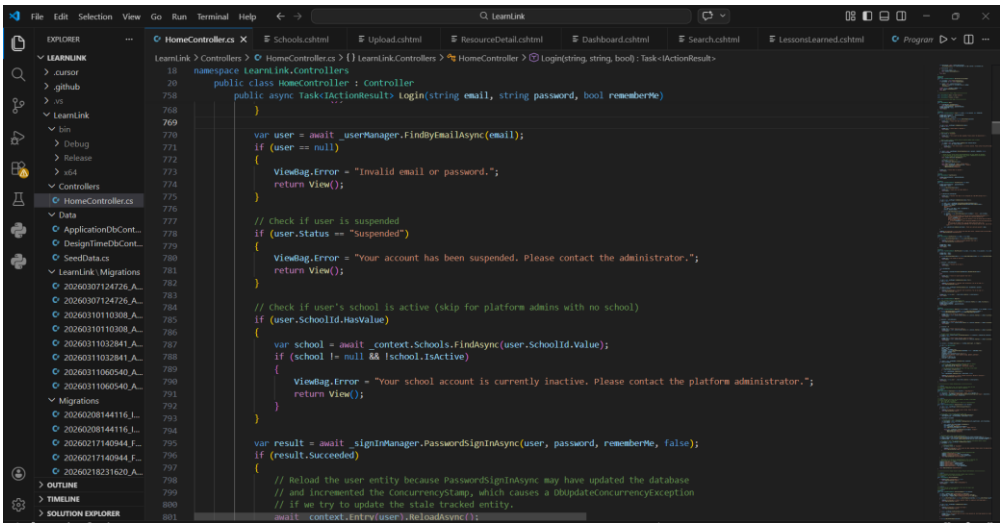
        // ================== Auth Pages ==================

        [HttpGet]
        0 references
        public IActionResult Login()
        {
            if (_signInManager.IsSignedIn(User))
                return RedirectToAction("Dashboard");
            ViewBag.GoogleAuthEnabled = _googleAuthEnabled;
            ViewBag.RememberMe = false;
            return View();
        }

        [HttpPost]
        0 references
        public async Task<IActionResult> Login(string email, string password, bool rememberMe)
        {
            ViewBag.GoogleAuthEnabled = _googleAuthEnabled;
            ViewBag.Email = email;
            ViewBag.RememberMe = rememberMe;

            if (string.IsNullOrEmpty(email) || string.IsNullOrEmpty(password))
            {
                ViewBag.Error = "Please enter email and password.";
                return View();
            }
```

HomeController.cs:758: Login(string email, string password, bool rememberMe) validates credentials, checks suspension state, checks whether the user's school is active, performs PasswordSignInAsync, marks the user as active, logs the event, and routes by role.



```
namespace LearnLink.Controllers
    public class HomeController : Controller
        public async Task<IActionResult> Login(string email, string password, bool rememberMe)
            }

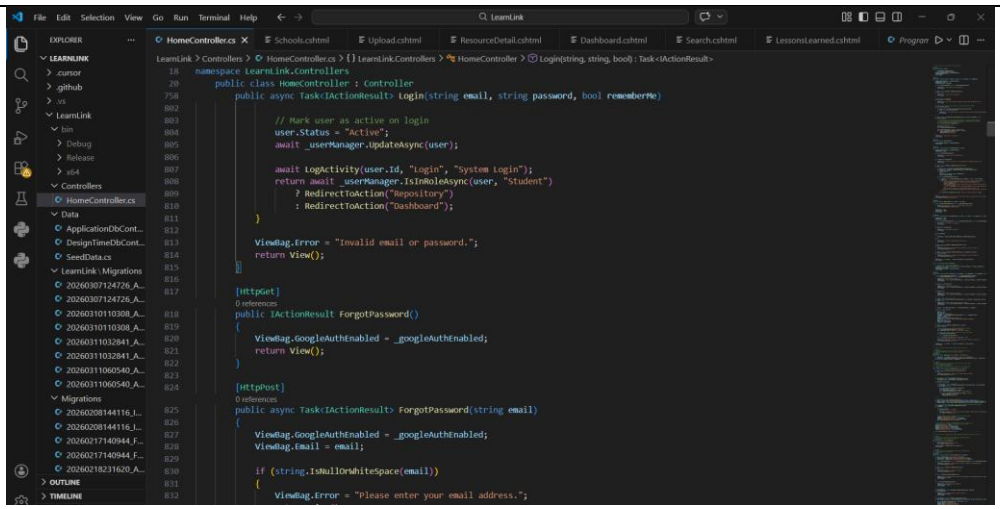
            var user = await _userManager.FindByEmailAsync(email);
            if (user == null)
            {
                ViewBag.Error = "Invalid email or password.";
                return View();
            }

            // Check if user is suspended
            if (user.Status == "Suspended")
            {
                ViewBag.Error = "Your account has been suspended. Please contact the administrator.";
                return View();
            }

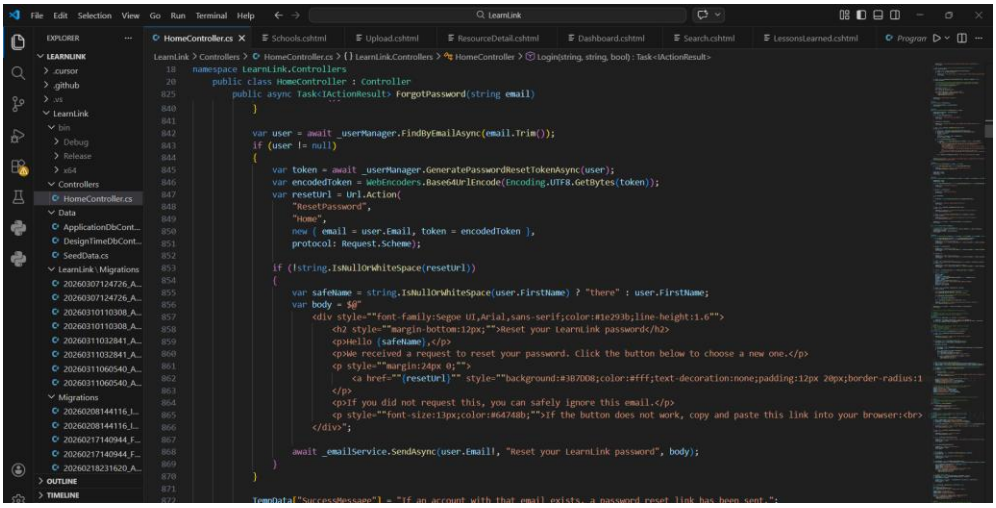
            // Check if user's school is active (skip for platform admins with no school)
            if (user.SchoolId.HasValue)
            {
                var school = await _context.Schools.FindAsync(user.SchoolId.Value);
                if (school != null && !school.IsActive)
                {
                    ViewBag.Error = "Your school account is currently inactive. Please contact the platform administrator.";
                    return View();
                }
            }

            var result = await _signInManager.PasswordSignInAsync(user, password, rememberMe, false);
            if (result.Succeeded)
            {
                // Reload the user entity because PasswordSignInAsync may have updated the database
                // and incremented the ConcurrencyStamp, which causes a DbUpdateConcurrencyException
                // if we try to update the stale tracked entity.
                await _context.Entry(user).ReloadAsync();
```

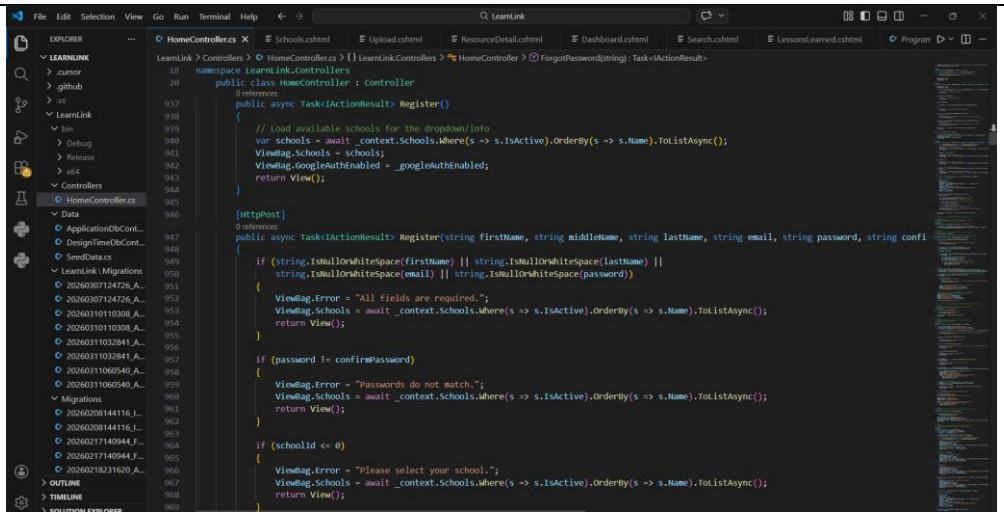
HomeController.cs:818: ForgotPassword() GET renders the reset-request page.
HomeController.cs:825: ForgotPassword(string email) generates an Identity reset token and sends it through the SMTP email service.

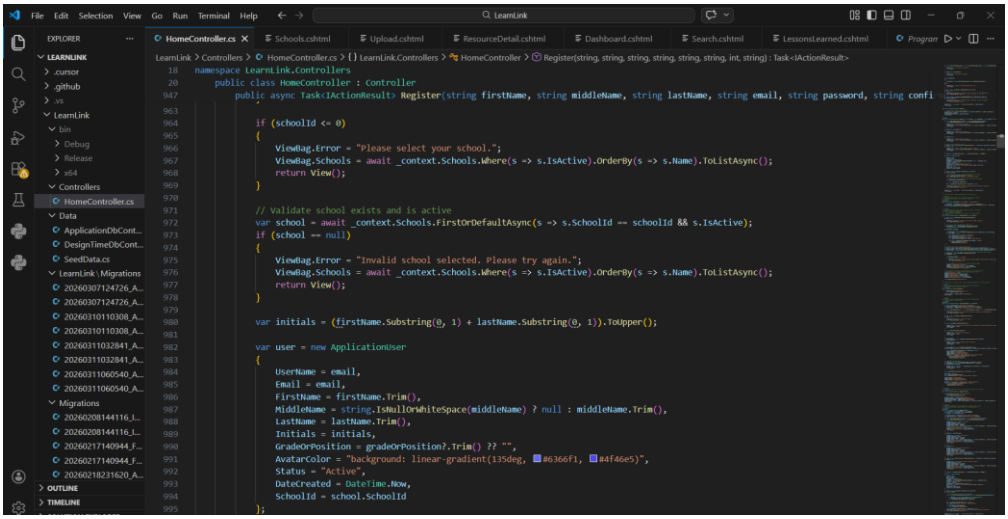


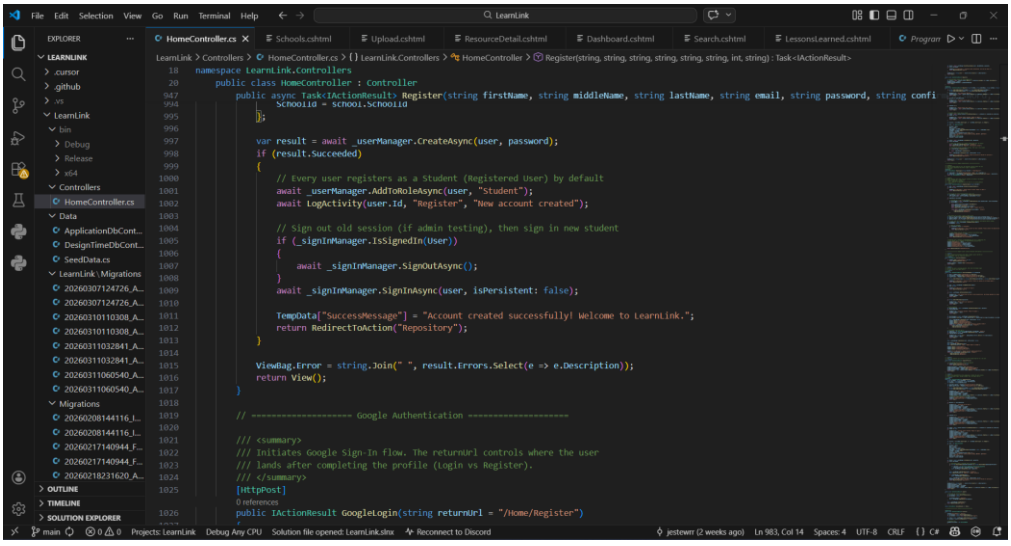
HomeController.cs:818: ForgotPassword() GET renders the reset-request page.
HomeController.cs:825: ForgotPassword(string email) generates an Identity reset token and sends it through the SMTP email service.
HomeController.cs:877: ResetPassword(string email, string token) validates the incoming reset link.
HomeController.cs:891: ResetPassword(...) decodes the token and executes ResetPasswordAsync.



HomeController.cs:937: Register() GET loads active schools for account creation.
HomeController.cs:947: Register(...) validates school membership, creates an ApplicationUser, assigns the default Student role, signs the user in, and logs registration.

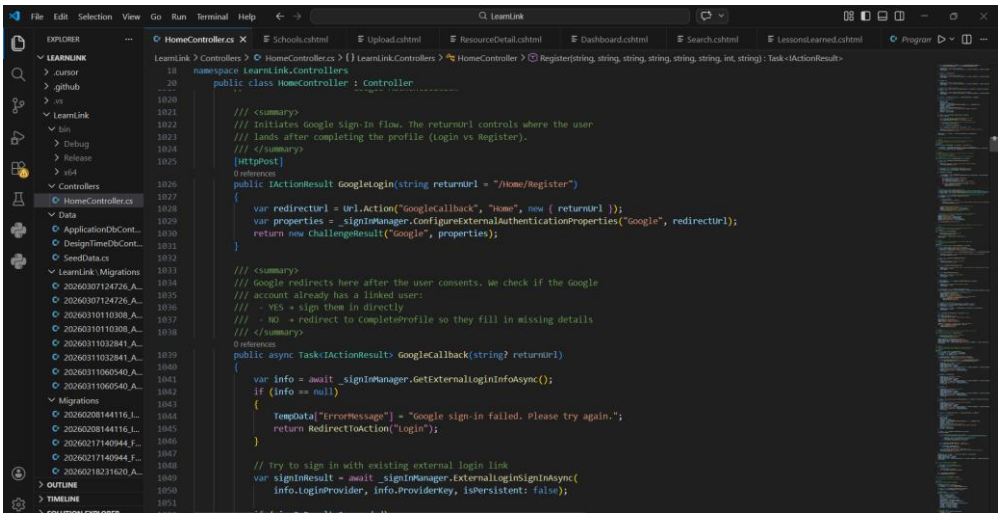






HomeController.cs:1026: GoogleLogin(...) begins the external authentication challenge.

HomeController.cs:1039: GoogleCallback(...) resolves whether the external account should sign in directly, be linked to an existing account, or complete a new profile.

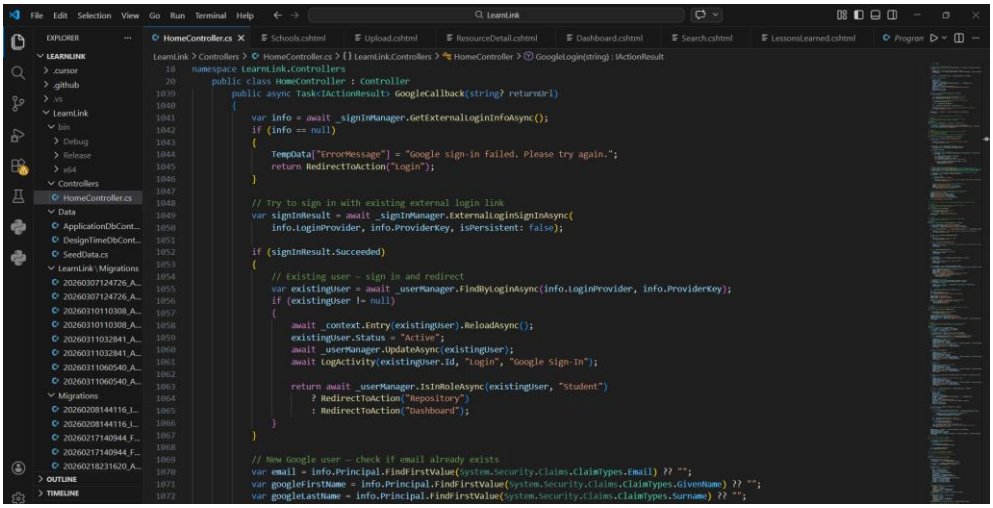


```
namespace LearnLink.Controllers
    public class HomeController : Controller

        /// <summary>
        /// Initiates Google Sign-In flow. The returnUrl controls where the user
        /// lands after completing the profile (Login vs Register).
        /// </summary>
        [HttpPost]
        0 references
        public IActionResult GoogleLogin(string returnUrl = "/Home/Register")
        {
            var redirectUrl = Url.Action("GoogleCallback", "Home", new { returnUrl });
            var properties = _signInManager.ConfigureExternalAuthenticationProperties("Google", redirectUrl);
            return new ChallengeResult("Google", properties);
        }

        /// <summary>
        /// Google redirects here after the user consents. We check if the Google
        /// account already has a linked user:
        ///   - YES → sign them in directly
        ///   - NO  → redirect to CompleteProfile so they fill in missing details
        /// </summary>
        0 references
        public async Task<IActionResult> GoogleCallback(string? returnUrl)
        {
            var info = await _signInManager.GetExternalLoginInfoAsync();
            if (info == null)
            {
                TempData["ErrorMessage"] = "Google sign-in failed. Please try again.";
                return RedirectToAction("Login");
            }

            // Try to sign in with existing external login link
            var signInResult = await _signInManager.ExternalLoginSignInAsync(
                info.LoginProvider, info.ProviderKey, isPersistent: false);
```



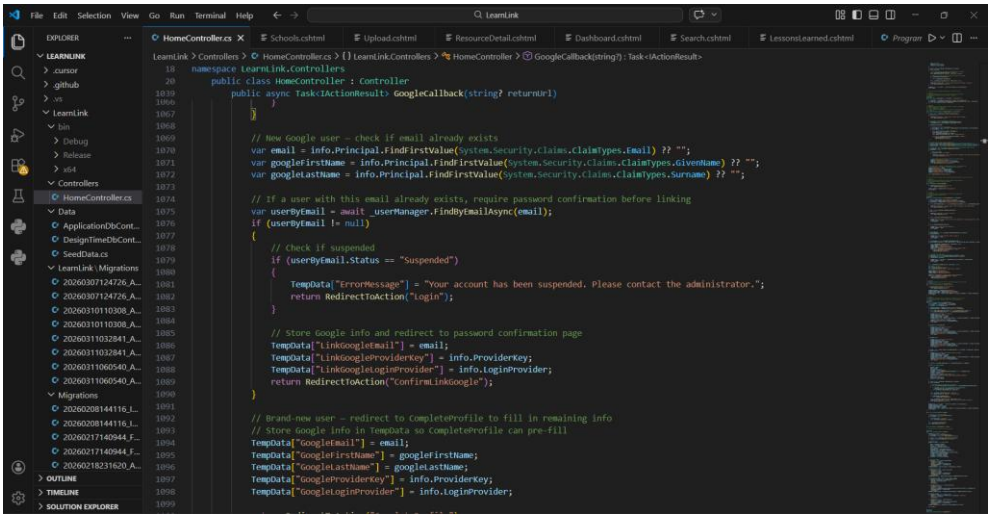
```
namespace LearnLink.Controllers
    public class HomeController : Controller
        public async Task<IActionResult> GoogleCallback(string? returnUrl)
        {
            var info = await _signInManager.GetExternalLoginInfoAsync();
            if (info == null)
            {
                TempData["ErrorMessage"] = "Google sign-in failed. Please try again.";
                return RedirectToAction("Login");
            }

            // Try to sign in with existing external login link
            var signInResult = await _signInManager.ExternalLoginSignInAsync(
                info.LoginProvider, info.ProviderKey, isPersistent: false);

            if (signInResult.Succeeded)
            {
                // Existing user — sign in and redirect
                var existingUser = await _userManager.FindByLoginAsync(info.LoginProvider, info.ProviderKey);
                if (existingUser != null)
                {
                    await _context.Entry(existingUser).ReloadAsync();
                    existingUser.Status = "Active";
                    await _userManager.UpdateAsync(existingUser);
                    await LogActivity(existingUser.Id, "Login", "Google Sign-In");

                    return await _userManager.IsInRoleAsync(existingUser, "Student")
                        ? RedirectToAction("Repository")
                        : RedirectToAction("Dashboard");
                }
            }

            // New Google user — check if email already exists
            var email = info.Principal.FindFirstValue(System.Security.Claims.ClaimTypes.Email) ?? "";
            var googleFirstName = info.Principal.FindFirstValue(System.Security.Claims.ClaimTypes.GivenName) ?? "";
            var googleLastName = info.Principal.FindFirstValue(System.Security.Claims.ClaimTypes.Surname) ?? "";
```



```
namespace LearnLink.Controllers
    public class HomeController : Controller
        public async Task<IActionResult> GoogleCallback(string? returnUrl)
            }

            // New Google user — check if email already exists
            var email = info.Principal.FindFirstValue(System.Security.Claims.ClaimTypes.Email) ?? "";
            var googleFirstName = info.Principal.FindFirstValue(System.Security.Claims.ClaimTypes.GivenName) ?? "";
            var googleLastName = info.Principal.FindFirstValue(System.Security.Claims.ClaimTypes.Surname) ?? "";

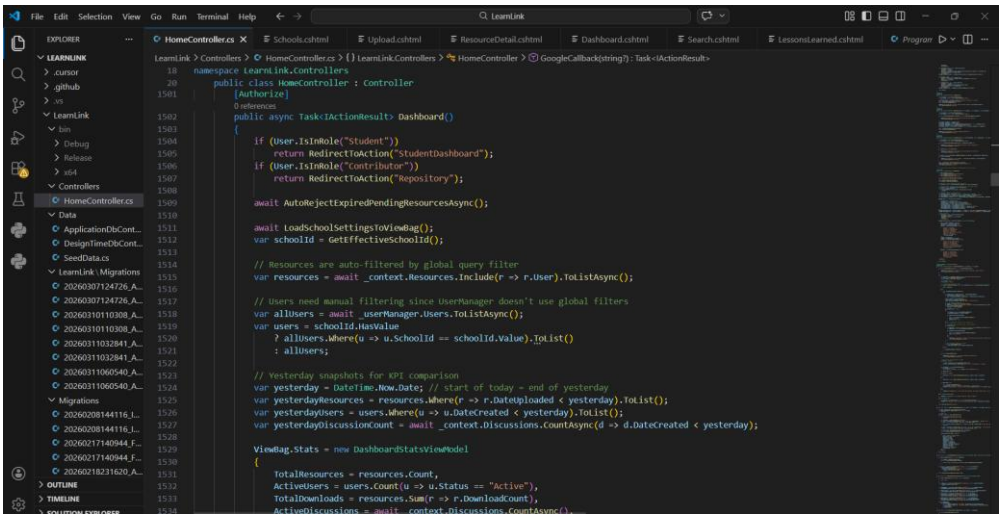
            // If a user with this email already exists, require password confirmation before linking
            var userByEmail = await _userManager.FindByEmailAsync(email);
            if (userByEmail != null)
            {
                // Check if suspended
                if (userByEmail.Status == "Suspended")
                {
                    TempData["ErrorMessage"] = "Your account has been suspended. Please contact the administrator.";
                    return RedirectToAction("Login");
                }

                // Store Google info and redirect to password confirmation page
                TempData["LinkGoogleEmail"] = email;
                TempData["LinkGoogleProviderKey"] = info.ProviderKey;
                TempData["LinkGoogleLoginProvider"] = info.LoginProvider;
                return RedirectToAction("ConfirmLinkGoogle");
            }

            // Brand-new user — redirect to CompleteProfile to fill in remaining info
            // Store Google info in TempData so CompleteProfile can pre-fill
            TempData["GoogleEmail"] = email;
            TempData["GoogleFirstName"] = googleFirstName;
            TempData["GoogleLastName"] = googleLastName;
            TempData["GoogleProviderKey"] = info.ProviderKey;
            TempData["GoogleLoginProvider"] = info.LoginProvider;
```

Navigation and Role Routing

Dashboard:
HomeController.cs:1502: Dashboard() acts as a role-aware entry point, redirecting Students to the student dashboard, Contributors to the repository, and administrators to the main dashboard.



```
[Authorize]
public async Task<IActionResult> Dashboard()
{
    if (User.IsInRole("Student"))
        return RedirectToAction("StudentDashboard");
    if (User.IsInRole("Contributor"))
        return RedirectToAction("Repository");

    await AutoRejectExpiredPendingResourcesAsync();

    await LoadSchoolSettingsToViewBag();
    var schoolId = GetEffectiveSchoolId();

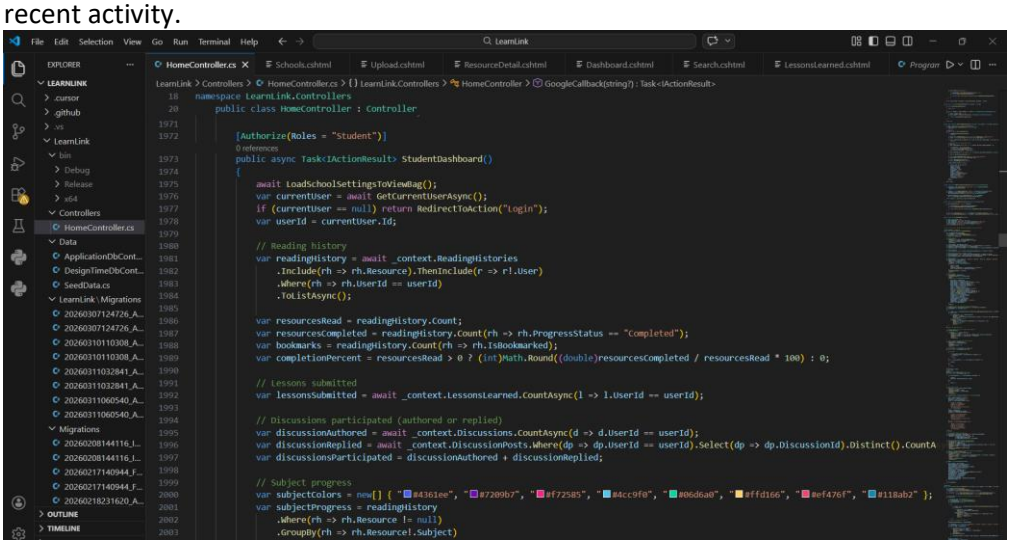
    // Resources are auto-filtered by global query filter
    var resources = await _context.Resources.Include(r => r.User).ToListAsync();

    // Users need manual filtering since UserManager doesn't use global filters
    var allUsers = await _userManager.Users.ToListAsync();
    var users = schoolId.HasValue
        ? allUsers.Where(u => u.SchoolId == schoolId.Value).ToList()
        : allUsers;

    // Yesterday snapshots for KPI comparison
    var yesterday = DateTime.Now.Date; // start of today = end of yesterday
    var yesterdayResources = resources.Where(r => r.DateUploaded < yesterday).ToList();
    var yesterdayUsers = users.Where(u => u.DateCreated < yesterday).ToList();
    var yesterdayDiscussionCount = await _context.Discussions.CountAsync(d => d.DateCreated < yesterday);

    ViewBag.Stats = new DashboardStatsViewModel
    {
        TotalResources = resources.Count,
        ActiveUsers = users.Count(u => u.Status == "Active"),
        TotalDownloads = resources.Sum(r => r.DownloadCount),
```

HomeController.cs:1973: StudentDashboard() assembles personalized student metrics, recommendation data, reading history, streaks, trending materials, and recent activity.



```
[Authorize(Roles = "Student")]
public async Task<IActionResult> StudentDashboard()
{
    await LoadSchoolSettingsToViewBag();
    var currentUser = await GetCurrentUserAsync();
    if (currentUser == null) return RedirectToAction("Login");
    var userId = currentUser.Id;

    // Reading history
    var readingHistory = await _context.ReadingHistories
        .Include(rh => rh.Resource).ThenInclude(r => r!.User)
        .Where(rh => rh.UserId == userId)
        .ToListAsync();

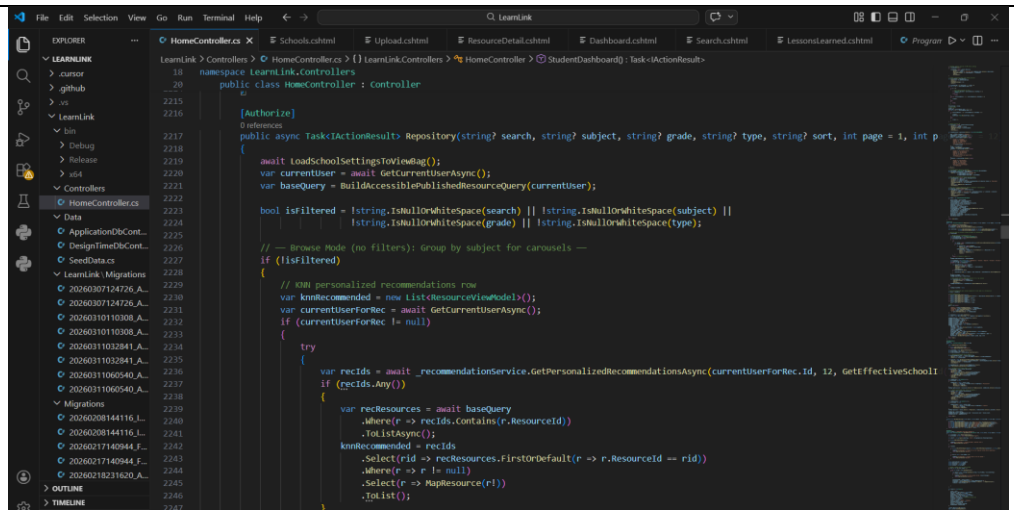
    var resourcesRead = readingHistory.Count;
    var resourcesCompleted = readingHistory.Count(rh => rh.ProgressStatus == "Completed");
    var bookmarks = readingHistory.Count(rh => rh.IsBookmarked);
    var completionPercent = resourcesRead > 0 ? (int)Math.Round((double)resourcesCompleted / resourcesRead * 100) : 0;

    // Lessons submitted
    var lessonsSubmitted = await _context.LessonsLearned.CountAsync(l => l.UserId == userId);

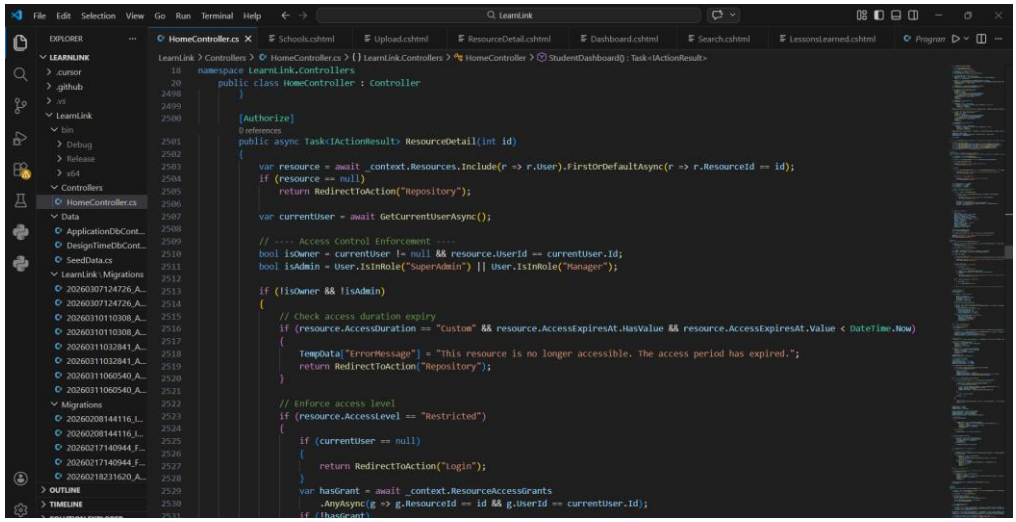
    // Discussions participated (authored or replied)
    var discussionAuthored = await _context.Discussions.CountAsync(d => d.UserId == userId);
    var discussionReplied = await _context.DiscussionPosts.Where(dp => dp.UserId == userId).Select(dp => dp.DiscussionId).Distinct().CountA
    var discussionsParticipated = discussionAuthored + discussionReplied;

    // Subject progress
    var subjectColors = new[] { "#4361ee", "#7209b7", "#f72585", "#4cc9f0", "#06d6a0", "#ffd166", "#ef476f", "#118ab2" };
    var subjectProgress = readingHistory
        .Where(rh => rh.Resource != null)
        .GroupBy(rh => rh.Resource!.Subject)
```

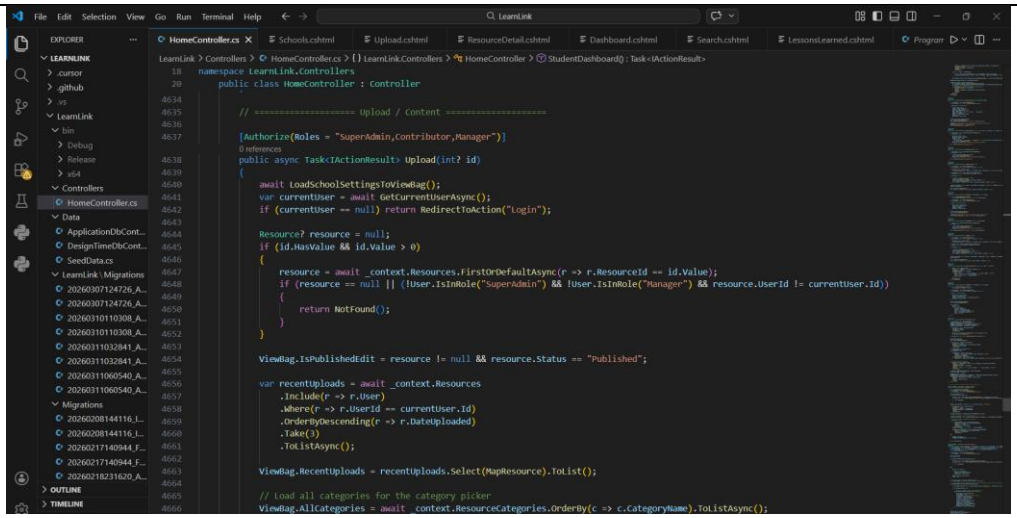
HomeController.cs:2217: Repository(...) builds the main published-resource query, applies filters, applies personalized recommendations, and supports both browse and search modes.



HomeController.cs:2501: ResourceDetail(int id) is the primary resource access action. It enforces ownership rules, access levels, access expiry, reading history updates, comment loading, file preview preparation, and related-resource retrieval.



HomeController.cs:4622: Upload(int? id) GET loads the upload or edit form for contributors and reviewers.
HomeController.cs:4682: Upload(...) POST is the main resource persistence pipeline. It handles create or edit operations, file upload to storage, tags, categories, access policies, version history, restricted user grants, and reviewer notifications.



```
namespace LearnLink.Controllers
    public class HomeController : Controller

        // ================== Upload / Content ==================

        [Authorize(Roles = "SuperAdmin,Contributor,Manager")]
        public async Task<IActionResult> Upload(int? id)
        {
            await LoadSchoolSettingsToViewBag();
            var currentUser = await GetCurrentUserAsync();
            if (currentUser == null) return RedirectToAction("Login");

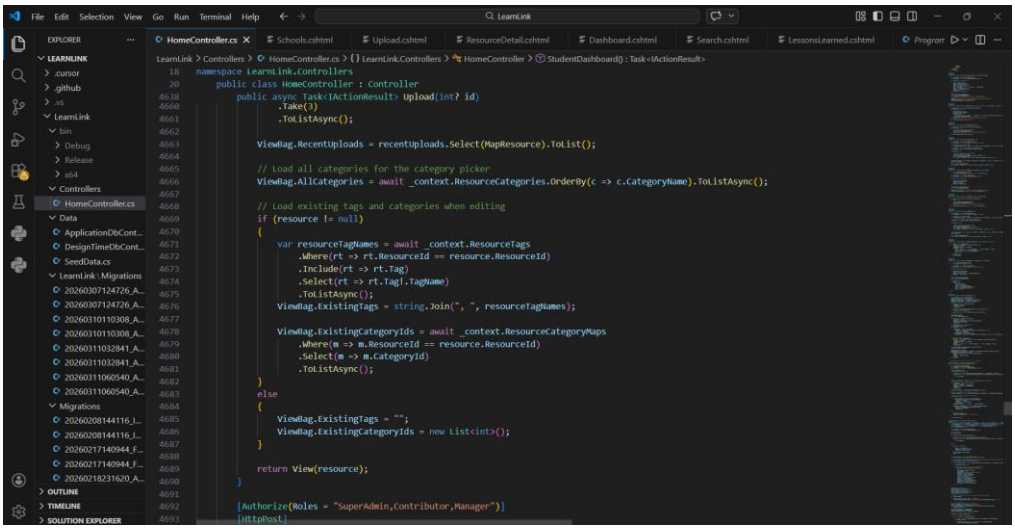
            Resource? resource = null;
            if (id.HasValue && id.Value > 0)
            {
                resource = await _context.Resources.FirstOrDefaultAsync(r => r.ResourceId == id.Value);
                if (resource == null || (!User.IsInRole("SuperAdmin") && !User.IsInRole("Manager") && resource.UserId != currentUser.Id))
                {
                    return NotFound();
                }
            }

            ViewBag.IsPublishedEdit = resource != null && resource.Status == "Published";

            var recentUploads = await _context.Resources
                .Include(r => r.User)
                .Where(r => r.UserId == currentUser.Id)
                .OrderByDescending(r => r.DateUploaded)
                .Take(3)
                .ToListAsync();

            ViewBag.RecentUploads = recentUploads.Select(MapResource).ToList();

            // Load all categories for the category picker
            ViewBag.AllCategories = await _context.ResourceCategories.OrderBy(c => c.CategoryName).ToListAsync();
```



```
namespace LearnLink.Controllers
    public class HomeController : Controller
        public async Task<IActionResult> Upload(int? id)
                .Take(3)
                .ToListAsync();

            ViewBag.RecentUploads = recentUploads.Select(MapResource).ToList();

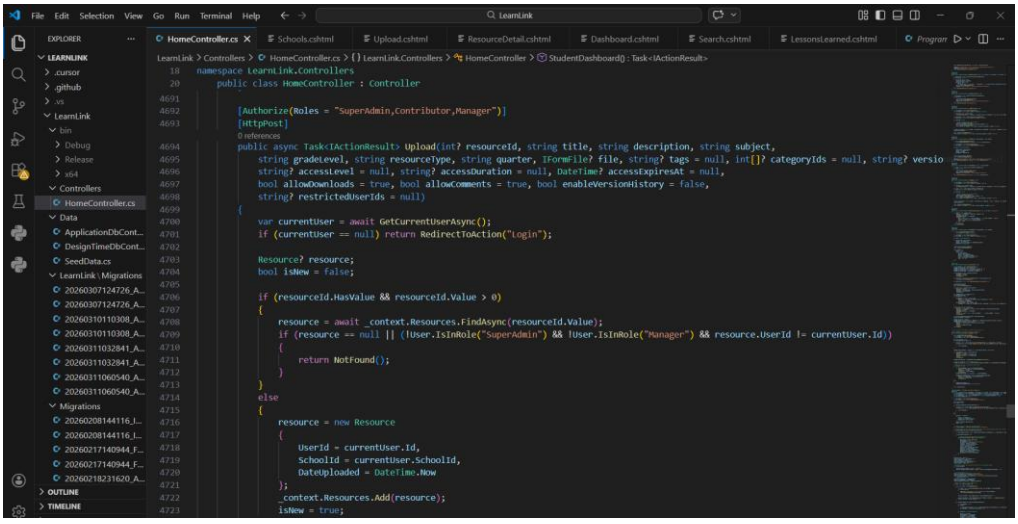
            // Load all categories for the category picker
            ViewBag.AllCategories = await _context.ResourceCategories.OrderBy(c => c.CategoryName).ToListAsync();

            // Load existing tags and categories when editing
            if (resource != null)
            {
                var resourceTagNames = await _context.ResourceTags
                    .Where(rt => rt.ResourceId == resource.ResourceId)
                    .Include(rt => rt.Tag)
                    .Select(rt => rt.Tag!.TagName)
                    .ToListAsync();
                ViewBag.ExistingTags = string.Join(", ", resourceTagNames);

                ViewBag.ExistingCategoryIds = await _context.ResourceCategoryMaps
                    .Where(m => m.ResourceId == resource.ResourceId)
                    .Select(m => m.CategoryId)
                    .ToListAsync();
            }
            else
            {
                ViewBag.ExistingTags = "";
                ViewBag.ExistingCategoryIds = new List<int>();
            }

            return View(resource);
        }

        [Authorize(Roles = "SuperAdmin,Contributor,Manager")]
        [HttpPost]
```

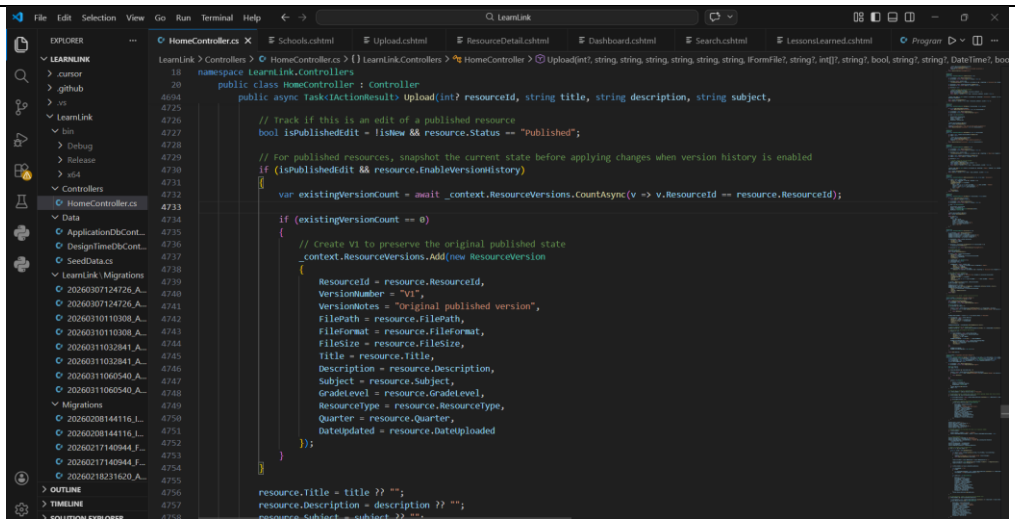


```
namespace LearnLink.Controllers
    public class HomeController : Controller

        [Authorize(Roles = "SuperAdmin,Contributor,Manager")]
        [HttpPost]
        public async Task<IActionResult> Upload(int? resourceId, string title, string description, string subject,
            string gradeLevel, string resourceType, string quarter, IFormFile? file, string? tags = null, int[]? categoryIds = null, string? versio
            string? accessLevel = null, string? accessDuration = null, DateTime? accessExpiresAt = null,
            bool allowDownloads = true, bool allowComments = true, bool enableVersionHistory = false,
            string? restrictedUserIds = null)
        {
            var currentUser = await GetCurrentUserAsync();
            if (currentUser == null) return RedirectToAction("Login");

            Resource? resource;
            bool isNew = false;

            if (resourceId.HasValue && resourceId.Value > 0)
            {
                resource = await _context.Resources.FindAsync(resourceId.Value);
                if (resource == null || (!User.IsInRole("SuperAdmin") && !User.IsInRole("Manager") && resource.UserId != currentUser.Id))
                {
                    return NotFound();
                }
            }
            else
            {
                resource = new Resource
                {
                    UserId = currentUser.Id,
                    SchoolId = currentUser.SchoolId,
                    DateUploaded = DateTime.Now
                };
                _context.Resources.Add(resource);
                isNew = true;
```



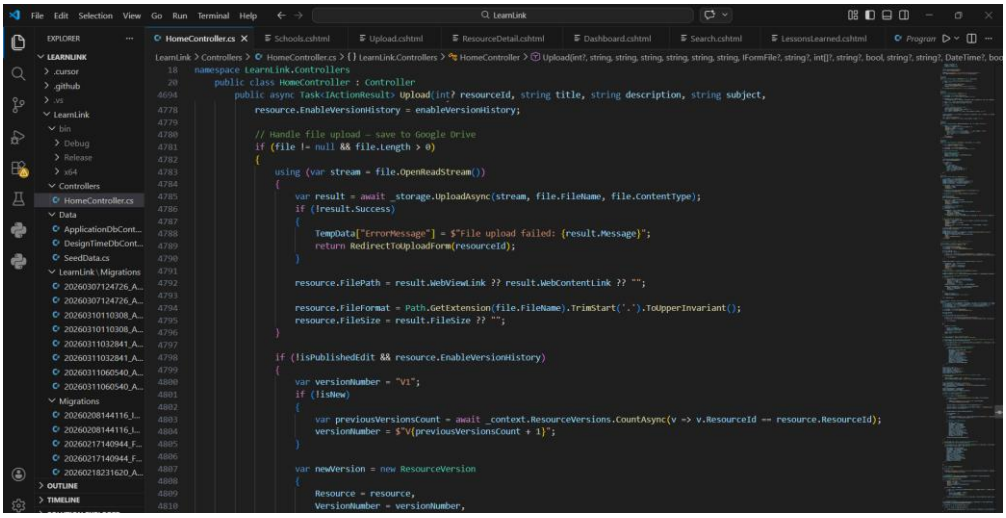
```
namespace LearnLink.Controllers
    public class HomeController : Controller
        public async Task<IActionResult> Upload(int? resourceId, string title, string description, string subject,

            // Track if this is an edit of a published resource
            bool isPublishedEdit = !isNew && resource.Status == "Published";

            // For published resources, snapshot the current state before applying changes when version history is enabled
            if (isPublishedEdit && resource.EnableVersionHistory)
            {
                var existingVersionCount = await _context.ResourceVersions.CountAsync(v => v.ResourceId == resource.ResourceId);

                if (existingVersionCount == 0)
                {
                    // Create V1 to preserve the original published state
                    _context.ResourceVersions.Add(new ResourceVersion
                    {
                        ResourceId = resource.ResourceId,
                        VersionNumber = "V1",
                        VersionNotes = "Original published version",
                        FilePath = resource.FilePath,
                        FileFormat = resource.FileFormat,
                        FileSize = resource.FileSize,
                        Title = resource.Title,
                        Description = resource.Description,
                        Subject = resource.Subject,
                        GradeLevel = resource.GradeLevel,
                        ResourceType = resource.ResourceType,
                        Quarter = resource.Quarter,
                        DateUpdated = resource.DateUploaded
                    });
                }
            }

            resource.Title = title ?? "";
            resource.Description = description ?? "";
```



```
namespace LearnLink.Controllers
    public class HomeController : Controller
        public async Task<IActionResult> Upload(int? resourceId, string title, string description, string subject,
            resource.EnableVersionHistory = enableVersionHistory;

            // Handle file upload — save to Google Drive
            if (file != null && file.Length > 0)
            {
                using (var stream = file.OpenReadStream())
                {
                    var result = await _storage.UploadAsync(stream, file.FileName, file.ContentType);
                    if (!result.Success)
                    {
                        TempData["ErrorMessage"] = $"File upload failed: {result.Message}";
                        return RedirectToUploadForm(resourceId);
                    }

                    resource.FilePath = result.WebViewLink ?? result.WebContentLink ?? "";

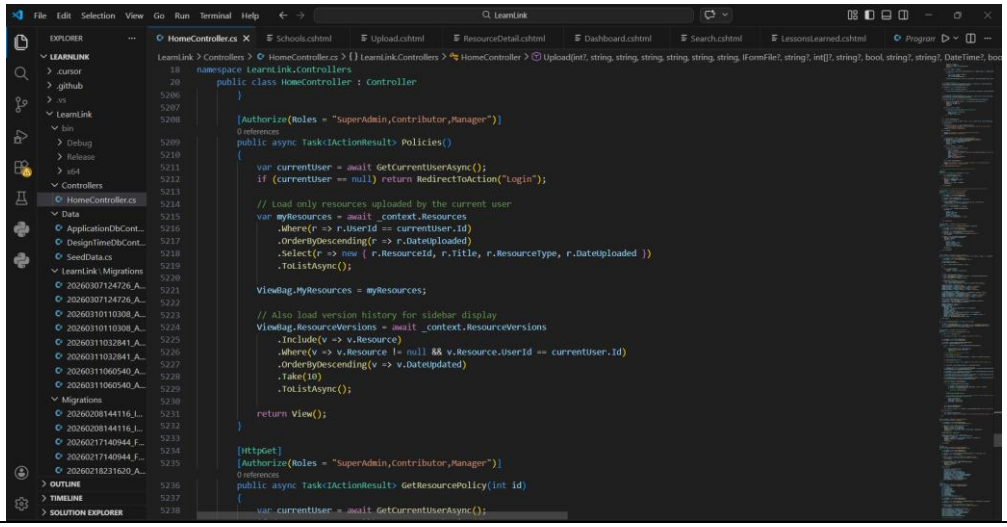
                    resource.FileFormat = Path.GetExtension(file.FileName).TrimStart('.').ToUpperInvariant();
                    resource.FileSize = result.FileSize ?? "";
                }

                if (!isPublishedEdit && resource.EnableVersionHistory)
                {
                    var versionNumber = "V1";
                    if (!isNew)
                    {
                        var previousVersionsCount = await _context.ResourceVersions.CountAsync(v => v.ResourceId == resource.ResourceId);
                        versionNumber = $"V{previousVersionsCount + 1}";
                    }

                    var newVersion = new ResourceVersion
                    {
                        Resource = resource,
                        VersionNumber = versionNumber,
```

HomeController.cs:5197: SaveResourcePolicy(...) updates access level, duration, interaction switches, moderation settings, and restricted-user assignments.

Lessons Learned Module



```
namespace LearnLink.Controllers
    public class HomeController : Controller
        }

        [Authorize(Roles = "SuperAdmin,Contributor,Manager")]
        0 references
        public async Task<IActionResult> Policies()
        {
            var currentUser = await GetCurrentUserAsync();
            if (currentUser == null) return RedirectToAction("Login");

            // Load only resources uploaded by the current user
            var myResources = await _context.Resources
                .Where(r => r.UserId == currentUser.Id)
                .OrderByDescending(r => r.DateUploaded)
                .Select(r => new { r.ResourceId, r.Title, r.ResourceType, r.DateUploaded })
                .ToListAsync();

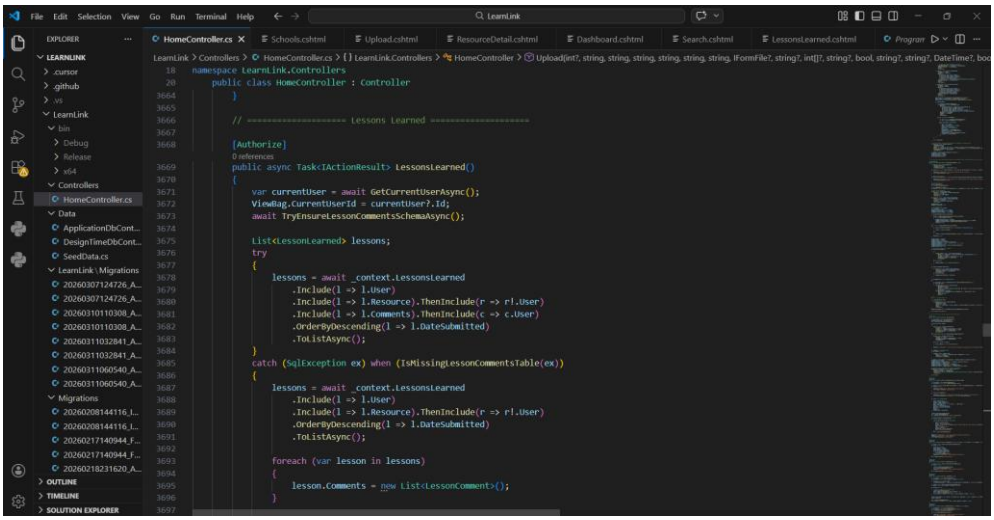
            ViewBag.MyResources = myResources;

            // Also load version history for sidebar display
            ViewBag.ResourceVersions = await _context.ResourceVersions
                .Include(v => v.Resource)
                .Where(v => v.Resource != null && v.Resource.UserId == currentUser.Id)
                .OrderByDescending(v => v.DateUpdated)
                .Take(10)
                .ToListAsync();

            return View();
        }

        [HttpGet]
        [Authorize(Roles = "SuperAdmin,Contributor,Manager")]
        0 references
        public async Task<IActionResult> GetResourcePolicy(int id)
        {
            var currentUser = await GetCurrentUserAsync();
```

HomeController.cs:3659: LessonsLearned() retrieves lessons, uploader metadata, likes, and lesson replies.



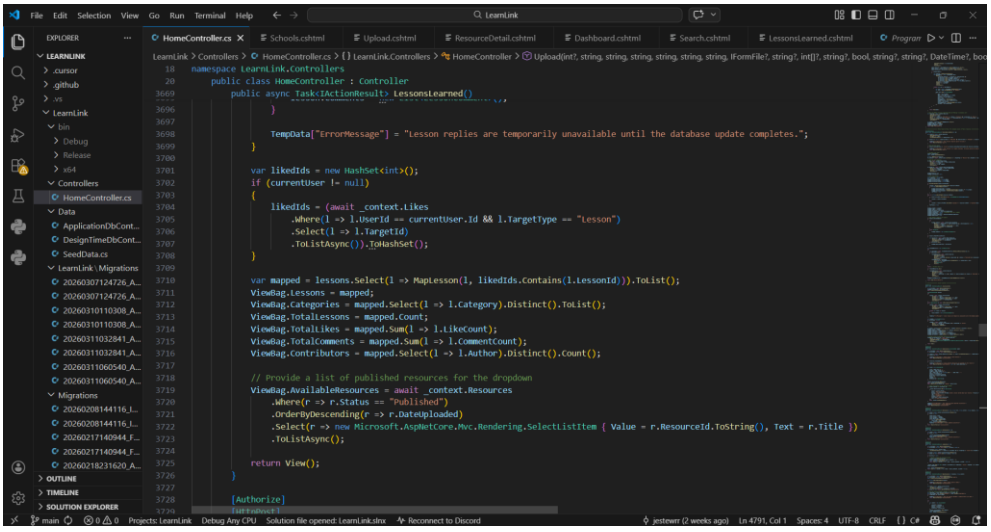
```
namespace LearnLink.Controllers
    public class HomeController : Controller
        // ================== Lessons Learned ==================

        [Authorize]
        public async Task<IActionResult> LessonsLearned()
        {
            var currentUser = await GetCurrentUserAsync();
            ViewBag.CurrentUserId = currentUser?.Id;
            await TryEnsureLessonCommentsSchemaAsync();

            List<LessonLearned> lessons;
            try
            {
                lessons = await _context.LessonsLearned
                    .Include(l => l.User)
                    .Include(l => l.Resource).ThenInclude(r => r!.User)
                    .Include(l => l.Comments).ThenInclude(c => c.User)
                    .OrderByDescending(l => l.DateSubmitted)
                    .ToListAsync();
            }
            catch (SqlException ex) when (IsMissingLessonCommentsTable(ex))
            {
                lessons = await _context.LessonsLearned
                    .Include(l => l.User)
                    .Include(l => l.Resource).ThenInclude(r => r!.User)
                    .OrderByDescending(l => l.DateSubmitted)
                    .ToListAsync();

                foreach (var lesson in lessons)
                {
                    lesson.Comments = new List<LessonComment>();
                }
```

HomeController.cs:3720: SubmitLesson(LessonViewModel model) creates a lesson attached to a selected resource and notifies the resource owner.



```
namespace LearnLink.Controllers
    public class HomeController : Controller
        public async Task<IActionResult> LessonsLearned()

            TempData["ErrorMessage"] = "Lesson replies are temporarily unavailable until the database update completes.";
        }

        var likedIds = new HashSet<int>();
        if (currentUser != null)
        {
            likedIds = (await _context.Likes
                .Where(l => l.UserId == currentUser.Id && l.TargetType == "Lesson")
                .Select(l => l.TargetId)
                .ToListAsync()).ToHashSet();
        }

        var mapped = lessons.Select(l => MapLesson(l, likedIds.Contains(l.LessonId))).ToList();
        ViewBag.Lessons = mapped;
        ViewBag.Categories = mapped.Select(l => l.Category).Distinct().ToList();
        ViewBag.TotalLessons = mapped.Count;
        ViewBag.TotalLikes = mapped.Sum(l => l.LikeCount);
        ViewBag.TotalComments = mapped.Sum(l => l.CommentCount);
        ViewBag.Contributors = mapped.Select(l => l.Author).Distinct().Count();

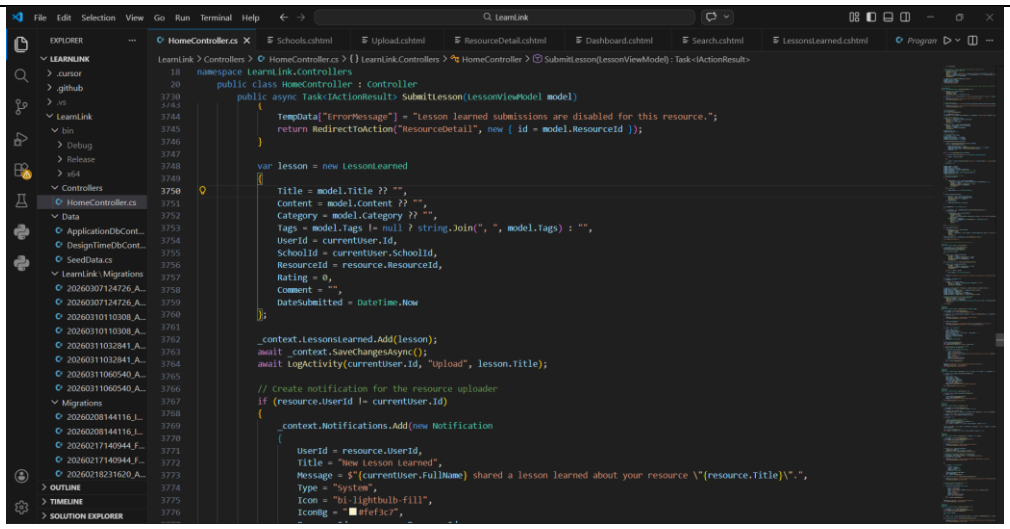
        // Provide a list of published resources for the dropdown
        ViewBag.AvailableResources = await _context.Resources
            .Where(r => r.Status == "Published")
            .OrderByDescending(r => r.DateUploaded)
            .Select(r => new Microsoft.AspNetCore.Mvc.Rendering.SelectListItem { Value = r.ResourceId.ToString(), Text = r.Title })
            .ToListAsync();

        return View();
    }

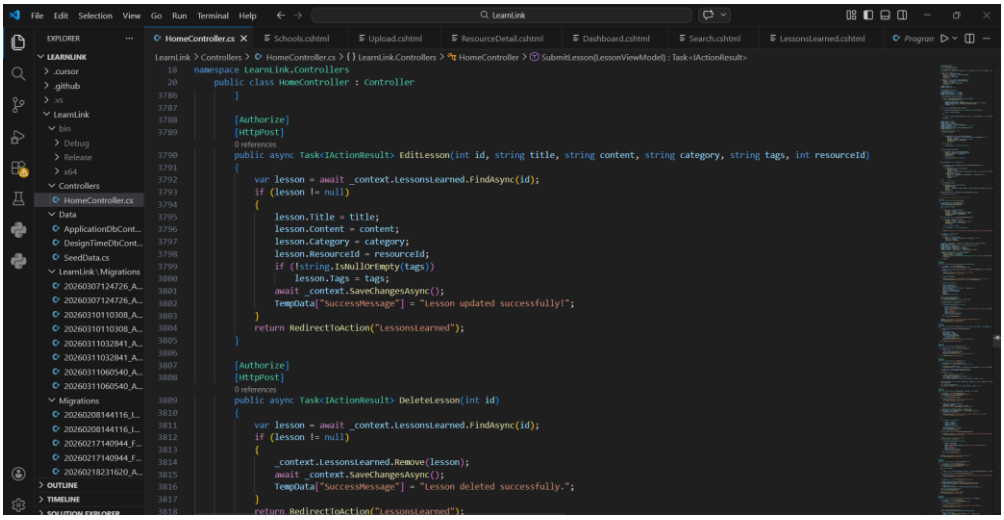
    [Authorize]
```

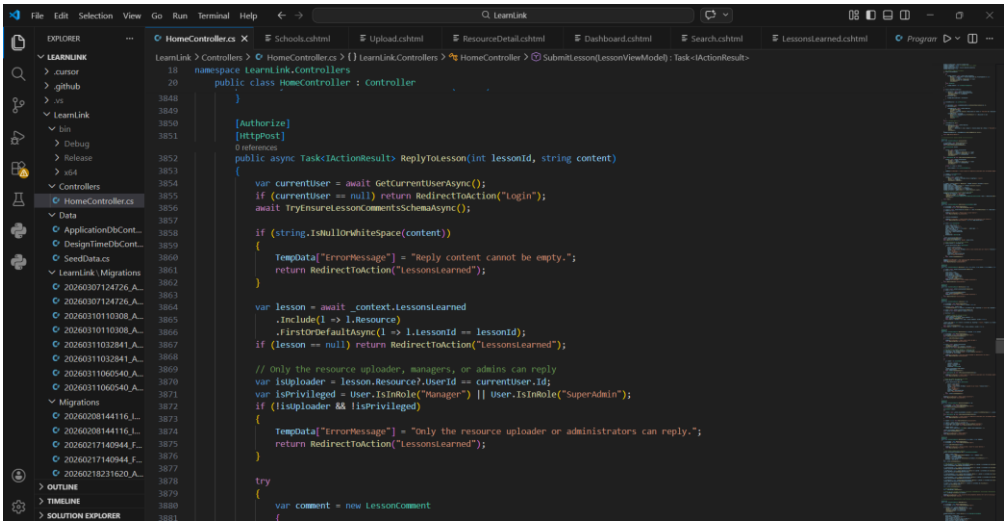
The CRUD of Lesson Learned: Add, Edit, Delete, Read

The users will create a lesson learned bassed on the resources that they read from Resource Detail. In the bottom of the resources there's a button for submitting it which the title, content, will be automatically display in the modal. Also they can edit or delete their lesson learned.



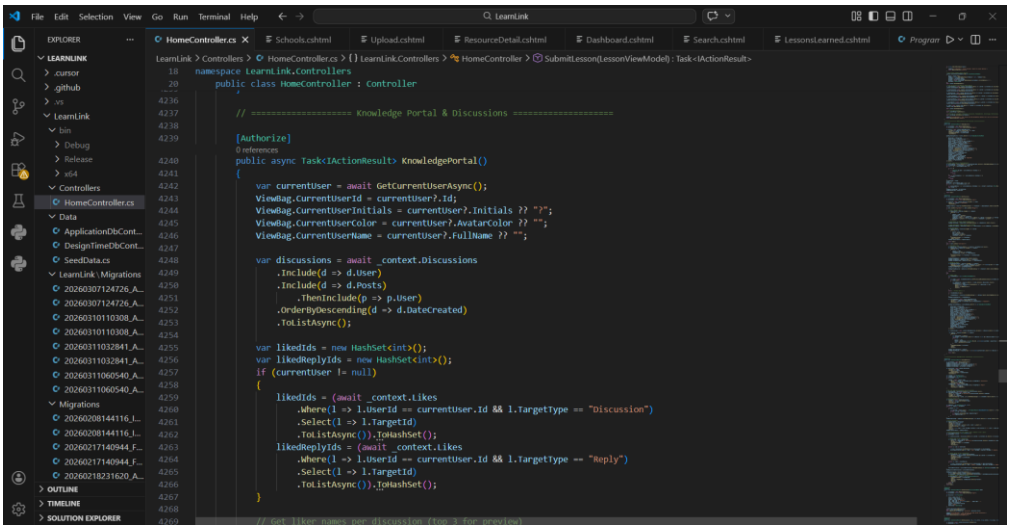
The contributor can reply to acknowledge their learnings from the user, and it will notify either from both users.





Knowledge Portal and Discussions:

HomeController: KnowledgePortal() loads the main discussion board, popular tags, active contributors, and activity notifications.



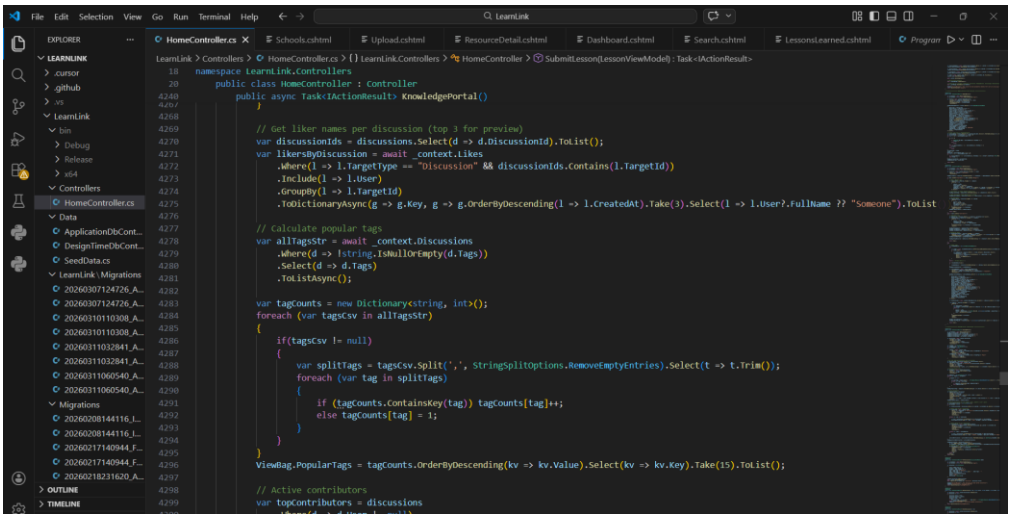
```
namespace LearnLink.Controllers
    public class HomeController : Controller
        // ================== Knowledge Portal & Discussions ==================

        [Authorize]
        public async Task<IActionResult> KnowledgePortal()
        {
            var currentUser = await GetCurrentUserAsync();
            ViewBag.CurrentUserId = currentUser?.Id;
            ViewBag.CurrentUserInitials = currentUser?.Initials ?? "?";
            ViewBag.CurrentUserColor = currentUser?.AvatarColor ?? "";
            ViewBag.CurrentUserName = currentUser?.FullName ?? "";

            var discussions = await _context.Discussions
                .Include(d => d.User)
                .Include(d => d.Posts)
                    .ThenInclude(p => p.User)
                .OrderByDescending(d => d.DateCreated)
                .ToListAsync();

            var likedIds = new HashSet<int>();
            var likedReplyIds = new HashSet<int>();
            if (currentUser != null)
            {
                likedIds = (await _context.Likes
                    .Where(l => l.UserId == currentUser.Id && l.TargetType == "Discussion")
                    .Select(l => l.TargetId)
                    .ToListAsync()).ToHashSet();
                likedReplyIds = (await _context.Likes
                    .Where(l => l.UserId == currentUser.Id && l.TargetType == "Reply")
                    .Select(l => l.TargetId)
                    .ToListAsync()).ToHashSet();
            }
```

HomeController.cs:4348: PostDiscussion(...) creates a new knowledge-sharing thread.



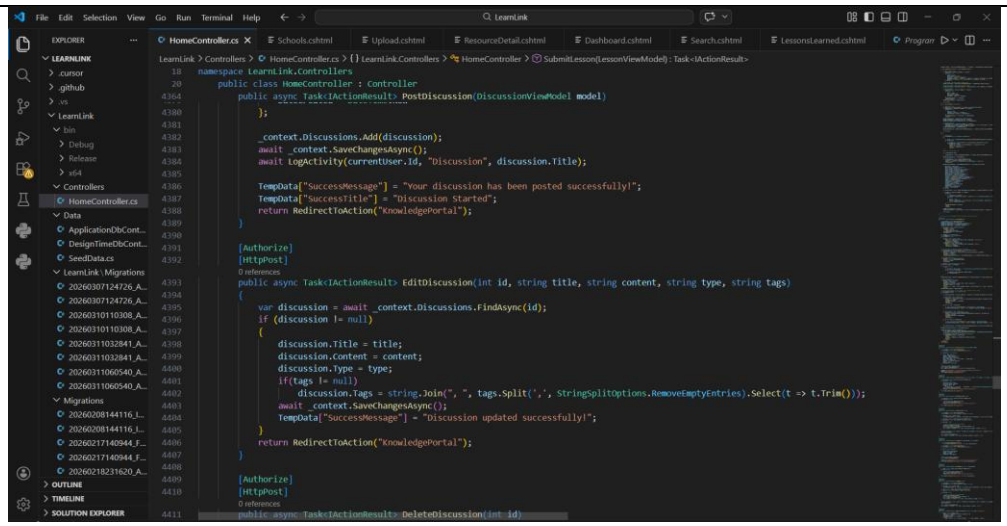
```
namespace LearnLink.Controllers
    public class HomeController : Controller
        public async Task<IActionResult> KnowledgePortal()

            // Get liker names per discussion (top 3 for preview)
            var discussionIds = discussions.Select(d => d.DiscussionId).ToList();
            var likersByDiscussion = await _context.Likes
                .Where(l => l.TargetType == "Discussion" && discussionIds.Contains(l.TargetId))
                .Include(l => l.User)
                .GroupBy(l => l.TargetId)
                .ToDictionaryAsync(g => g.Key, g => g.OrderByDescending(l => l.CreatedAt).Take(3).Select(l => l.User?.FullName ?? "Someone").ToList

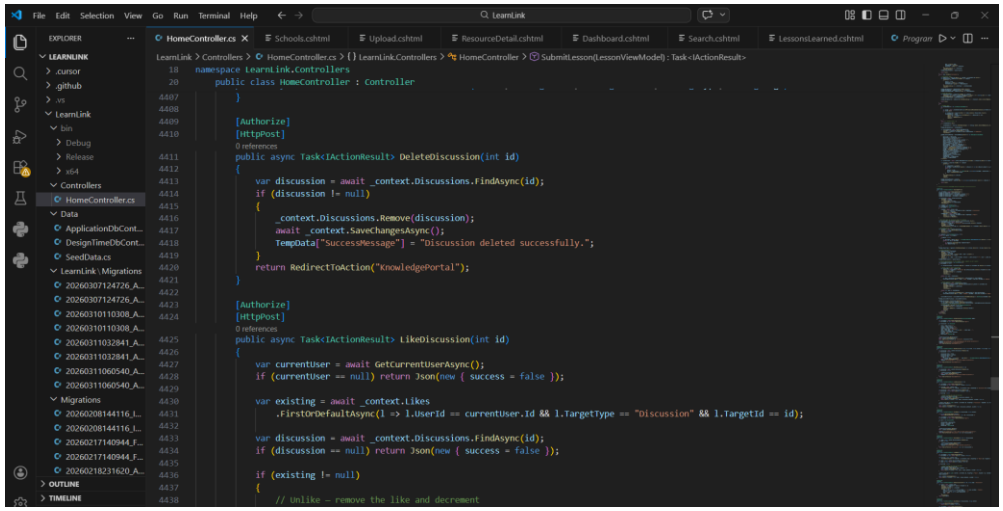
            // Calculate popular tags
            var allTagsStr = await _context.Discussions
                .Where(d => !string.IsNullOrEmpty(d.Tags))
                .Select(d => d.Tags)
                .ToListAsync();

            var tagCounts = new Dictionary<string, int>();
            foreach (var tagsCsv in allTagsStr)
            {
                if(tagsCsv != null)
                {
                    var splitTags = tagsCsv.Split(',', StringSplitOptions.RemoveEmptyEntries).Select(t => t.Trim());
                    foreach (var tag in splitTags)
                    {
                        if (tagCounts.ContainsKey(tag)) tagCounts[tag]++;
                        else tagCounts[tag] = 1;
                    }
                }
            }
            ViewBag.PopularTags = tagCounts.OrderByDescending(kv => kv.Value).Select(kv => kv.Key).Take(15).ToList();

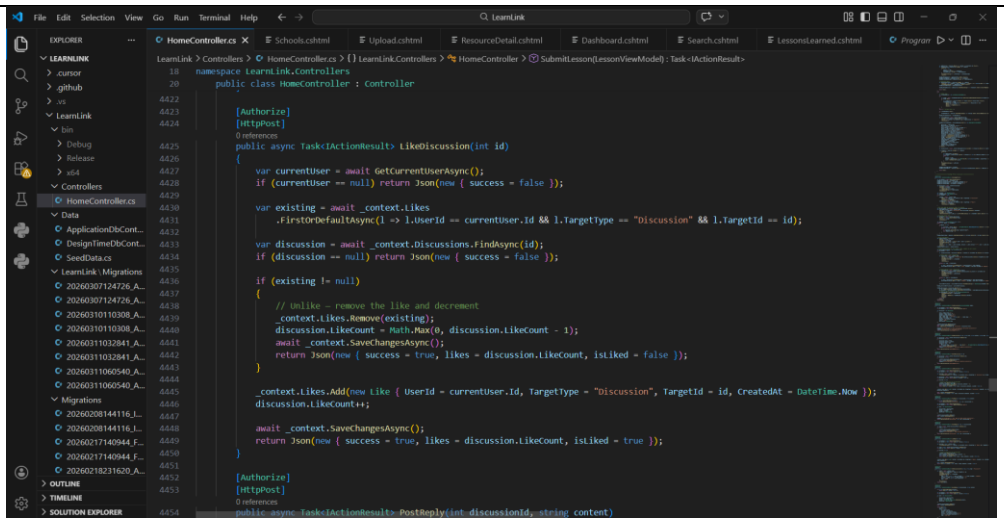
            // Active contributors
            var topContributors = discussions
```



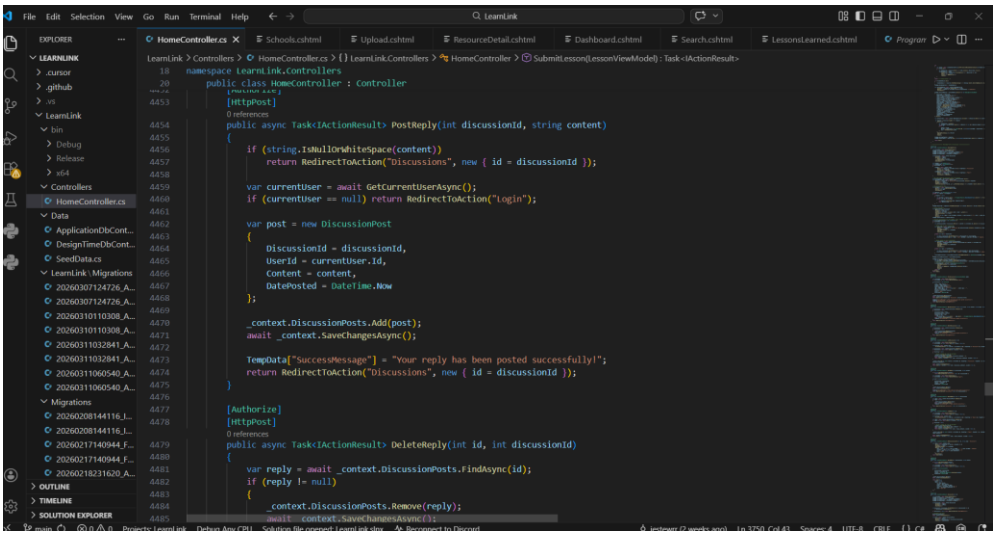
HomeController.cs:4377: EditDiscussion(...) updates an existing discussion.
HomeController.cs:4395: DeleteDiscussion(int id) removes a discussion.

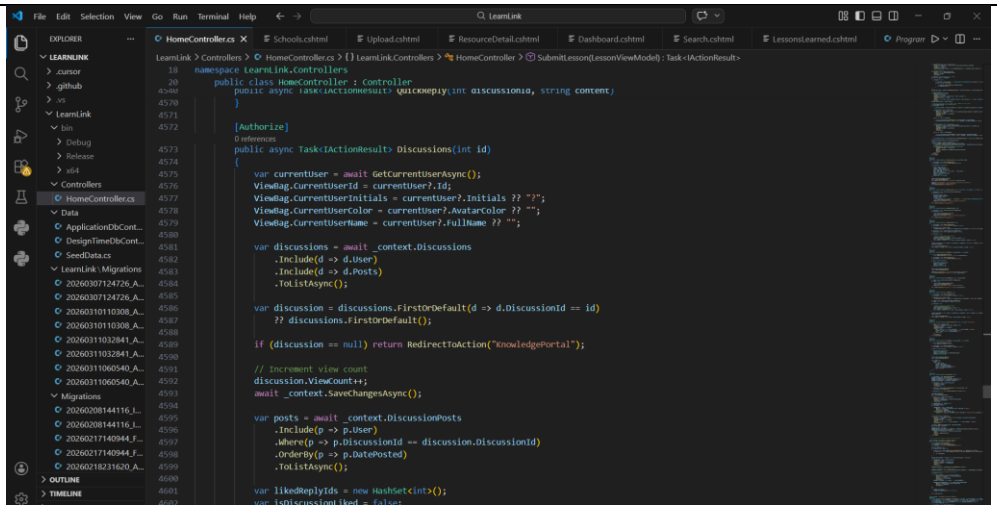


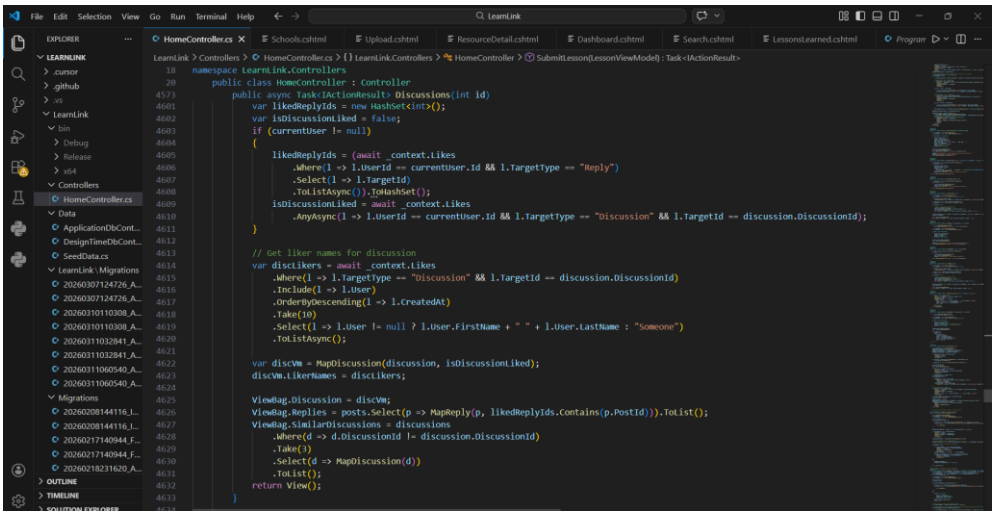
HomeController.cs:4409: LikeDiscussion(int id) toggles likes on a discussion thread.
HomeController.cs:4438: PostReply(int discussionId, string content) inserts a standard reply.



HomeController.cs:4463: DeleteReply(int id, int discussionId) removes a discussion reply.
HomeController.cs:4477: LikeReply(int id) toggles likes for discussion replies.
HomeController.cs:4557: Discussions(int id) renders a full discussion thread and loads ordered replies.

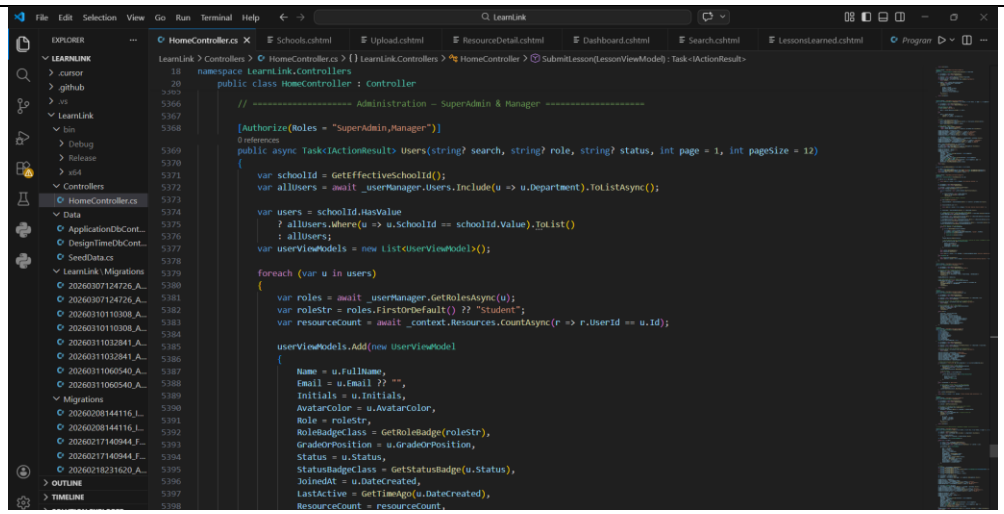




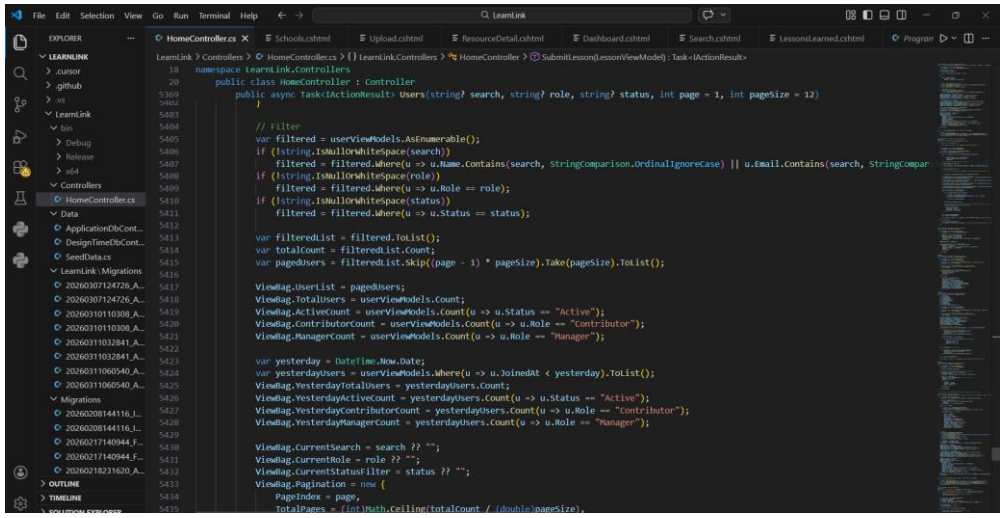


User Management:

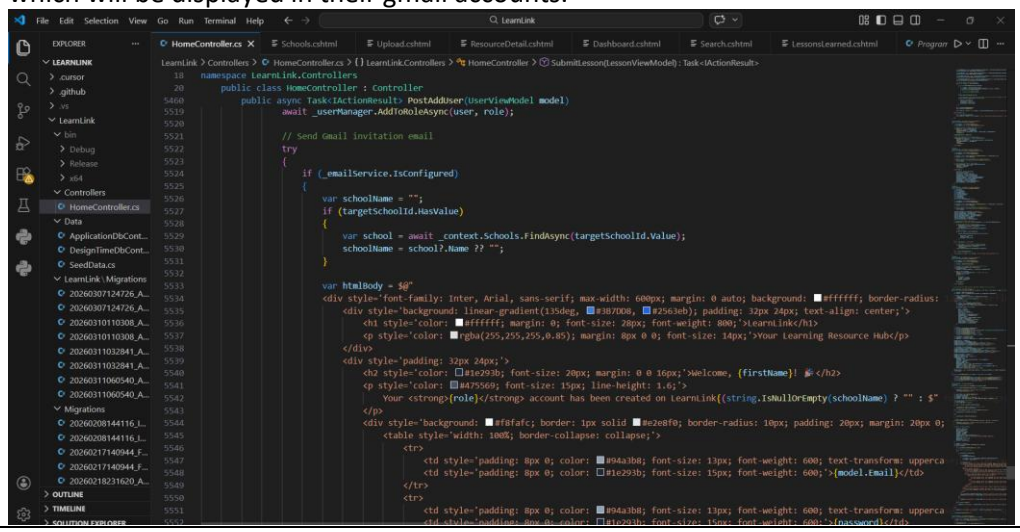
HomeController.cs:5294: Users(...) is the main administrative user management page.
HomeController.cs:5510: UserDetails(string email) loads a user profile together with activity and uploaded resources.



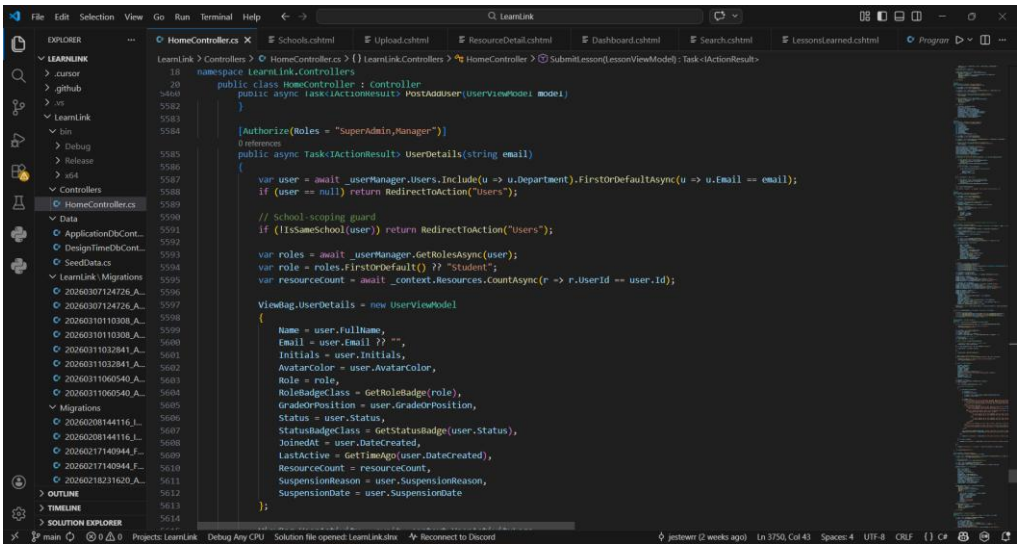
HomeController.cs:5385: PostAddUser(UserViewModel model) creates users administratively, assigns roles, and optionally sends invitation email.



Invitation after the admin clicks “send an invitation” . The code here is the string which will be displayed in their gmail accounts.



HomeController.cs:5510: UserDetails(string email) loads a user profile together with activity and uploaded resources.



```
namespace LearnLink.Controllers
    public class HomeController : Controller
        public async Task<IActionResult> PostAddUser(UserViewModel model)
        }

        [Authorize(Roles = "SuperAdmin,Manager")]
        0 references
        public async Task<IActionResult> UserDetails(string email)
        {
            var user = await _userManager.Users.Include(u => u.Department).FirstOrDefaultAsync(u => u.Email == email);
            if (user == null) return RedirectToAction("Users");

            // School-scoping guard
            if (!IsSameSchool(user)) return RedirectToAction("Users");

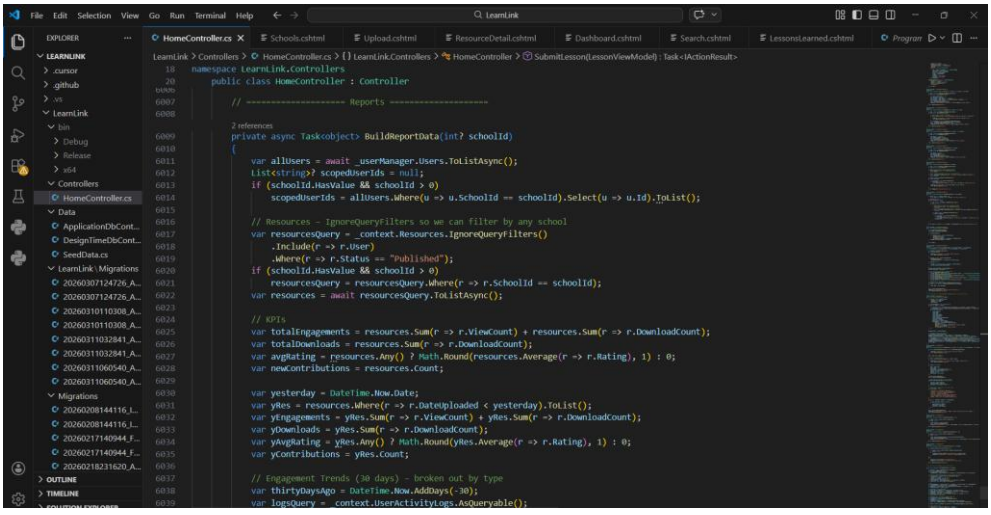
            var roles = await _userManager.GetRolesAsync(user);
            var role = roles.FirstOrDefault() ?? "Student";
            var resourceCount = await _context.Resources.CountAsync(r => r.UserId == user.Id);

            ViewBag.UserDetails = new UserViewModel
            {
                Name = user.FullName,
                Email = user.Email ?? "",
                Initials = user.Initials,
                AvatarColor = user.AvatarColor,
                Role = role,
                RoleBadgeClass = GetRoleBadge(role),
                GradeOrPosition = user.GradeOrPosition,
                Status = user.Status,
                StatusBadgeClass = GetStatusBadge(user.Status),
                JoinedAt = user.DateCreated,
                LastActive = GetTimeAgo(user.DateCreated),
                ResourceCount = resourceCount,
                SuspensionReason = user.SuspensionReason,
                SuspensionDate = user.SuspensionDate
            };
```

In creating manager role is different because the admin will gonna assign them in each school like for example. School 1 – Manager 1, so the manager will be assigned to that school.

Reports and Analytics

HomeController.cs:6094: Reports() renders the reporting dashboard and preloads initial data.



```
namespace LearnLink.Controllers
    public class HomeController : Controller

        // ========== Reports ==========

        2 references
        private async Task<object> BuildReportData(int? schoolId)
        {
            var allUsers = await _userManager.Users.ToListAsync();
            List<string>? scopedUserIds = null;
            if (schoolId.HasValue && schoolId > 0)
                scopedUserIds = allUsers.Where(u => u.SchoolId == schoolId).Select(u => u.Id).ToList();

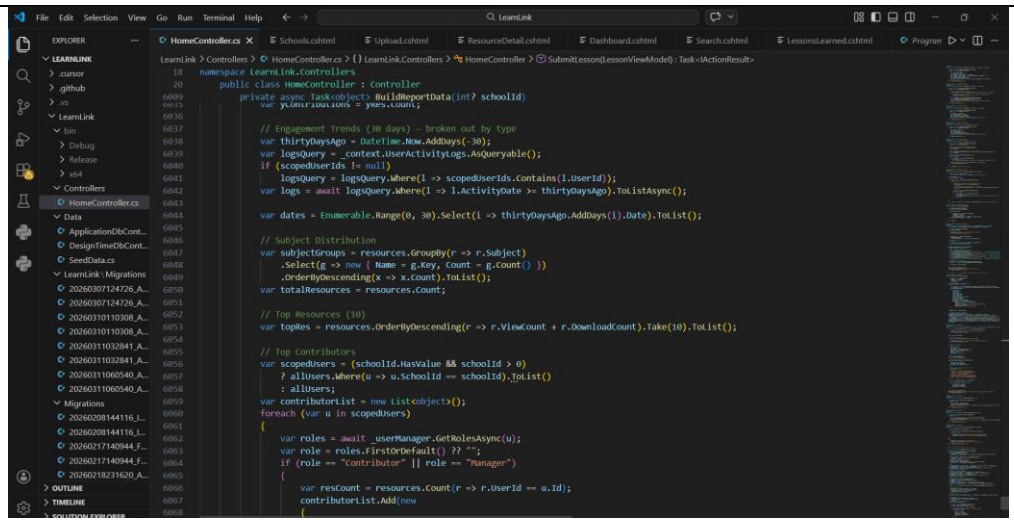
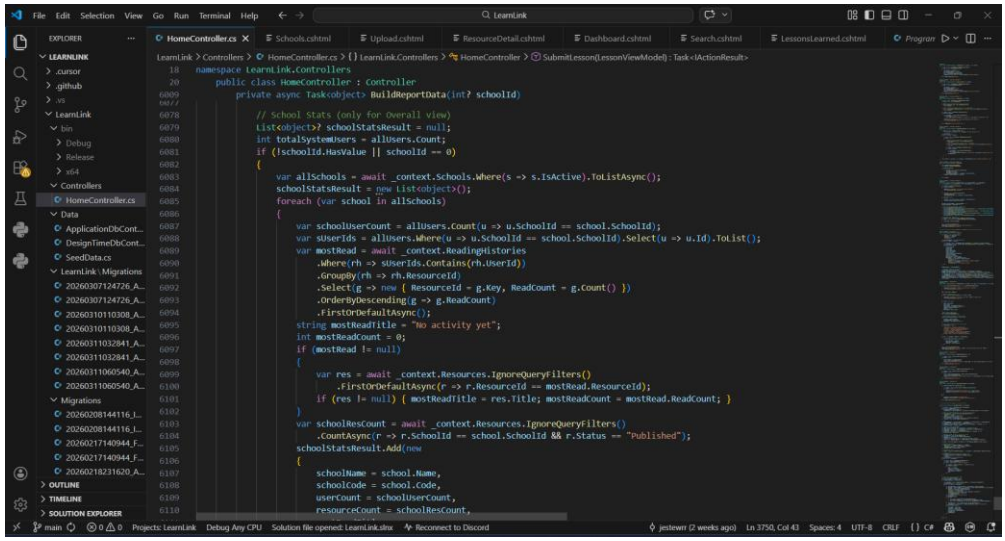
            // Resources - IgnoreQueryFilters so we can filter by any school
            var resourcesQuery = _context.Resources.IgnoreQueryFilters()
                .Include(r => r.User)
                .Where(r => r.Status == "Published");
            if (schoolId.HasValue && schoolId > 0)
                resourcesQuery = resourcesQuery.Where(r => r.SchoolId == schoolId);
            var resources = await resourcesQuery.ToListAsync();

            // KPIs
            var totalEngagements = resources.Sum(r => r.ViewCount) + resources.Sum(r => r.DownloadCount);
            var totalDownloads = resources.Sum(r => r.DownloadCount);
            var avgRating = resources.Any() ? Math.Round(resources.Average(r => r.Rating), 1) : 0;
            var newContributions = resources.Count;

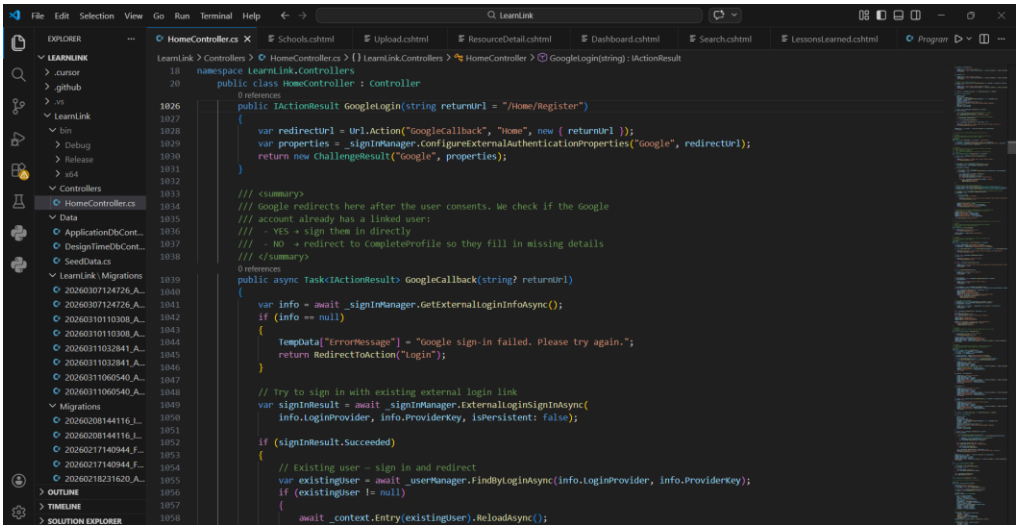
            var yesterday = DateTime.Now.Date;
            var yRes = resources.Where(r => r.DateUploaded < yesterday).ToList();
            var yEngagements = yRes.Sum(r => r.ViewCount) + yRes.Sum(r => r.DownloadCount);
            var yDownloads = yRes.Sum(r => r.DownloadCount);
            var yAvgRating = yRes.Any() ? Math.Round(yRes.Average(r => r.Rating), 1) : 0;
            var yContributions = yRes.Count;

            // Engagement Trends (30 days) - broken out by type
            var thirtyDaysAgo = DateTime.Now.AddDays(-30);
            var logsQuery = _context.UserActivityLogs.AsQueryable();
```

HomeController.cs:6113: ReportData(int? schoolId) returns report datasets for the currently selected scope.

	  The reports section is where all data will be gathered and display all the records from the number of users, resources perfromance, subject distributions, and more.
API FUNCTIONS/ FEATURES	**1.Google OAuth 2.0** Description: Google's authentication service for external sign-in. Purpose: lets users sign in or register with a Google account instead of only using local email/password authentication. Where the API is first implemented:

```
// Google Authentication
var googleClientId = builder.Configuration["Authentication:Google:ClientId"];
var googleClientSecret = builder.Configuration["Authentication:Google:ClientSecret"];
bool googleAuthEnabled = !string.IsNullOrEmpty(googleClientId) && !string.IsNullOrEmpty(googleClientSecret);
if (googleAuthEnabled)
{
    builder.Services.AddAuthentication()
        .AddGoogle(options =>
        {
            options.ClientId = googleClientId!;
            options.ClientSecret = googleClientSecret!;
            options.CallbackPath = "/signin-google";
        });
}
builder.Services.AddSingleton(new GoogleAuthFlag { IsEnabled = googleAuthEnabled });
builder.Services.Configure<SmtpSettings>(builder.Configuration.GetSection("Smtp"));
builder.Services.AddScoped<IEmailService, SmtpEmailService>();
```



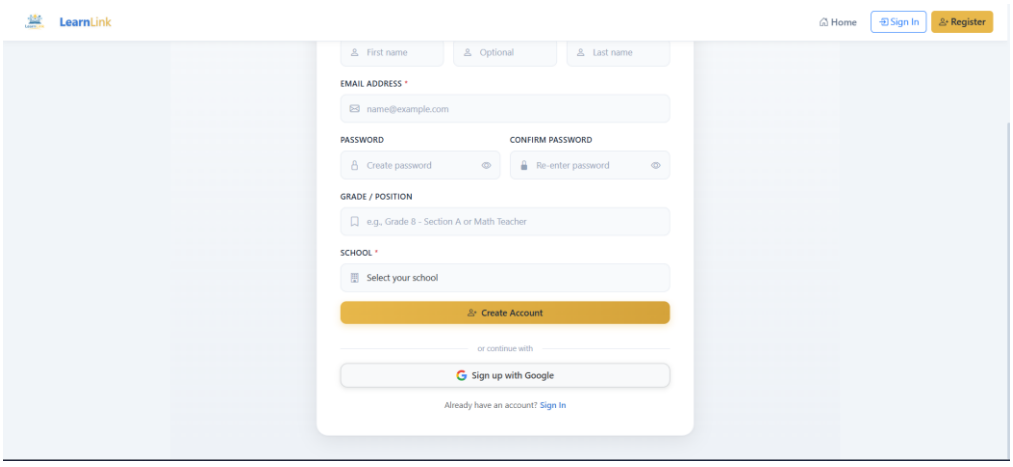
```
namespace LearnLink.Controllers
    public class HomeController : Controller
        public IActionResult GoogleLogin(string returnUrl = "/Home/Register")
        {
            var redirectUrl = Url.Action("GoogleCallback", "Home", new { returnUrl });
            var properties = _signInManager.ConfigureExternalAuthenticationProperties("Google", redirectUrl);
            return new ChallengeResult("Google", properties);
        }

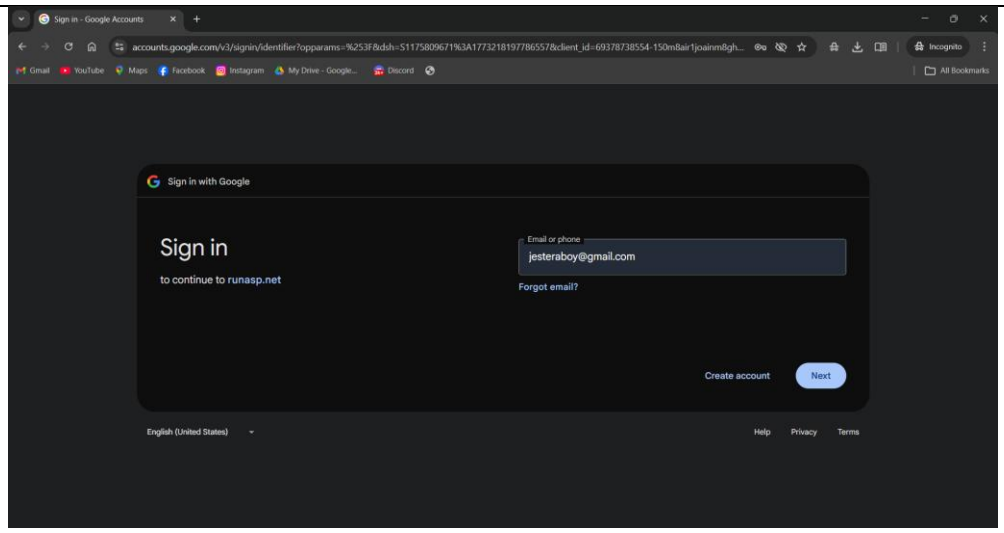
        /// <summary>
        /// Google redirects here after the user consents. We check if the Google
        /// account already has a linked user:
        ///   - YES → sign them in directly
        ///   - NO  → redirect to CompleteProfile so they fill in missing details
        /// </summary>
        public async Task<IActionResult> GoogleCallback(string? returnUrl)
        {
            var info = await _signInManager.GetExternalLoginInfoAsync();
            if (info == null)
            {
                TempData["ErrorMessage"] = "Google sign-in failed. Please try again.";
                return RedirectToAction("Login");
            }

            // Try to sign in with existing external login link
            var signInResult = await _signInManager.ExternalLoginSignInAsync(
                info.LoginProvider, info.ProviderKey, isPersistent: false);

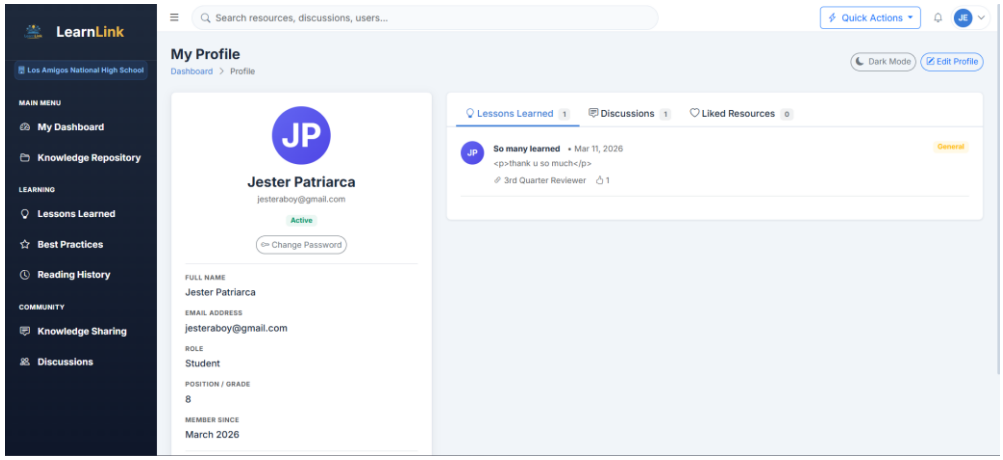
            if (signInResult.Succeeded)
            {
                // Existing user — sign in and redirect
                var existingUser = await _userManager.FindByLoginAsync(info.LoginProvider, info.ProviderKey);
                if (existingUser != null)
                {
                    await _context.Entry(existingUser).ReloadAsync();
```

Where the user clicks the sign in with google it will call the API and directly go to google authentication page.





For validation: (but when the user will sign in with google via register page they will be automatically assigned as Student Role/End User Role)



2. Google Drive API

Description: Google's cloud file storage API.

Purpose: stores uploaded learning resources in Google Drive, deletes files, downloads them, and generates preview/download links for users.

```
// Storage Configuration
var storageProvider = builder.Configuration["Storage:Provider"] ?? "Local";
builder.Services.AddHttpContextAccessor(); // Needed for LocalStorageService to build URLs

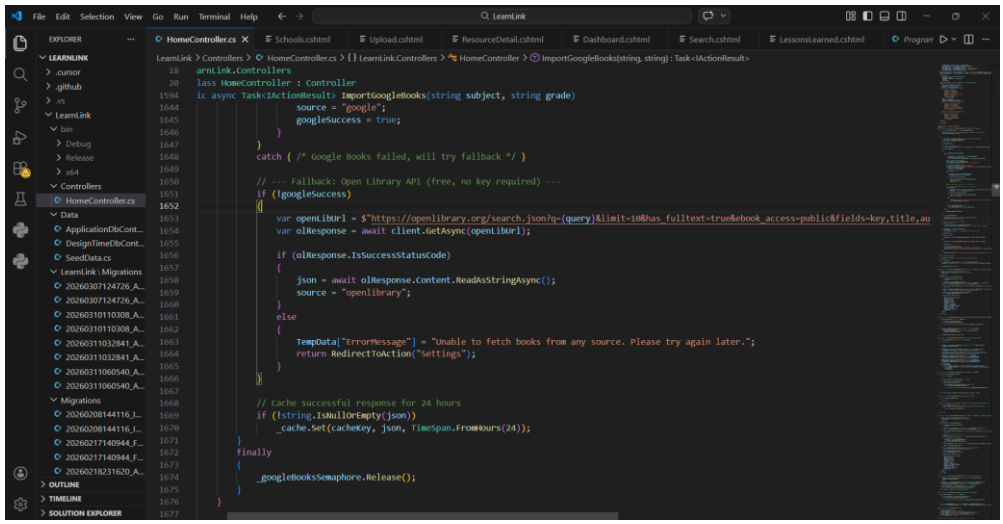
if (storageProvider == "GoogleDrive")
{
    builder.Services.Configure<GoogleDriveOptions>(builder.Configuration.GetSection("GoogleDrive"));
    builder.Services.AddScoped<IStorageService, GoogleDriveStorageService>();
}
else
{
    builder.Services.AddScoped<IStorageService, LocalStorageService>();
}
```

```
4 references
public class GoogleDriveStorageService : IStorageService
{
    8 references
    private readonly DriveService? _driveService;
    2 references
    private readonly string _sharedFolderId;
    8 references
    private readonly ILogger<GoogleDriveStorageService> _logger;

    0 references
    public GoogleDriveStorageService(IOptions<GoogleDriveOptions> options, ILogger<GoogleDriveStorageService> logger)
    {
        _logger = logger;
        _sharedFolderId = options.Value.SharedFolderId ?? "";

        if (string.IsNullOrEmpty(options.Value.ServiceAccountJson))
        {
            _logger.LogWarning("GoogleDrive:ServiceAccountJson not configured. Google Drive uploads will fail.");
            return;
        }

        try
        {
#pragma warning disable CS0618 // Type or member is obsolete
            var credential = GoogleCredential.FromJson(options.Value.ServiceAccountJson)
```



3. Google Books API

Description: Google's public books search API.

Purpose: imports free educational books into the repository so admins/managers can quickly add book resources by subject and grade level.

```
namespace LearnLink.Controllers
    public class HomeController : Controller

        [Authorize(Roles = "SuperAdmin,Manager")]
        [HttpPost]
        0 references
        public async Task<IActionResult> ImportGoogleBooks(string subject, string grade)
        {
            if (string.IsNullOrEmpty(subject) || string.IsNullOrEmpty(grade))
            {
                TempData["ErrorMessage"] = "Subject and Grade Level are required to import books.";
                return RedirectToAction("Settings");
            }

            try
            {
                var searchTerms = $"{subject} {grade} junior high school textbook education";
                var query = Uri.EscapeDataString(searchTerms);
                string cacheKey = $"BookImport_{query}";

                string? json = null;
                string source = "cache";

                // 1. Check if the response is cached
                if (_cache.TryGetValue(cacheKey, out string? cachedJson) && !string.IsNullOrEmpty(cachedJson))
                {
                    json = cachedJson;
                }
                else
                {
                    await _googleBooksSemaphore.WaitAsync();
                    try
                    {
                        var timeSinceLast = DateTime.UtcNow - _lastGoogleBooksRequest;
                        if (timeSinceLast.TotalSeconds < 1.0)
                            await Task.Delay(TimeSpan.FromSeconds(1.0 - timeSinceLast.TotalSeconds));
```

```
// --- Try Google Books API first ---
var apiKey = _configuration["GoogleBooks:ApiKey"];
var googleUrl = $"https://www.googleapis.com/books/v1/volumes?q={query}&maxResults=10&printType=books&langRestrict=en&filter=free-ebooks";
if (!string.IsNullOrEmpty(apiKey))
    googleUrl += $"&key={apiKey}";

bool googleSuccess = false;
try
{
    var response = await client.GetAsync(googleUrl);
    _lastGoogleBooksRequest = DateTime.UtcNow;

    if (response.IsSuccessStatusCode)
    {
        json = await response.Content.ReadAsStringAsync();
        source = "google";
        googleSuccess = true;
    }
}
catch { /* Google Books failed, will try fallback */ }
```

```
private async Task<int> ImportFromGoogleBooksJson(string json, string subject, string grade, ApplicationUser currentUser)
{
    var data = System.Text.Json.JsonDocument.Parse(json);
    if (!data.RootElement.TryGetProperty("items", out var items))
        return 0;

    int count = 0;
    foreach (var item in items.EnumerateArray())
    {
        if (!IsGoogleBookFreelyAccessible(item))
            continue;

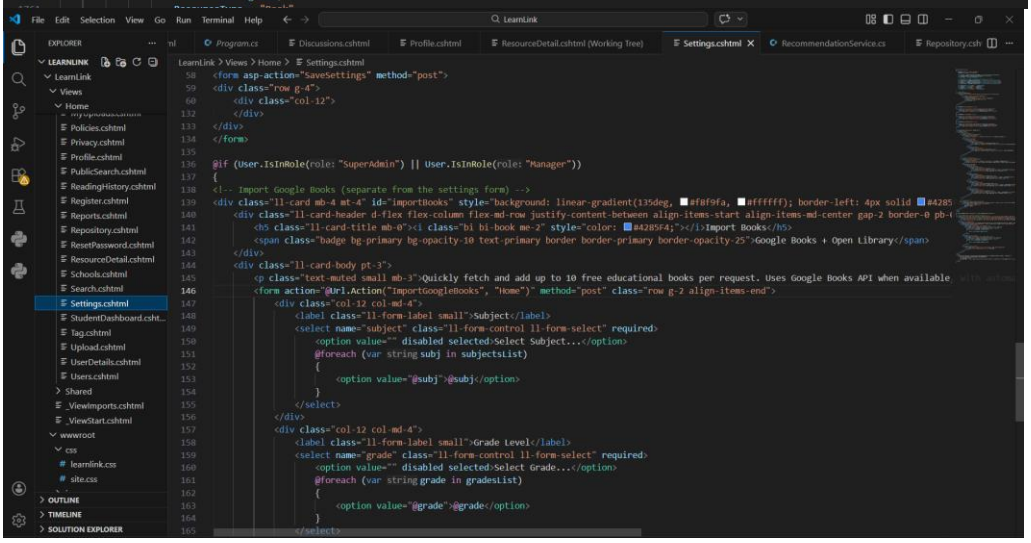
        var volumeInfo = item.GetProperty("volumeInfo");
        var title = volumeInfo.TryGetProperty("title", out var t) ? t.GetString() : "Unknown Title";
        if (title != null && title.Length > 95) title = title.Substring(0, 95) + "...";

        if (await _context.Resources.AnyAsync(r => r.Title == title && r.ResourceType == "Book" && r.SchoolId == currentUser.SchoolId))
            continue;

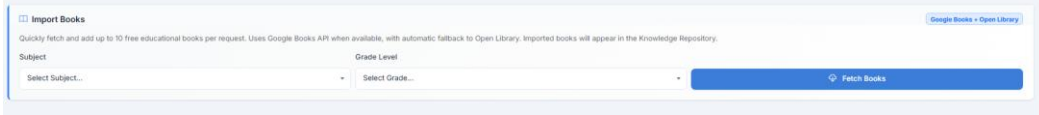
        var desc = volumeInfo.TryGetProperty("description", out var d) ? d.GetString() : "No description available.";
        if (desc != null && desc.Length > 490) desc = desc.Substring(0, 490) + "...";

        var previewLink = BuildGoogleBooksLink(item, volumeInfo);
        if (string.IsNullOrEmpty(previewLink)) continue;

        _context.Resources.Add(new Resource
        {
            Title = title ?? "Unknown Book",
            Description = desc ?? "",
            Subject = subject,
            GradeLevel = grade,
```



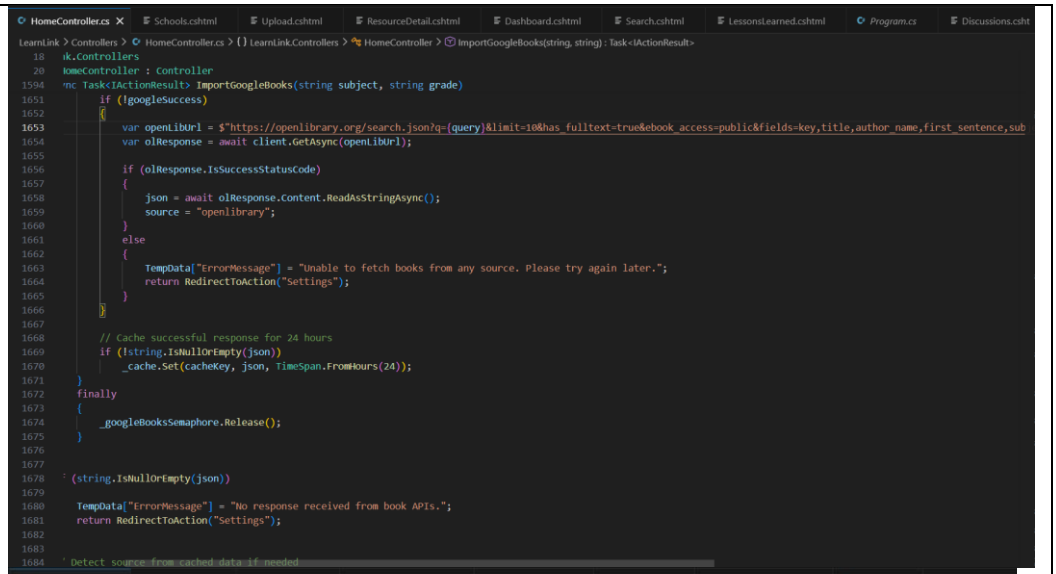
In the system:



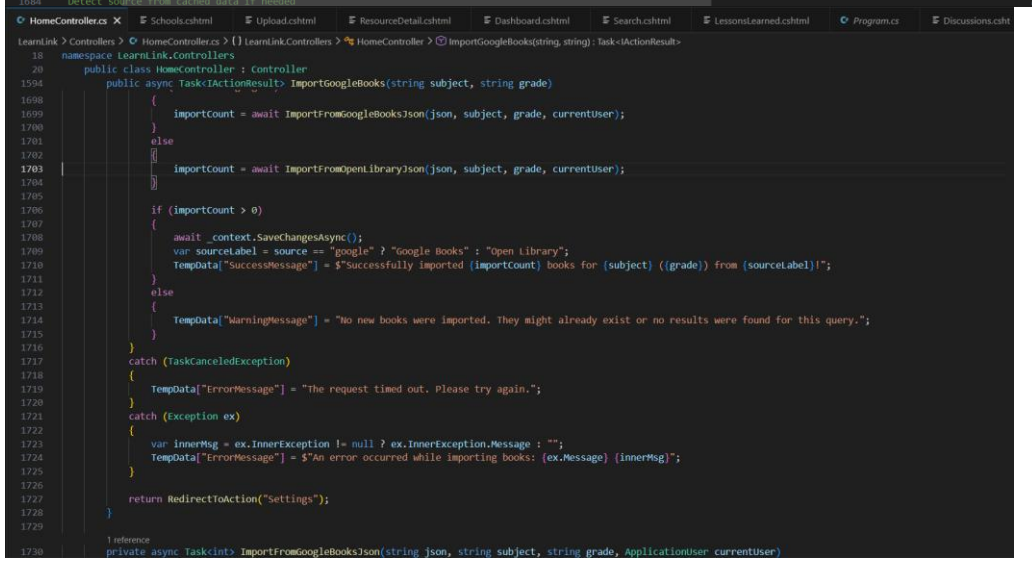
4. Open Library API

Description: Open Library's public catalog/search API for open books.

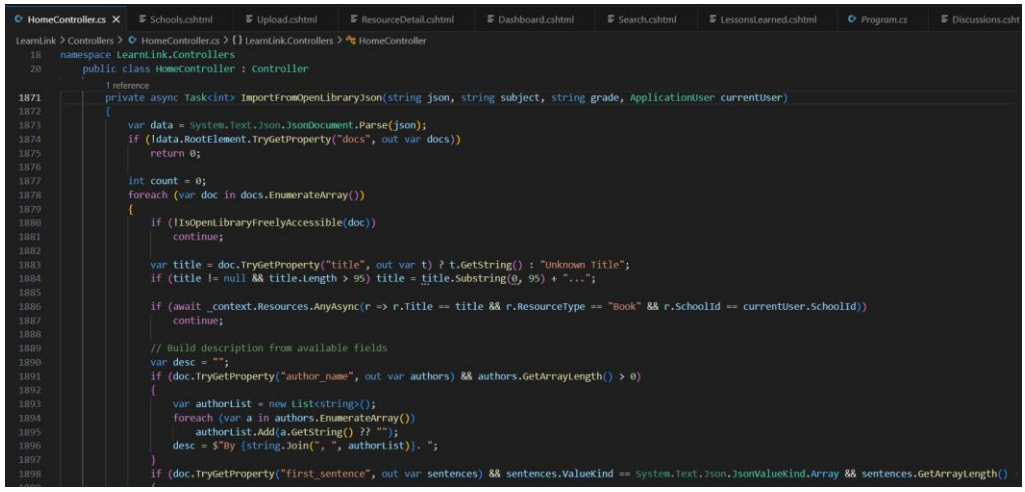
Purpose: acts as a fallback source when Google Books is unavailable or returns no usable result, so LearnLink can still import publicly accessible books.



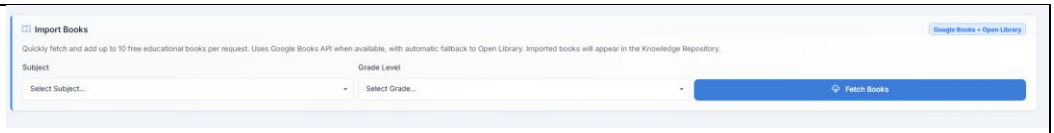
```
HomeController.cs | Schools.cshtml | Upload.cshtml | ResourceDetail.cshtml | Dashboard.cshtml | Search.cshtml | LessonsLearned.cshtml | Program.cs | Discussions.csht
LearnLink > Controllers > HomeController.cs > { } LearnLink.Controllers > HomeController > ImportGoogleBooks(string, string) : Task<IActionResult>
  18  ik.Controllers
  20  omeController : Controller
1594  nc Task<IActionResult> ImportGoogleBooks(string subject, string grade)
1651          if (!googleSuccess)
1652          {
1653              var openLibUrl = $"https://openlibrary.org/search.json?q={query}&limit=10&has_fulltext=true&ebook_access=public&fields=key,title,author_name,first_sentence,sub
1654              var olResponse = await client.GetAsync(openLibUrl);
1655
1656              if (olResponse.IsSuccessStatusCode)
1657              {
1658                  json = await olResponse.Content.ReadAsStringAsync();
1659                  source = "openlibrary";
1660              }
1661              else
1662              {
1663                  TempData["ErrorMessage"] = "Unable to fetch books from any source. Please try again later.";
1664                  return RedirectToAction("Settings");
1665              }
1666          }
1667
1668          // Cache successful response for 24 hours
1669          if (!string.IsNullOrEmpty(json))
1670              _cache.Set(cacheKey, json, TimeSpan.FromHours(24));
1671      }
1672      finally
1673      {
1674          _googleBooksSemaphore.Release();
1675      }
1676
1677
1678  (string.IsNullOrEmpty(json))
1679
1680      TempData["ErrorMessage"] = "No response received from book APIs.";
1681      return RedirectToAction("Settings");
1682
1683
1684  Detect source from cached data if needed
```



```
HomeController.cs | Schools.cshtml | Upload.cshtml | ResourceDetail.cshtml | Dashboard.cshtml | Search.cshtml | LessonsLearned.cshtml | Program.cs | Discussions.csht
LearnLink > Controllers > HomeController.cs > { } LearnLink.Controllers > HomeController > ImportGoogleBooks(string, string) : Task<IActionResult>
  18  namespace LearnLink.Controllers
  20      public class HomeController : Controller
1594          public async Task<IActionResult> ImportGoogleBooks(string subject, string grade)
1698                  {
1699                      importCount = await ImportFromGoogleBooksJson(json, subject, grade, currentUser);
1700                  }
1701                  else
1702                  {
1703                      importCount = await ImportFromOpenLibraryJson(json, subject, grade, currentUser);
1704                  }
1705
1706                  if (importCount > 0)
1707                  {
1708                      await _context.SaveChangesAsync();
1709                      var sourceLabel = source == "google" ? "Google Books" : "Open Library";
1710                      TempData["SuccessMessage"] = $"Successfully imported {importCount} books for {subject} ({grade}) from {sourceLabel}!";
1711                  }
1712                  else
1713                  {
1714                      TempData["WarningMessage"] = "No new books were imported. They might already exist or no results were found for this query.";
1715                  }
1716              }
1717              catch (TaskCanceledException)
1718              {
1719                  TempData["ErrorMessage"] = "The request timed out. Please try again.";
1720              }
1721              catch (Exception ex)
1722              {
1723                  var innerMsg = ex.InnerException != null ? ex.InnerException.Message : "";
1724                  TempData["ErrorMessage"] = $"An error occurred while importing books: {ex.Message} {innerMsg}";
1725              }
1726
1727              return RedirectToAction("Settings");
1728          }
1729
                  1 reference
1730          private async Task<int> ImportFromGoogleBooksJson(string json, string subject, string grade, ApplicationUser currentUser)
```

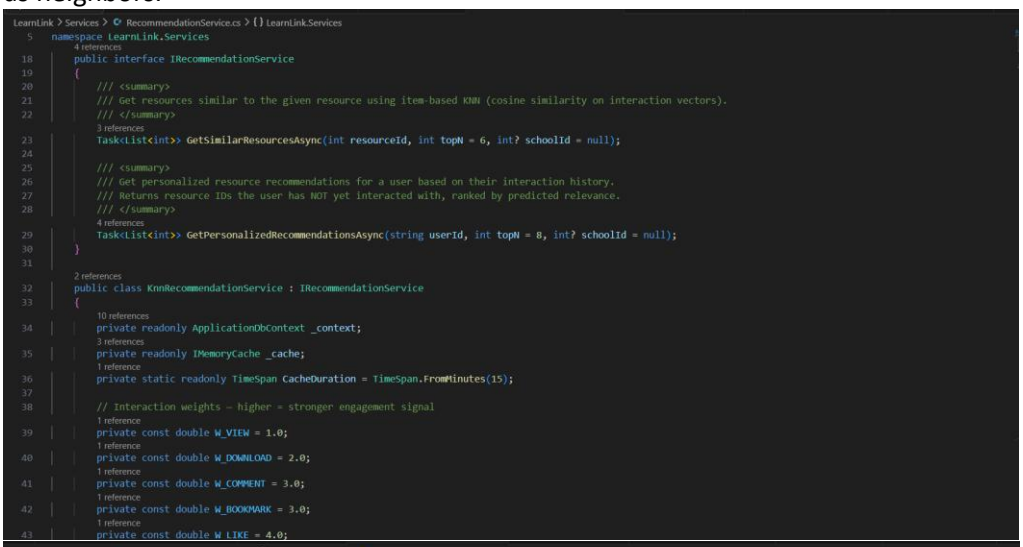


```
HomeController.cs | Schools.cshtml | Upload.cshtml | ResourceDetail.cshtml | Dashboard.cshtml | Search.cshtml | LessonsLearned.cshtml | Program.cs | Discussions.csht
LearnLink > Controllers > HomeController.cs > { } LearnLink.Controllers > HomeController
  18  namespace LearnLink.Controllers
  20      public class HomeController : Controller
              1 reference
1871          private async Task<int> ImportFromOpenLibraryJson(string json, string subject, string grade, ApplicationUser currentUser)
1872          {
1873              var data = System.Text.Json.JsonDocument.Parse(json);
1874              if (!data.RootElement.TryGetProperty("docs", out var docs))
1875                  return 0;
1876
1877              int count = 0;
1878              foreach (var doc in docs.EnumerateArray())
1879              {
1880                  if (!IsOpenLibraryFreelyAccessible(doc))
1881                      continue;
1882
1883                  var title = doc.TryGetProperty("title", out var t) ? t.GetString() : "Unknown Title";
1884                  if (title != null && title.Length > 95) title = title.Substring(0, 95) + "...";
1885
1886                  if (await _context.Resources.AnyAsync(r => r.Title == title && r.ResourceType == "Book" && r.SchoolId == currentUser.SchoolId))
1887                      continue;
1888
1889                  // Build description from available fields
1890                  var desc = "";
1891                  if (doc.TryGetProperty("author_name", out var authors) && authors.GetArrayLength() > 0)
1892                  {
1893                      var authorList = new List<string>();
1894                      foreach (var a in authors.EnumerateArray())
1895                          authorList.Add(a.GetString() ?? "");
1896                      desc = $"By {string.Join(", ", authorList)}. ";
1897                  }
1898                  if (doc.TryGetProperty("first_sentence", out var sentences) && sentences.ValueKind == System.Text.Json.JsonValueKind.Array && sentences.GetArrayLength()
```

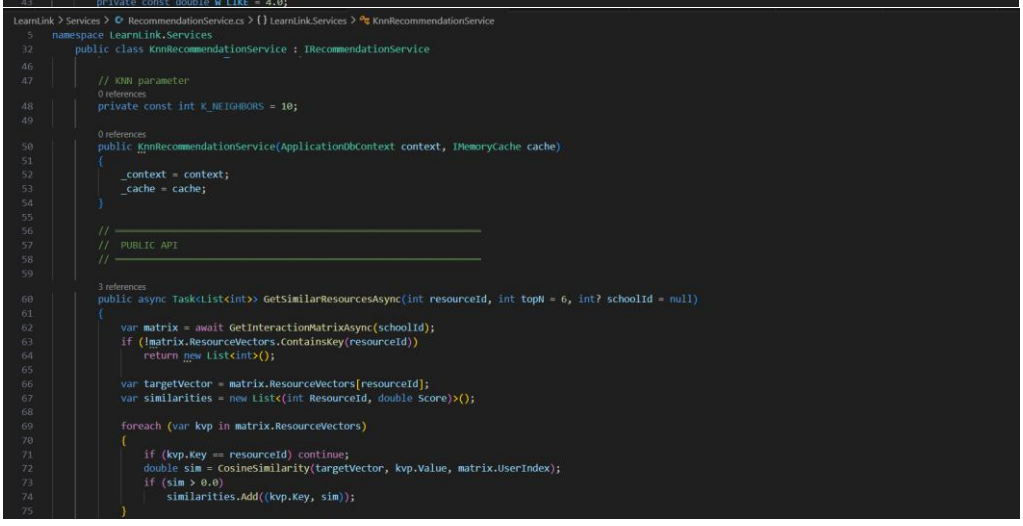


6.KNN (K-Nearest Neighbors) / Algorithm

KNN means “K-Nearest Neighbors”. Each resource is represented by user interaction behavior. If two resources are used by similar users in similar ways, they are treated as neighbors.



```
LearnLink > Services > RecommendationService.cs > {} LearnLink.Services
5   namespace LearnLink.Services
        4 references
18      public interface IRecommendationService
19      {
20          /// <summary>
21          /// Get resources similar to the given resource using item-based KNN (cosine similarity on interaction vectors).
22          /// </summary>
            3 references
23          Task<List<int>> GetSimilarResourcesAsync(int resourceId, int topN = 6, int? schoolId = null);
24
25          /// <summary>
26          /// Get personalized resource recommendations for a user based on their interaction history.
27          /// Returns resource IDs the user has NOT yet interacted with, ranked by predicted relevance.
28          /// </summary>
            4 references
29          Task<List<int>> GetPersonalizedRecommendationsAsync(string userId, int topN = 8, int? schoolId = null);
30      }
31
        2 references
32      public class KnnRecommendationService : IRecommendationService
33      {
            10 references
34          private readonly ApplicationDbContext _context;
            3 references
35          private readonly IMemoryCache _cache;
            1 reference
36          private static readonly TimeSpan CacheDuration = TimeSpan.FromMinutes(15);
37
38          // Interaction weights — higher = stronger engagement signal
            1 reference
39          private const double W_VIEW = 1.0;
            1 reference
40          private const double W_DOWNLOAD = 2.0;
            1 reference
41          private const double W_COMMENT = 3.0;
            1 reference
42          private const double W_BOOKMARK = 3.0;
            1 reference
43          private const double W_LIKE = 4.0;
```

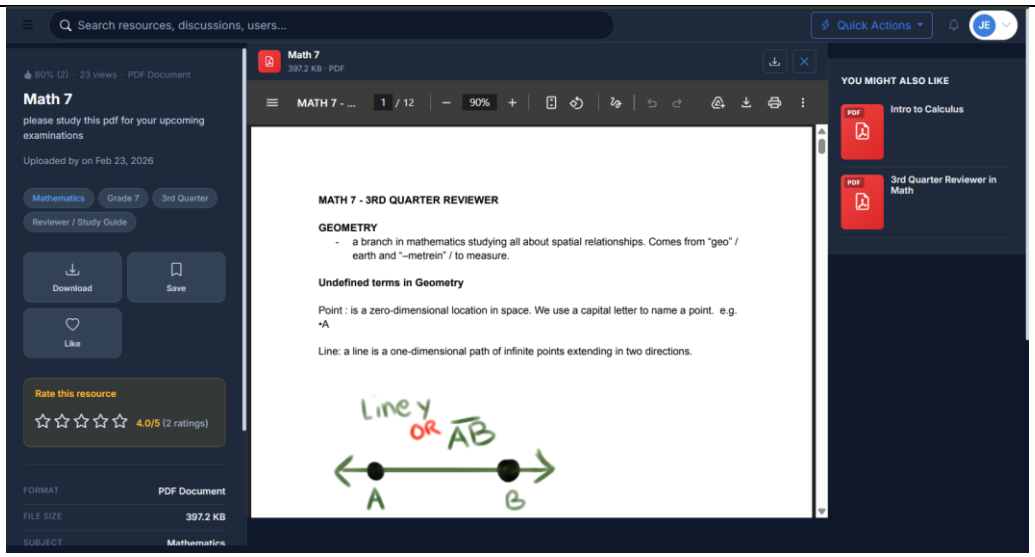


```
LearnLink > Services > RecommendationService.cs > {} LearnLink.Services > KnnRecommendationService
5   namespace LearnLink.Services
32      public class KnnRecommendationService : IRecommendationService
46
47          // KNN parameter
            0 references
48          private const int K_NEIGHBORS = 10;
49
            0 references
50          public KnnRecommendationService(ApplicationDbContext context, IMemoryCache cache)
51          {
52              _context = context;
53              _cache = cache;
54          }
55
56          // ═══════════════════════════════════════════════════════
57          //  PUBLIC API
58          // ═══════════════════════════════════════════════════════
59
            3 references
60          public async Task<List<int>> GetSimilarResourcesAsync(int resourceId, int topN = 6, int? schoolId = null)
61          {
62              var matrix = await GetInteractionMatrixAsync(schoolId);
63              if (!matrix.ResourceVectors.ContainsKey(resourceId))
64                  return new List<int>();
65
66              var targetVector = matrix.ResourceVectors[resourceId];
67              var similarities = new List<(int ResourceId, double Score)>();
68
69              foreach (var kvp in matrix.ResourceVectors)
70              {
71                  if (kvp.Key == resourceId) continue;
72                  double sim = CosineSimilarity(targetVector, kvp.Value, matrix.UserIndex);
73                  if (sim > 0.0)
74                      similarities.Add((kvp.Key, sim));
75              }
```

It analyzes interactions such as views, downloads, comments, bookmarks, likes, completions, and ratings, assigns weights to each action, then compares resources.

Like for example the user reads too much Mathematics content the algorithm will suggest the user with more Mathematics resources.

Notice in the “You Might Also Like”, all of them is Math Related.



This is where the signals are being weighted by. The higher the signal or decimal the algorithm will follow or learn your behaviour and will suggest the best possible way to make the recommendations more personalized.

```
private const double W_VIEW = 1.0;
private const double W_DOWNLOAD = 2.0;
private const double W_COMMENT = 3.0;
private const double W_BOOKMARK = 3.0;
private const double W_LIKE = 4.0;
private const double W_COMPLETE = 4.0;
private const double W_RATING = 5.0;
```

How it's done in backend (Each of them will get the decimal value)

```
211
212             // Views (from activity logs)
213             var views = await _context.UserActivityLogs
214                 .Where(a => a.ActivityType == "View" && a.ResourceId != null)
215                 .Select(a => new { a.UserId, ResourceId = a.ResourceId!.Value })
216                 .ToListAsync();
217             foreach (var v in views)
218                 AddScore(v.ResourceId, v.UserId, W_VIEW);
219
220             // Downloads
221             var downloads = await _context.UserActivityLogs
222                 .Where(a => a.ActivityType == "Download" && a.ResourceId != null)
223                 .Select(a => new { a.UserId, ResourceId = a.ResourceId!.Value })
224                 .ToListAsync();
225             foreach (var d in downloads)
226                 AddScore(d.ResourceId, d.UserId, W_DOWNLOAD);
227
228             // Comments
229             var comments = await _context.ResourceComments
230                 .Where(c => c.ParentCommentId == null) // only top-level
231                 .Select(c => new { c.UserId, c.ResourceId })
232                 .ToListAsync();
233             foreach (var c in comments)
234                 AddScore(c.ResourceId, c.UserId, W_COMMENT);
235
236             // Bookmarks / Saves
237             var bookmarks = await _context.ReadingHistories
238                 .Where(h => h.IsBookmarked)
239                 .Select(h => new { h.UserId, h.ResourceId })
240                 .ToListAsync();
241             foreach (var b in bookmarks)
242                 AddScore(b.ResourceId, b.UserId, W_BOOKMARK);
243
```

```
243
244             // Completions
245             var completions = await _context.ReadingHistories
246                 .Where(h => h.ProgressStatus == "Completed")
247                 .Select(h => new { h.UserId, h.ResourceId })
248                 .ToListAsync();
249             foreach (var c in completions)
250                 AddScore(c.ResourceId, c.UserId, W_COMPLETE);
251
252             // Likes (TargetType == "Resource")
253             var likes = await _context.Likes
254                 .Where(l => l.TargetType == "Resource")
255                 .Select(l => new { l.UserId, l.TargetId })
256                 .ToListAsync();
257             foreach (var l in likes)
258                 AddScore(l.TargetId, l.UserId, W_LIKE);
259
260             // Ratings (TargetType == "ResourceRating")
261             var ratings = await _context.Likes
262                 .Where(l => l.TargetType == "ResourceRating")
263                 .Select(l => new { l.UserId, l.TargetId })
264                 .ToListAsync();
265             foreach (var r in ratings)
266                 AddScore(r.TargetId, r.UserId, W_RATING);
267
268             return new InteractionMatrix
269             {
270                 UserIndex = userIndex,
271                 ResourceVectors = resourceVectors
272             };
273         }
```

SECURITY FEATURES

1.ASP.NET Identity & Password Hashing

Secures user login, password handling, and authentication cookies. This is the foundation of account security in the system.

```
// Configure EF Core + Identity with Roles
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection"),
        sqlOptions => sqlOptions.EnableRetryOnFailure(maxRetryCount: 5, maxRetryDelay: TimeSpan.FromSeconds(10), errorNumbersToAdd: null)));

builder.Services.AddIdentity<ApplicationUser, IdentityRole>(options =>
{
    options.SignIn.RequireConfirmedAccount = false;
    options.Password.RequireDigit = false;
    options.Password.RequireLowercase = false;
    options.Password.RequireUppercase = false;
    options.Password.RequireNonAlphanumeric = false;
    options.Password.RequiredLength = 6;
})
.AddEntityFrameworkStores<ApplicationDbContext>()
.AddDefaultTokenProviders()
.AddClaimsPrincipalFactory<SchoolClaimsPrincipalFactory>();

builder.Services.ConfigureApplicationCookie(options =>
{
    options.LoginPath = "/Home/Login";
    options.AccessDeniedPath = "/Home/AccessDenied";
    options.ExpireTimeSpan = TimeSpan.FromDays(30);
    options.Cookie.MaxAge = options.ExpireTimeSpan;
    options.SlidingExpiration = true;
});
```

It ensures accounts are authenticated properly, passwords are managed by ASP.NET Identity, and unauthorized users are redirected away from protected pages.

2. Role-Based Authorization (Authorize Attributes)

It prevents ordinary users from performing administrative actions. Only users with the correct role can access sensitive management functions.

```
592         // ===== Approve / Reject (SuperAdmin, Manager) =====
593
594         [Authorize(Roles = "SuperAdmin,Manager")]
            0 references
595         public async Task<IActionResult> ApproveResource(int id)
596         {
597             await AutoRejectExpiredPendingResourcesAsync();
598
599             var resource = await _context.Resources.FindAsync(id);
600             if (resource != null)
601             {
602                 resource.Status = "Published";
603                 resource.RejectionReason = null; // Clear any prior reason
604                 await _context.SaveChangesAsync();
605
606                 var currentUser = await GetCurrentUserAsync();
607                 if (currentUser != null)
608                     await LogActivity(currentUser.Id, "Approve", resource.Title, resource.ResourceId);
609
610                 // Create notification for the resource uploader
611                 _context.Notifications.Add(new Notification
612                 {
613                     UserId = resource.UserId,
614                     Title = "Resource Approved",
615                     Message = $"Your resource \"{resource.Title}\" has been approved and is now published!",
616                     Type = "Approved",
617                     Icon = "bi-check-circle-fill",
618                     IconBg = "#d1fae5",
619                     ResourceId = resource.ResourceId,
620                     Link = $"/Home/ResourceDetail/{resource.ResourceId}"
621                 });
622                 await _context.SaveChangesAsync();
623
624                 TempData["SuccessMessage"] = $"Resource \"{resource.Title}\" has been approved and published.";
```

This was implemented when the contributor is uploading the resources and it shall go to process to check by the Admin/Manager to verify.

```
629
630         [Authorize(Roles = "SuperAdmin,Manager")]
631         [HttpPost]
            0 references
632         public async Task<IActionResult> RejectResource(int id, string? reason)
633         {
634             await AutoRejectExpiredPendingResourcesAsync();
635
636             var resource = await _context.Resources.FindAsync(id);
637             if (resource != null)
638             {
639                 resource.Status = "Rejected";
640                 resource.RejectionReason = reason;
641                 await _context.SaveChangesAsync();
642
643                 var currentUser = await GetCurrentUserAsync();
644                 if (currentUser != null)
645                     await LogActivity(currentUser.Id, "Reject", resource.Title, resource.ResourceId);
646
647                 // Create notification for the resource uploader
648                 var reasonText = !string.IsNullOrWhiteSpace(reason)
649                     ? $" Reason: {reason}"
650                     : "";
651                 _context.Notifications.Add(new Notification
652                 {
653                     UserId = resource.UserId,
654                     Title = "Resource Rejected",
655                     Message = $"Your resource \"{resource.Title}\" has been rejected.{reasonText}",
656                     Type = "Rejected",
657                     Icon = "bi-x-circle-fill",
658                     IconBg = "#fee2e2",
659                     ResourceId = resource.ResourceId,
660                     Link = $"/Home/ResourceDetail/{resource.ResourceId}"
661                 });
```

3. Multi-Tenancy Isolation (SchoolId Filtering)

Every query filters by SchoolId; schools cannot see each other's data. This protects tenant boundaries. Users are tied to a school, and the application uses school context to keep records isolated.

```
public int? SchoolId { get; set; }

private int? GetEffectiveSchoolId()
    => _schoolContext.CurrentSchoolId;

return targetUser.SchoolId == _schoolContext.CurrentSchoolId;

var school = await _context.Schools.FirstOrDefaultAsync(s => s.SchoolId == schoolId && s.IsActive);

builder.Services.AddScoped<ISchoolContext, HttpSchoolContext>();
```

4. Resource Access Control and Permission Grants
Controls who can open or download a resource, or can access it fully.

Each resources can be access by level for example in (Public : even the non-registered users can see the resources, Registered Users: Only the users that has an account can access, Restricted : The Uploader can select which users will access the resources.

```
235
236         // ---- Policy Settings (per-resource) ----
237         [StringLength(20)]
            23 references
238         public string AccessLevel { get; set; } = "Registered";  // Public, Registered, Restricted
239
240         [StringLength(30)]
            11 references
241         public string AccessDuration { get; set; } = "Unlimited";  // Unlimited, 7 Days, 30 Days, 90 Days, Until End of School Year
242
            10 references
243         public bool AllowDownloads { get; set; } = true;
            13 references
244         public bool EnableVersionHistory { get; set; } = true;
            9 references
245         public bool AllowComments { get; set; } = true;
            4 references
246         public bool AllowRatings { get; set; } = true;
            2 references
247         public bool ModerateComments { get; set; } = false;
            3 references
248         public bool RequireVersionNotes { get; set; } = false;
249
            14 references
250         public DateTime? AccessExpiresAt { get; set; }  // Custom expiry date for access duration
251
252         // Navigation for restricted access grants
            1 reference
253         public ICollection<ResourceAccessGrant> AccessGrants { get; set; } = new List<ResourceAccessGrant>();
254     }
255
```

How it’s being implemented:

```
if (resource.AccessLevel == "Restricted")
{
    if (currentUser == null)
    {
        return RedirectToAction("Login");
    }
    var hasGrant = await _context.ResourceAccessGrants
        .AnyAsync(g => g.ResourceId == id && g.UserId == currentUser.Id);
    if (!hasGrant)
    {
        TempData["ErrorMessage"] = "You do not have permission to access this resource. It is restricted to selected
        return RedirectToAction("Repository");
    }
}
else if (resource.AccessLevel == "Registered")
{
    if (currentUser == null)
    {
        return RedirectToAction("Login");
    }
}
```

	5.Password Reset via Secure Email Token
	
	`var token = await _userManager.GeneratePasswordResetTokenAsync(user);`
	`var encodedToken = WebEncoders.Base64UrlEncode(Encoding.UTF8.GetBytes(token));`
	`var resetUrl = Url.Action("ResetPassword", "Home",`
	`new { email = user.Email, token = encodedToken }, protocol: Request.Scheme);`
	
	`await _emailService.SendAsync(user.Email!, "Reset your LearnLink password", body);`
	
	Secure password reset flow using time-limited, URL-safe tokens sent via email. Prevents account takeover without email access.

Date Submitted:	March, 12, 2026
Teacher's Feedback	_____ _____ _____ _____ _____ _____ _____ _____ Student's Signature (after feedbacking) Teacher's Signature